Danmarks Tekniske Universitet



Programmeringsprojekt (30010)
Starfighter

Forfattere

Group 08
August A. A. Feldthusen - s246569
Carl Johann T. B. Falkenhayn - s233318
Jacob J. Olsen - s246048
Simon S. Hansen - s246226

26. januar 2025

Indholdsfortegnelse

1 Introduktion **2**

2 Kravspecifikationer **3**
2.1 Minimumskrav . . . 3

3 Analyse **5**
3.1 Spiltilstande . . . 5
3.1.1 Hovedmenu . . . 5
3.1.2 Spilmenu . . . 5
3.1.3 Hjælpemenu . . . 6
3.1.4 Boss-key/Pauseskærm . . . 6
3.2 Skud . . . 6
3.2.1 Skudstruktur . . . 6
3.2.2 Skudvinkler . . . 7
3.2.3 Håndtering af flere skud . . . 7
3.2.4 Kollisionsregistrering . . . 7
3.2.5 Tyngdekraftseffekt . . . 8
3.2.6 Skudtyper . . . 8
3.3 Fjender . . . 8
3.3.1 Fjendefrembringelse . . . 8
3.3.2 Fjendebevægelse . . . 9
3.3.3 Stigende alienhastighed ved timer . . . 9
3.3.4 Forsinkelser . . . 9
3.4 LCD . . . 10
3.4.1 LCD-skærmens opbygning . . . 10
3.4.2 Liv . . . 10
3.4.3 Tekst . . . 10
3.4.4 Skud tilstand . . . 10
3.4.5 Pointsystem . . . 11
3.4.6 RGB-LED . . . 11

4 Design **12**
4.1 Kvalitetssikring . . . 12
4.2 Hardware- og tidsmæssige begrænsninger . . . 12
4.3 Grafisk design . . . 12
4.3.1 Grafikkens bevægelse . . . 13
4.4 System abstraktionslag . . . 14
4.4.1 Application layer . . . 14
4.4.2 Application interface layer . . . 15
4.4.3 Hardware abstraction layer . . . 15

5 Projektet **16**

5.1 GitHub . . . 16
5.1.1 Tidsstyring og prioritering . . . 16
5.2 Projektverificering . . . 16

6 Brugermanual **17**
6.1 Putty konfiguration . . . 17
6.2 Spilstyring . . . 17
6.3 **Pinopsætning** . . . 17

7 Diskussion **18**
7.1 Projektets styrker og svagheder . . . 18
7.1.1 Styrker . . . 18
7.1.2 Svagheder . . . 18
7.2 Joystick-implementering . . . 18
7.3 Krav . . . 18

8 Konklusion **22**
8.1 Hvem gjorde hvad i STM32-IDE . . . 22
8.2 Hvem gjorde hvad i rapporten . . . 22

Liste over figurer **I**

Nomenklatur **II**

Referencer **II**

9 Appendiks A **III**

10 Appendiks B - Source Code **1**

11 Appendix C - Exercise journal **85**
11.1 Kontrol over skærmen (Exercise. 1) . . . 85
11.2 Vinduer med titler(Exercise.2) . . . 86
11.3 Trigonometri med Look-Up Tables (LUT)(Exercise.3) . . . 87
11.3.1 Vektorfunktioner . . . 88
11.4 Ball in window (Exercise. 4) . . . 88
11.5 GPIO Joystick(Exercise.5) . . . 90

Resumé

Dette projekt omhandler udvikling og implementering af et computerspil, der kører på en STM32-mikrocontroller. Spillet er skrevet i programmeringssproget C og er inspireret af det klassiske arkadespil "Space Invaders"med fokus på både funktionalitet og en optimal brugeroplevelse. Projektet benytter desuden en systemarkitektur, som gør mange af spillets komponenter, moduler og funktioner genanvendelige.

Spillet indeholder funktioner som meteorer med tyngdekraft, otte skudvinkler, power-ups samt point og liv vist på en LCD-skærm og kører hurtigt og effektivt. Spillet styres ved hjælp af det eksterne joystick 30010, hvor man både kan bevæge rumskibet, affyre skud og navigere rundt i menuen.

Projektet har givet værdifuld erfaring inden for C-programmering, samarbejde i udviklingsprocessen og softwarearkitektur. Denne rapport dokumenterer design, implementering, udfordringer og resultater og konkluderer, at spillet opfylder kravspecifikationerne, og leverer en unik og god brugeroplevelse.

Abstract

This project focuses on the development and implementation of a computer game that runs on a STM32-microcontroller. The game is written in the programming language C and is inspired by the classic arcade game *Space Invaders*. It focuses on both functionality and achieving an optimal user experience. Additionally, the project utilizes a system architecture that ensures most game components, modules and functions are reusable. The game features meteors with gravitational pull, eight different shooting angles, power-ups and an LCD-display showing lives and points, and operates smoothly and efficiently. The game is controlled using an external joystick 30010, which allows the player to maneuver the spaceship, shoot, and navigate the menu.

The project has provided valuable experience in C programming, collaboration during the development process, and software architecture. This report documents the designs, implementations, challenges, and results concluding that the game meets the required specifications and delivers a unique and excellent user experience.

1 Introduktion

Formålet med projektet er at designe og implementere et interaktivt arkadespil ved hjælp af en mikrocontroller baseret på en ARM Cortex-M4 CPU og STM32CubeIDE. Projektet fokuserer ikke kun på at skabe et funktionelt og engagerende spil, men også på at overholde industristandarder for spiludvikling. Derudover analyserer projektet et mellemstort programmeringsproblem og håndterer ressourcer og softwarelag på en struktureret og modulær måde.

Spillet, inspireret af det klassiske arkadespil "Space Invaders", indeholder funktioner som meteorer med simuleret tyngdekraft, otte skudvinkler, power-ups vist på en PC-terminal, samtidigt med at point- og livtracking vist på en LCD-skærm. Disse funktioner styres ved hjælp af timere, interrupts og et eksternt joystick. Implementeringens arkitektur består af et applikationslag, applikationsgrænsefladelag og hardwareabstraktionslag, hvilket sikrer god genbrugelighed, så man nemt kan tilføje eller ændre eksisterende funktioner.

Spillet opfylder de obligatoriske krav i projektbeskrivelsen ved at inkludere følgende funktioner: flere kugler, pointtæller, information via RGB-LED, information på LCD-skærmen på en STM32-mikrocontroller, og spillet styres via et eksternt 30010 joystick, der forbindes til mikrocontrolleren. Disse krav er hovedsageligt valgt, ud fra et formål om at genskabe det meget populære arkadespil "Space Invaders".

Ved at følge programmeringsprincipper, såsom minimal brug af globale variabler og modulær kode, sikrer vi præcision og ydeevne, samtidigt med at koden er fokuseret og overskuelig.

Udover at lave et sjovt og funktionelt spil handler projektet også om at lære, hvordan man arbejder med indlejrede systemer og C-programmering. Projektet har også til formål at lære, hvordan man får joystick, RGB-LED'erne og LCD-skærmen til at arbejde sammen. Projektet handler ikke kun om at skrive C-kode – det handler også om at få funktionerne til at fungere på en mikrocontroller med begrænset RAM og ROM.

Denne rapport dokumenterer designovervejelser, implementering og testprocessen for spillet. Rapporten giver indsigt i udfordringer, der blev mødt, og løsninger, der blev implementeret, samt de overordnede resultater af projektet.

2 Kravspecifikationer

2.1 Minimumskrav

Spillet skal kunne køre på en STM32-mikrocontroller med begrænset hardware, og systemet skal være robust over for spillerens inputs.
Spilleren skal kunne navigere skibet op, ned, til højre og venstre ved hjælp af joysticket samt affyre skud ved at trykke på en knap. Desuden skal spilleren have adgang til en "Boss Key", som skal kunne aktiveres nemt og hurtigt via en af knapperne på joysticket. Spillet skal understøtte flere skudtyper og inkludere tre skydefunktioner: "Laser", "Standard"og "Shotgun", som kan aktiveres af power-ups. Skuddene skal påvirkes af en tilfældig vinkel (random delta). Der skal implementeres kollision mellem skud og fjender, som registreres og fjerner fjenden. Spilleren skal starte med et fast antal liv, som reduceres ved kollision med fjender eller meteorer, eller hvis fjender ikke dræbes, før de rammer skærmens bagside. På den måde er spillerens mål at skyde og eliminere fjenderne, før de forsvinder.
Når spilleren mister alle liv, skal spillet afsluttes, og scoren skal vises på LCD-skærmen. Under spillet viser LCD-skærmen spillerens liv, point og den valgte skudtype, hvilket opdateres løbende under spillet.
Spillet skal visualiseres i terminalen, hvor skib, skud, fjender og meteorer vises. Teksten "Game Over"skal vises i terminalen, når spillet slutter. RGB-LED'en skal vise antallet af liv tilbage: Grønt lys indikerer tre liv, blåt lys indikerer to liv, og rødt lys indikerer ét liv. LED-skærmen skal slukke, når alle liv er tabt.
Koden skal skrives modulært, så nye funktioner som flere fjendetyper eller power-ups nemt kan tilføjes.

Ekstra funktioner

Der skal tilføjes flere niveauer, der løbende stiger i sværhedsgrad under spillet. I de forskellige niveauer kan spillets sværhedsgrad øges ved at øge fjendernes og meteorernes hastighed eller deres antal. Man kan også gøre fjenderne mindre og meteorerne større, så det bliver udfordrende at ramme fjender, samtidig med at man undviger meteorer. For at undgå at overbelaste mikrocontrolleren er det dog mest hensigtsmæssigt at øge hastigheden på fjender og meteorer fremfor at tilføje flere.

Funktionerne under "**Minimumskrav**"er prioriterede, fordi de er afgørende for spillets kernefunktionalitet og nødvendige for at gøre det spilbart. Disse funktioner danner fundamentet for spillet og er valgt ud fra deres relevans i forhold til spillets mål og projektbeskrivelsen. Det er vigtigt, at spillet opfylder de grundlæggende krav og er stabilt i sin basisform.

"**Ekstra funktioner**"inkluderer funktioner, der kan forbedre spiloplevelsen og gøre spillet mere interessant og varieret, men som ikke er nødvendige for, at spillet kan fungere. Disse funktioner kan implementeres løbende, når det essentielle er på plads, og hvis der er tilstrækkelige ressourcer og tid til rådighed.

- Systemet skal kunne klare forkerte inputs og fejl i joystick og knapper.
- Spilleren skal kunne styre skibet op, ned, til højre og venstre med joysticket.

- Spilleren skal kunne affyre skud ved at trykke på en knap.
- En "Boss Key"skal kunne aktiveres nemt via en knap på joysticket.
- Spillet skal understøtte flere skud-typer "Laser", "Normal", "Shotgun", aktiveret via power-ups.
- Skud skal påvirkes af en tilfældig deltavinkel.
- Kollision mellem skud og fjender skal registreres og fjenderne fjernes.
- Spilleren starter med tre liv, som reduceres ved:
  - Kollision med fjender eller meteorer.
  - Fjender, der undslipper og når skærmens bagside.
- Spillet slutter, når spilleren mister alle liv, og scoren vises på LCD-skærmen.
- LCD-skærmen viser spillerens liv, point og valgte skud-type, opdateres løbende.
- Joysticket skal bruges til skibets bevægelser og menu-navigation.
- Koden skal skrives modulært for nemt at kunne tilføje nye funktioner, fjendetyper eller power-ups.
- Nye niveauer med stigende sværhedsgrad skal implementeres.

3 Analyse

Udviklingen og implementeringen af diverse funktioner er udført i overensstemmelse med blockdiagrammet, der starter med de mest grundlæggende funktioner og udvides derfra.

3.1 Spiltilstande

Vores main.c-fil indeholder tre separate spilstadier (1). Disse er inddelt i tre menuer, som findes i menufolderen under Src-mappen i StarFighter. De tre menuer, der har deres egen fil, står for håndteringen af diverse spilrelevante filer udover initialiseringsfilerne i main.c.
Variablen *nav* gives som en pointer i hver spilmenu. Denne variabel opdateres i hver spilmenu afhængigt af spillerens input.

```
uint8_t nav = 0;
while (1) {
    if (nav == 0) {
        main_menu(&nav);
    }
    if (nav == 1) {
        help_menu(&nav);
    }
    if (nav == 2) {
        game_menu();
        game_over(&nav);
    }
}
```

Liste 1: Main-løkke kode

3.1.1 Hovedmenu

Filen *main_ menu.c* indeholder to funktioner. Den ene håndterer al logikken bag hovedmenuen, og den anden bruges til at tegne vinduerne på skærmen. Når spillet initialiseres, sættes *nav* til 0. Dette fører spilleren ind i hovedmenuen.
Udover at initialisere hovedmenuens grafik indeholder filen en while-løkke og en integer, *z*. Variablen *z* opdateres i while-løkken afhængigt af funktionen read_joystick, som aflæser positionen af 30010-joysticket i realtid. Variablen *z* styrer også de visuelle input, der hjælper spilleren med at indikere, hvilken menu der vælges.
Derudover indeholder while-løkken to if-sætninger, der tjekker værdien af *z* og inputtet fra read_button, som er en funktion fra joystick_reader.c. Kombinationen af disse to afgør, hvad *nav* sættes til, og dermed hvilken menu spilleren føres ind i.

3.1.2 Spilmenu

Filen *game_ menu.c* indeholder to funktioner: Funktionen *game_ menu()* indeholder al initialisering, der er relevant for spilstyring, grafik og andre elementer. Derudover indeholder

den en while-løkke, der opdaterer alt fra kollisionsmekanik til LCD-skærmen. Denne while-løkke brydes først, når spillerens liv når nul. Herefter initialiseres funktionen *game_over()*, der bruger pointeren *nav*.
Funktionen game_over har sin egen while-løkke, der tjekker for input fra joysticket og sender spilleren tilbage til hovedmenuen.

3.1.3 Hjælpemenu

Filen *help_menu.c* indeholder en af de tre menuskærme, der bringer spilleren rundt i spillet. Den indeholder en funktion, der bruger variablen *nav* til at tegne skærmens grafik samt en while-løkke, der tjekker, om knap ét eller to trykkes.
Variablen *nav* angiver en værdi: 0, 1 eller 2, som angiver, hvilken af de tre menuer den skal gå ind i. Herefter bliver man sendt tilbage til hovedmenuen.

3.1.4 Boss-key/Pauseskærm

Filen boss_key.c indeholder opsætningen af funktionen *boss_key()*. Funktionen indeholder en if-sætning, der evalueres hver gang game-while-løkken kører. Dette kan ses i game_menu.c.
Funktionen sætter den globale variabel *cs*, som er spillets timer, på pause. Dette forhindrer spillerens point i at stige, og positionerne af aliens, meteorer og skud ændres ikke. Herefter vises boss-key-skærmen og går ind i en while-løkke, hvor den afventer input fra "knap to". Dette genstarter timeren *cs* og game-løkken, så alt fortsætter fra dér, hvor man var nået til. Grafikken er udarbejdet ved hjælp af forskellige ASCII-relevante hjemmesider: [1] [2].

3.2 Skud

3.2.1 Skudstruktur

Skuddets struktur indeholder position, hastighed, hvorvidt skuddet er "aktivt" eller ej, samt om det er blevet påvirket af tyngdekraft. Disse oplysninger bruges til at håndtere og opdatere skuddets position. Hvis et skud er aktivt, opdateres skuddets position jævnligt ved at addere med hastigheden.

```
1 typedef struct {
2     uint8_t bx;
3     uint8_t by;
4     int8_t vx;
5     int8_t vy;
6     uint8_t active;
7     uint8_t gravity;
8 } bullet;
```

Liste 2: Skudarray kode

3.2.2 Skudvinkler

Rumskibet har en variabel, der registrerer den seneste retning, som rumskibet har bevæget sig i (*pos->direction*). Variablen kan have en værdi fra 1 til 8, som angives af *read_joystick()*.
Når rumskibet skyder, indstilles skuddets hastighedsvektor ud fra retningsvariablen. Hvis rumskibet for eksempel peger mod højre, sættes hastigheden langs x-aksen til 2 og langs y-aksen til 0. Hvis rumskibet peger skråt ned mod venstre, sættes hastigheden langs x-aksen til -2 og langs y-aksen til 1.
Eftersom pixlerne fylder dobbelt så meget på y-aksen som på x-aksen, er det nødvendigt at fordoble hastigheden langs x-aksen i forhold til y-aksen, så alle retninger føles lige hurtige.

3.2.3 Håndtering af flere skud

Muligheden for at affyre flere skud er implementeret ved hjælp af et array. Når spillet startes, oprettes et array, hvor størrelsen er det maksimale antal skud, der kan eksistere samtidigt. Derefter opdateres skuddenes positioner ved at iterere over arrayet og udføre de nødvendige beregninger.
For at sikre jævne opdateringer opdateres skuddene forskudt af hinanden baseret på deres indeks.

```
1 bullet bullets_array[MAX_BULLETS];
```

Liste 3: Skudarray kode

3.2.4 Kollisionsregistrering

Kollisionsregistrering udføres ved at sammenligne positioner med if-sætninger. Da objekternes positioner i spillet (rumskibet, skud, aliens, meteorer) er placeret i det øvre venstre hjørne af objektet, håndteres skudkollision med meteorer som følger:
Hvis skuddets x-værdi er større end meteorens x-værdi og mindre end meteorens x-værdi plus 20, og skuddets y-værdi er større end meteorens y-værdi og mindre end meteorens y-værdi plus 10, udføres udsagnet indeni. I dette tilfælde fjernes skuddet.

```
1  for (uint8_t i = 0; i < MAX_BULLETS; i++) {
2      for (uint8_t j = 0; j < MAX_METEORS; j++) {
3          if ((bullets_array[i].bx >= meteors_array[j].x)
4              && (bullets_array[i].bx < meteors_array[j].x + 20)
5              && (bullets_array[i].by >= meteors_array[j].y)
6              && (bullets_array[i].by < meteors_array[j].y + 10)) {
7              bullets_array[i].active = 0;
8          }
9      }
10 }
```

Liste 4: Kollisionsregistrering kode

3.2.5 Tyngdekraftseffekt

Tyngdefeltet omkring meteorerne er implementeret på en simpel, men effektiv måde. Det fungerer ved at tjekke, om et skud passerer forbi en meteor inden for en rækkevidde. Hvis det gør, afbøjes skuddet ved at ændre dets hastighedsvektor.
Selvom det ikke er fysisk realistisk, fungerer det godt i spillet, da det er let for computeren at beregne og forudsigeligt for spilleren.
Derudover indeholder skuddets struktur en variabel, der registrerer, om skuddet er blevet påvirket af tyngdekraft, så det ikke afbøjes flere gange.

```
for (uint8_t i = 0; i < MAX_BULLETS; i++) {
    if (bullets_array[i].gravity == 0) {
        for (uint8_t j = 0; j < MAX_METEORS; j++) {
            if ((bullets_array[i].bx >= meteors_array[j].x - 20) &&
                (bullets_array[i].bx <= meteors_array[j].x) &&
                (bullets_array[i].by == meteors_array[j].y + 5)) {

                bullets_array[i].vx++;
                bullets_array[i].gravity = 1;
            }
        }
    }
}
```

Liste 5: Tyngdekraftseffekt kode

3.2.6 Skudtyper

Implementeringen af nye skudtyper er meget ligetil, når skudfunktionaliteten allerede er opsat.
Vores "shotgun"er implementeret ved, at rumskibets retning drejes én gang mod venstre, idet der skydes. Derefter drejes rumskibet én gang mod højre og skyder igen, hvorefter det drejes tilbage og skyder en sidste gang. Dette sker så hurtigt, at spilleren ikke bemærker, at rumskibet drejer og skyder gentagne gange. Resultatet er, at rumskibet praktisk og visuelt skyder tre spredte skud.
Laseren er implementeret anderledes, da programmet undersøger, om der er aliens foran rumskibet, når skuddet affyres. Hvis der er, elimineres de. Derefter vises et blink af en hvid streg på skærmen for at kommunikere visuelt til spilleren, at laseren er blevet affyret.

3.3 Fjender

3.3.1 Fjendefrembringelse

Meteorer og aliens fremkaldes på stort set samme måde: Først tjekkes det, om fjende-nummer i er aktiv. Hvis den ikke er aktiv, sættes dens x-koordinat til 243, hvilket er den

maksimale x-værdi. Derefter bestemmes dens y-koordinat "tilfældigt"ved at beregne:

$$cs \text{ mod } ((\text{højden af skærmen} - 2) - \text{højden af fjenden}) + 1.$$

Dette sikrer, at fjenden ikke genereres for højt eller for lavt, samtidig med at det føles tilfældigt for spilleren. Fjenden sættes derefter til at være aktiv. Når en fjende er aktiv, bevæger den sig mod venstre, og man tager skade af at ramme den med rumskibet.

3.3.2 Fjendebevægelse

Fjender bevæges ved først at slette den bagerste række af deres grafik, hvorefter deres nye position beregnes. Hvis en fjendes nye position er uden for skærmen, sættes fjenden til at være inaktiv, og spilleren mister et liv. Hvis ikke, printes fronten af modstanderen. På denne måde bevæger fjenderne sig glat hen over skærmen ved simple operationer.

3.3.3 Stigende alienhastighed ved timer

Når runden starter, sættes alienes bevægelseshastighed til 15. Derefter stiger deres bevægelseshastighed gradvist gennem spillet for at øge sværhedsgraden. Dette implementeres ved at ændre deres opdateringsinterval. Al bevægelse i spillet fungerer via opdateringer, hvor hyppigheden af opdateringerne bestemmer bevægelsens hastighed. Derfor nedsættes alienes opdateringsinterval gradvist gennem spillet.

```
1 if (cs % 2500 == 0) {
2     alien_speed--;
3 }
```

Liste 6: Stigende alienhastighed kode

3.3.4 Forsinkelser

Der er indsatte forsinkelser flere steder i koden for at undgå, at funktioner kaldes for ofte. Dette implementeres ved at sætte en variabel (for eksempel *last_fire_time*) til værdien af *cs*, når funktionen kaldes. Næste gang funktionen så kaldes, tjekkes det, om forsinkelsen siden sidste kald er tilstrækkelig til, at funktionen kan kaldes igen.
Dette kan for eksempel ses i funktionerne *fire_bullet()* og *spawn_alien()*:

```
1 if (cs - last_fire_time < BULLET_FIRE_DELAY) {
2     return;
3 }
4 last_fire_time = cs;
```

Liste 7: Forsinkelse kode

3.4 LCD

3.4.1 LCD-skærmens opbygning

Vores Liquid-crystal display (LCD) bruges til at vise spillerens liv, point og hvilken skudtype spilleren har aktiveret. Displayet er opdelt i 512 byte, som yderligere er inddelt i 4 rækker med 128 byte hver.
Displayet fungerer ved, at en usigneret 8-bit buffer med 512 bytes hukommelse (*uint8_t lcd_buffer[512]*) modtager start-inputtet og ved hjælp af binære tal (omskrevet til hexadecimaler) angiver, hvad den givne byte skal vise. For eksempel skrives hexadecimalen *0xF7* som *1111 0111* i binært og vil blive vist som en streg, hvor "1-tallene svarer til tændte (mørke) felter på displayet. Ved at ændre hexadecimale værdier og startpositionen kan funktionen bruges til at tegne statiske figurer, som for eksempel hjerter, tekst eller omridset af den valgte skydetilstand.
For at aktivere displayet skal displayet først initialiseres ved hjælp af funktionen *lcd_init()*. Derefter bruges *memset(lcd_buffer, 0x00, 512)* til at nulstille alle 512 bytes i displayet for at undgå "gryn" på skærmen. Til sidst uploades de ønskede data til displayet ved hjælp af funktionen *lcd_push_buffer(lcd_buffer)*.

3.4.2 Liv

Liv visualiseres med tre hjerter i det øverste venstre hjørne, som opdateres i realtid, når spilleren mister liv i spillet. Funktionen *hearts()* tager en startposition som input og tegner et enkelt hjerte på displayet. Her angiver startpositionen, hvor på displayet hjertet skal placeres. Funktionen benytter en række buffere til at udfylde hjertet kolonne for kolonne. Til sidst holder funktionen *hearts_left()* styr på antallet af tilbageværende liv og opdaterer displayet med tilføjelse eller fjernelse af hjerter.

3.4.3 Tekst

For at skrive tekst som liv og point på displayet har vi oprettet funktionen *lcd_write_string*. Den tager en streng og en startposition som input, og så gennemløber while-løkke strengen karakter for karakter og fortsætter udskrivningsprocessen, så længe strengen ikke er nul. *uint8_t c = *str++* læser den enkelte karakter i strengen, gemmer den i variablen *c* og går derefter videre til næste karakter. Der trækkes 32 fra *c*, da ASCII-tabellen i *charset.c/h* giver bogstaver fra plads 32.
En for-løkke sørger for at udskrive hver karakter – i dette tilfælde bogstaver – én pixelkolonne ad gangen. Karaktererne genereres ud fra *character_data*, som er et array i *charset.c*, hvor pixelopbygningen af hver karakter er defineret pixelkolonne for pixelkolonne. Karaktererne er 5 pixelkolonner brede.

3.4.4 Skud tilstand

Funktionen *shooting_mode_box()* kommunikerer med den aktuelt aktiverede skudtype via en struktur. Ved at forbinde opsætningen af funktionen *heart()* med if-sætninger

kan displayet skifte mellem de tre viste skudtyper. Den samme opsætning bruges i *remove_shooting_mode_box()* til at fjerne gamle linjer, når skudtypen ændres.

3.4.5 Pointsystem

De point, som vises på LCD-skærmen, er knyttet til timeren *cs*. LCD-skærmen opdateres kun, mens spillet er i gang. Når spilleren dør, stopper timeren. Når spillet genstartes, nulstilles timeren (cs) til 0, hvilket også nulstiller pointene.

3.4.6 RGB-LED

LED-skærmen på STM32-mikrocontrolleren styres af variablen *health*. Når *health* er 3, lyser LED'en grønt. Ved 2 liv lyser den blåt, og når der kun er 1 liv tilbage, lyser den rødt. Dette implementeres med en funktion, der tager tre binære inputs (R, G og B) og konverterer dem til værdier, som sendes til LED-skærmens GPIO-port.

4 Design

4.1 Kvalitetssikring

Kvalitetssikring er en central del af projektet for at sikre, at spillet fungerer stabilt og opfylder alle krav. For at opnå dette har vi implementeret flere strategier til at teste systemets funktionalitet og sikre robustheden af vores kode.
Sideløbende med udviklingen af de individuelle moduler har vi testet dem hyppigt for at sikre, at de fungerer som forventet. Der opstår ofte problemer, når modulerne implementeres i en main-løkke, men disse er lettere at løse, hvis de individuelle dele allerede er verificeret.
Når vi støder på udfordringer med at finde fejl i koden, bruger vi debuggingværktøjerne, der er en del af STM32CubeIDE. Denne tilgang hjælper os med at finde og rette logiske fejl samt løse kommunikationsproblemer mellem softwarelagene.

4.2 Hardware- og tidsmæssige begrænsninger

En af de største udfordringer ved dette projekt er at arbejde inden for de begrænsninger, som STM32-mikrocontrolleren stiller. Selvom STM32-platformen er meget alsidig, udgør den stadig en væsentlig begrænsning ift. RAM og ROM. Dette betyder, at vi konstant skal overveje, hvordan vi udnytter de ressourcer, den stiller til rådighed. Spillets funktioner skal være optimeret, så spillet kører gnidningsfrit og optimalt, uden at overbelaste mikrocontrolleren. Derudover skal antallet af globale variabler være minimeret, og der sikres, at vores funktioner kun bruger de nødvendige ressourcer.
Med kun to uger til rådighed har det tidsmæssige pres gjordt det nødvendigt at balancere mellem design, implementering og test af spillet, hvilket har krævet en stram tidsplan og klare prioriteringer. De essentielle funktioner er prioriteret først – såsom joystickstyring, kollision og skudmekanik – før vi tilføjede avancerede funktioner som "power-ups"og RGB-indikatorer. Grundet den faste tidsramme, skaber prioriteringen en buffer, som i tilfælde af at tidsplanen skubbes, stadig efterlader os med et funktionelt spil.

Spillet indeholder en global varibel, hvilket er *cs*. Den er implimenteret for at holde styr på alle vores variabler. Den kaldes hver gang, der bliver lavet interupt af den indbyggede timer i mikrocontrolleren, hvilken øger dens værdi med én.

```
1 void TIM1_BRK_TIM15_IRQHandler(void) {
2     cs++;
3     TIM15->SR &= ~0x0001; // Clear interrupt bit
4 }
```

Liste 8: Timer kode

4.3 Grafisk design

Det grafisk design er lavet med ASCII-tegn, da det grundet tidligere erfaringer har vidst sig at være en nem og tilstrækkelig måde at opbygge spillet på. Grundet tekniske forskelle i

størrelse af computer/laptop skærme, blev terminalens opløsning bestemt til 243x76 pixels. Dette er med til at sikre at spillet kan køres optimalt på samtlige computerer og give den sammen bruger oplevelse, uden yderligere visuelle begrænsninger.

Dette gør det muligt at prioritere gameplay og funktionelle krav uden at introducere unødvendige kompatibilitetsproblemer forårsaget af skift i opløsningen. Derudover gør dette det muligt at "hard-code"vores grafik til præcist denne opløsning.
Generelt blev avanceret grafik nedprioriteret gennem forløbet af to årsager: Den første er fordi det er meget ensformigt og tidskrævende at kode store figurer med ASCII-tegn. Den anden skyldes problemer med opdatering af terminalen, da samtlige objekter på skærmen blev hhv. slettet og genprintet for at skabe følelsen af at de bevægede sig. Simplerer figurer og karakterer er derfor foretrukket grundet, at de er lettere at manipulerer med. Det simple udtryk passer også godt til det overordnede "Space Invaders-tema.

4.3.1 Grafikkens bevægelse

Vi afprøvede forskellige metoder til at optimere animationen af grafikken, blandt andet ved at øge Baud-raten, som påvirker print-hastigheden i terminalen. Vi startede med en Baud-rate på 9600, men opdagede hurtigt, at bevægelser i terminalen (fx et skud, der flyttede sig) var meget langsomme og forsinkede. Derfor undersøgte vi muligheden for at øge Baud-raten og opdagede, at 115.200 fungerede godt. Denne hastighed kunne håndtere mere bevægelse, så længe der kun var én bevægende del ad gangen.
Da vi begyndte at introducere større figurer og grafik, der skulle bevæge sig over baggrunden, blev det dog tydeligt, at dette ikke var tilstrækkeligt.I den konfiguration, vi brugte, anvendte vi en *clrscr*-funktion, som ryddede hele skærmen og derefter tegnede baggrunden og den opdaterede grafik. Denne tilgang var for tidskrævende og skabte forstyrrelser i den jævne overgang mellem grafikkens positioner.
For at løse dette optimerede vi baggrundsfunktionen *init_baggrund*, så den nu tager fire parametre, der repræsenterer koordinaterne for hjørnerne af en firkant. Dette gør det muligt kun at opdatere de baggrundsdele, der bliver påvirket af bevægelsen. Derudover udviklede vi en ny funktion til at rydde skærmen selektivt, så kun de relevante dele af grafikken blev fjernet. Både aliens og meteorer bevægede sig meget forudsigeligt, hvilket gjorde det oplagt kun at slette og tegne de relevante dele af dem ved opdatering af deres bevægelse.
En anden vigtig optimering, vi implementerede, var at skifte fra if-sætninger til switch-sætninger flere steder i koden, der håndterer objekters bevægelser og handlinger. Ved at anvende switch-sætninger reducerede vi kompleksiteten og forbedrede læsbarheden af koden, samtidig med at vi optimerede dens ydeevne. Dette skyldes, at switch-sætninger er mere effektive til at håndtere flere mulige cases sammenlignet med lange kæder af if-else-sætninger, især når mange scenarier skal behandles.
Disse forbedringer resulterede i en mere jævn og effektiv opdatering af baggrunden og grafikken. Selvom disse ændringer fik spillet til at køre tilfredsstillende, valgte vi at øge Baud-raten til 345.600, hvilket på vores computere var den maksimale hastighed uden tilfældige fejl. Dette resulterede i, at spillet kørte både bedre og hurtigere, hvilket markant forbedrede spiloplevelsen.

4.4 System abstraktionslag

Vi anvender en lagdelt struktur i vores projekt. Dette sikrer både en tydelig struktur og høj effektivitet i vores arbejde. Denne struktur hjælper os med at holde styr på prioriteringerne, da funktionaliteten i Hardware Abstraction Layer (HAL) skal være på plads, før elementer i Application Layer kan fungere. I det følgende beskrives de tre lag, samt hvilke filer der indgår i projektet, og hvad deres funktion er.

4.4.1 Application layer

Til vores application layer opretter vi en mappe kaldet "graphics". Her placerer vi alt spilgrafik, bortset fra de forskellige spil-menuer og boss-key, som også indeholder noget grafik.
Det første, vi udvikler i "graphics-mappen, er baggrunden. Vi starter med funktionen *big-planet*, som tegner en planet med to farver på en given position. Oprindeligt ønskede vi at bruge planeter som baggrund under spillet, men det krævede for mange ressourcer at gentegne dem, når noget bevægede sig over dem. Derfor valgte vi at bruge planeterne udelukkende i menuen *game_start*, hvor gentegning ikke var nødvendig.
Til baggrunden tilføjede vi *stars*, som placerede "*"forskellige steder på skærmen for at simulere stjerner. Selvom vi overvejede at bruge en funktion til at generere deres placeringer tilfældigt, valgte vi i sidste ende at hardkode deres positioner. For det første er det mere effektivt for mikrocontrolleren at arbejde med hardcodede positioner, og for det andet er det nemmere at genskabe baggrunden, når stjernernes placering kendes præcist. Ulempen ved denne tilgang er, at hardcodning gør funktionen mindre genanvendelig. Da vi startede med to funktioner (*big_planet* og *stars*), udviklede vi også *init_background*, som skulle kombinere disse funktioner til én samlet baggrund. I praksis endte *init_background* dog med kun at kalde *stars*-funktionen.

Enemy

Til fjender starter vi med at fastlægge grafikken. Vi vælger en simpel grafik, da den skal bevæge sig og opdateres ofte.

Funktionen *Init_enemy_state* initialiserer fjenderne ved at sætte alle elementer i både *aliens_array* og *meteors_array* til inaktive.

Når der findes en inaktiv meteor, sørger *spawn_meteor* for at placere den i højre side af skærmen og sætte den til aktiv. *Spawn_alien* fungerer lidt anderledes, da den også tjekker, hvor lang tid der er gået siden den sidste alien blev spawnet, før en ny alien skabes.

I funktionen *enemy_go* opdateres både meteorer og aliens. Meteorer genereres, flyttes mod venstre på skærmen og opdateres. For aliens sker det samme, men deres grafik gør processen en smule mere kompleks.

Powerup

I *powerup* er der kun 2 funktioner: *powerup* og drawwindow. *Drawwindow* funktionen tegner et 16x8 vindue med en bestemt farve. Vinduet bliver lavet med ASCII-karakterer. I *powerup* tegnes vinduet fra *drawwindow* og midten af dette beregnes. Funktionen tager så nogle cases alt efter hvilken powerup der skal tegnes. Her har vi 3: Laser, hjerte og shotgun.

Spaceship

Da vi udvikler grafikken til rumskibet, beslutter vi, at det skal kunne dreje i fire retninger. Derfor skaber vi fire forskellige grafikker til rumskibet. Ideelt set ville vi gerne have haft otte retninger, så det kunne dække alle bevægelsesretninger, men vi støder på begrænsninger med ASCII-tegn, da det er svært at designe et rumskib, der eksempelvis peger 45 grader opad.

Vi laver to forskellige designs: ét, der håndterer retningen opad, og ét, der peger mod højre. Disse designs inverterer vi for at dække alle fire retninger. De fire grafikker samles i funktionen *draw_spaceship*, som tegner én af de fire mulige orienteringer baseret på dens parameter.

4.4.2 Application interface layer

ANSI

I graphics-mappen findes vores ansi-fil, der giver adgang til ANSI escape-sekvenser. Disse sekvenser gør det muligt at skrive funktioner som *fgcolor*, der sætter forgrundsfarven for teksten i terminalen, *bgcolor*, der sætter baggrundsfarven, og *resetbgcolor*, der nulstiller farverne.

Ansi-filen bruger ASCII-tegn, så eksempelvis vil *fgcolor(6)* sætte forgrundsfarven til cyan.

LCD

LCD'ens funktion er at fremvise spillerens liv, point samt hvilken *bullet-type* spilleren har aktiveret. Livet er visualiseret med tre hjerter i topvenstre hjørne, som opdateres i realtid med at spilleren mister liv i spillet. Funktionen *hearts* tager en startsposition som input og printer et enkelt hjerte. Startspositionen er baseret på, hvor på LCD-skærmen vi ønsker at placere hjertet. LCD-skærmen er opdelt i 512 bytefelter, som yderligere er opdelt i 4 rækker med hhv. 128 byte felter i hver. Displayet fungerer ved at

4.4.3 Hardware abstraction layer

30010 joystick

30010-joysticket blev sat op efter *Assignment 8* i kurset 30010 på DTU [3]. Under *joystick_reader.c* i funktionen *configure_adv_joystick* kan det ses, at ADC1 bruges til at konvertere det analoge signal til et digitalt. Pin C2 og C3 bliver brugt til at modtage joystickpositionen. For at aflæse joystickets x- og y-koordinat bruges channel 8 og 9 som der bruges til vores pinopsætning[4]. De to funktioner read_adv_joystick_x og -y returnerer værdien fra joystickens to akser. Disse to funktioner bliver videre brugt til kategorisering af joystickens retning som kan ses under funktionen read_joystick Pin C10 og C12 bliver brugt til at aflæse de to knappers input. Alle fire pins bliver sat til pull-down registre der måler en spændingsforskel når de bliver trykket.

5 Projektet

5.1 GitHub

Til dette projekt brugte vi GitHub til samarbejde og fildeling. GitHub hjalp os med at organisere projektets filer, holde styr på ændringer og arbejde på forskellige dele af koden parallelt uden at risikere store konflikter. Desuden gjorde det fildeling nemmere og hjalp med at undgå, at det blev rodet.
Repositoryet var struktureret med en main-branch til stabile versioner og separate branches til hvert gruppemedlem for at holde main-branchen stabil. Brugen af GitHub og versionskontrol gjorde ikke kun projektarbejdet mere struktureret, men lærte os også vigtige principper for samarbejde og dokumentation i softwareudvikling.

5.1.1 Tidsstyring og prioritering

For at prioritere vores tid bedst muligt lavede vi både en dag-for-dag tidsplan og et GANTT-diagram for at sikre et godt overblik over vores tid. Planen blev opfattet som dynamisk og brugt som en guideline, da ændringer kunne ske dagligt. Det vigtigste udbytte af tidsplanen var at planlægge, hvordan vi skulle arbejde sammen. Det var vigtigt, at vi havde de samme prioriteringer for, hvad der var vigtigt for spillet, og sikrede, at funktionerne blev implementeret og testet i en logisk rækkefølge. Vi begyndte med at programmere spillets "need to haves"og tilføjede derefter eventuelle "nice to haves".

5.2 Projektverificering

Under udviklingen af spillet stødte vi på flere udfordringer og fejl, som krævede systematisk debugging for at sikre stabilitet og korrekt funktionalitet. En af de mest almindelige udfordringer var fejl i kommunikationen mellem lagene i vores program, som ofte skyldtes enten forkert initialisering af GPIO'er og timere eller fejl i logikken i vores kode.
For at identificere og rette logiske fejl anvendte vi forskellige debugging-teknikker, herunder breakpoints i STM32CubeIDE med variabelvisning og serielle udskrifter til terminalen. Dette gjorde det muligt at kontrollere og verificere værdierne af variabler og signaler i realtid.

6 Brugermanual

6.1 Putty konfiguration

Sæt '*Connection type*' til '*Serial*' og '*Serial line*' til den pågældende indgang MCU'en er tilsluttet til. Baudrate sættes til 345.600.

6.2 Spilstyring

Brug 30010-joysticket til at navigere rundt i menuerne, og brug B1 og B2 til at navigere ind i menuerne. Hjælp-menuen giver et kort resumé af spillet og dets udfordringer. Start-knappen initialiserer spillet, og du spawner til venstre i midten af skærmen.
LCD-skærmen aktiveres herefter, og du kan se både antallet af liv (også indikeret af LED'en) og din aktuelle score. Brug B2 til at skyde med den valgte ammunition. B1 aktiverer boss-key, som pauser din score. Et tryk på B2 bringer dig tilbage til spillet, præcis hvor du slap, før boss-key blev aktiveret.
Du mister et liv, hvis du rammer en alien, eller hvis en alien når den venstre side af skærmen. En kollision mellem dit skib og en meteor dør du af. Power-ups aktiveres automatisk ved kollision med dem.
Der findes tre power-ups, hver med deres unikke ikon: Shotgun, laser og heal. Shotgun affyrer tre skud ad gangen i hver sin retning. Laser affyrer en stråle, der kan trænge igennem både aliens og meteorer, og heal giver spilleren et ekstra liv. LCD'en indikerer, hvilken af de tre ammunitionstyper spilleren er udstyret med.
Når spilleren mister alle sine liv, stopper spillet, og 'GAME OVER' vises på skærmen. Spilleren kan nu aflæse sin endelige score på LCD'en. Et tryk på en af knapperne sender spilleren tilbage til menuen, hvor spillet kan startes forfra.

6.3 Pinopsætning

- ADC1 -> PC_3
- ADC2 -> PC_2
- GND -> GND
- VDD -> 3V3
- B1 -> PC_12
- B2 -> PC_10

7 Diskussion

7.1 Projektets styrker og svagheder

7.1.1 Styrker

En af projektets største styrker er den modulære arkitektur, som udgør fundamentet for projektets design. Den modulære tilgang gør det muligt at arbejde parallelt på forskellige moduler og samle dem senere. Konsekvent spiltestning og debugging sikrer, at større bugs og problemer undgås senere i udviklingsprocessen.
Derudover indeholder projektet en række avancerede funktioner og udnytter mange af de indbyggede egenskaber i STM32CubeIDE, såsom LCD-skærmen. Dette forbedrer både brugerens spiloplevelse og gør spillet mere unikt.

7.1.2 Svagheder

En af de største udfordringer er projektets tidsbegrænsninger: Man skal både lære C og udvikle et spil på blot tre uger. Selvom man har en klar plan, kan nogle opgaver vise sig at være væsentligt mere tidskrævende, end man oprindeligt forudser. Dette kan føre til, at man uforudset må nedprioritere funktioner, der forbedrer spilleoplevelsen, for i stedet at fokusere på de grundlæggende funktioner.
Det komplekse miljø i STM32CubeIDE er desuden svært at navigere for begyndere i C. Der opstår mange udfordringer, når man skal forstå den omfattende brugermanual på over 1000 sider og samtidig håndtere maskinens skavanker. Desuden har forskellige programmører ofte forskellige skrivestile, og uden en klart aftalt navngivningskonvention for filer og funktioner kan det skabe problemer.
Alt dette kan skabe udfordringer under implementeringen af koden, men det er stadig overkommeligt. Vi aftalte for eksempel tidligt at bruge snake$_{c}aseogsrgedeforatimplementeredetkonsekventu$

7.2 Joystick-implementering

Ved implementeringen af 30010-joysticket stødte vi på flere problemer. Med et analogt signal på 12 bits ville man forvente, at den læste en signalstyrke mellem 0 og 4095. Vi målte dog værdier mellem 750 og 3500 med mindre variation mellem joystickets x- og y-akse.
Dette resulterede i, at if-sætningerne blev konfigureret baseret på vores målinger. Disse if-sætninger ville ikke fungere korrekt, hvis 30010-joysticket virkede normalt.
I appendix findes kode til brug med et normalt fungerende joystick (9). Denne kode skal indsættes i if-sætningerne i funktionen *read_joystick()* i filen *joystick_reader.c*, som findes i mappen *init*.

7.3 Krav

- Systemet skal kunne klare forkerte inputs og fejl i joystick og knapper.

Systemet reagerer ikke på ugyldige inputs, og dermed er dette ikke et problem. Hvis man for eksempel er i "Help Menu", og klikker på den hvide knap, sker der ingenting.

- Spilleren skal kunne styre skibet op, ned, til højre og venstre med joysticket.

I spillet kan man styre skibet i otte retninger, dog kan grafikken kun vise fire retninger. Det sker, da grafikken er lavet med ASCII-karakterer, og vi konluderede, at et diagonalt rumskib ikke kan eksekveres pænt.

- Spilleren skal kunne affyre skud ved at trykke på en knap.

Man kan skyde i 8 forskellige retninger. Det kræver dog meget processorkraft i mikrocontrolleren, hvilket resulterer i, at man må begrænse antallet af skud.

- En "Boss Key"skal kunne aktiveres nemt via en knap på joysticket.

Der er implementeret en boss-key, som viser et portræt og en lommeregner lavet med ASCII-karakterer. Boss-key'en pauser spillet og scoren indtil man klikker på den igen.

- Spillet skal understøtte flere skud-typer "Laser", "Normal", "Shotgun", aktiveret via power-ups.

Spillet har 3 power-ups: "Laser", "Shotgun"og "Health". Når man starter spillet bruger rumskibet standard skud, indtil man får en power-up.

- Skud skal påvirkes af en tilfældig deltavinkel.

Vi havde implementeret en tilfældig deltavinkel, men da vi spiltestede det, gjorde det spilleoplevelsen væsentligt værre og væsentligt mere frustrerende.
Det var implementeret på følgende måde, hvor der var tilfældige udfald for alle rumskibets retninger:

```
switch (pos->direction) {
case 1:
    switch (rand() % 3) {
    case 0:
        bullets_array[i].vx = 0;
        bullets_array[i].vy = -1;
        break;
    case 1:
        bullets_array[i].vx = 2;
        bullets_array[i].vy = -1;
        break;
    case 2:
        bullets_array[i].vx = 2;
        bullets_array[i].vy = 0;
        break;
```

```
16      }
17      break;
18  case 2: ...
```

Liste 9: Tilfældig deltavinkel kode

- Kollision mellem skud og fjender skal registreres og fjenderne fjernes.

Der er kollision mellem spilleren, bullets, aliens, meteorer og power-ups. Meteoren bliver ikke påvirket af noget, men stopper bullets, og spilleren mister alle liv, hvis rumskibet støder ind i den.

- Spilleren starter med tre liv, som reduceres ved:
  - Kollision med fjender eller meteorer.
  - Fjender, der undslipper og når skærmens bagside.

Dette er implementeret i spillet. Derudover er der tilføjet en power-up, der giver spilleren et ekstra liv, med mindre spilleren allerede har tre liv.

- Spillet slutter, når spilleren mister alle liv, og scoren vises på LCD-skærmen.

Når man mister alle tre liv i spillet, vises der en "game-over"skærm og scoren pauses på LCD'en.

- LCD-skærmen viser spillerens liv, point og valgte skud-type, opdateres løbende.

LCD'en er programmeret til at vise liv i øvre venstre hjørne, point i øvre højre hjørne, og skud typer i bunden. Alle disse værdier opdateres løbende, så de altid er ajourført.

- Joysticket skal bruges til skibets bevægelser og menu-navigation.

I spillet kan en 30010 joystick bruges til at navigere i menuen, styre skibet i 8 retninger, affyre skud og aktivere boss-keyen.

- Koden skal skrives modulært for nemt at kunne tilføje nye funktioner, fjendetyper eller power-ups.

Vores spil er modulært opbygget i 3 lag: AP, API og HAL. Dette gør det nemt at tilføje nye funktioner.

- Nye niveauer med stigende sværhedsgrad skal implementeres.

Spillets niveauer stiger gradvist hen over en runde hvert 25'ende sekund. Der er implementeret 15 niveauer, men ved spiltestning har vi konstateret at spillet bliver meget svært allerede ved niveau fem. Niveauet kan ikke vælges fra menuskærmen, for at gøre GUI'en mere simpel. Implementeringen ville dog nok se sådan her ud:

```
void game_menu(void) {
    ...
    switch (difficulty) {
    case 0: // Easy
        uint8_t alien_speed = 20;
        break;
    case 1: // Normal
        uint8_t alien_speed = 15;
        break;
    case 2: // Hard
        uint8_t alien_speed = 10;
        break;
    } ...
```

Liste 10: Valg af sværhedsgrad kode

8 Konklusion

Spillet *Starfighter* opfylder de kravspecifikationer, som er givet i opgaveformuleringen. Spillet er skrevet i programmeringssproget C og kører på en STM32-mikrocontroller med eksternt tilsluttet joystick 30010. Spilleren kan navigere spillet og menuen via joysticket, affyreskud, kolliderer med fjender og objekter, opnå midlertidige power-ups, samtidigt med at liv, point og skud-typer er visualiseret på den tilhørende LCD-skærm. Spillet indeholder hardware og software elementer, som let kan genanvendes i en videreudvikling af lignende projekter.

8.1 Hvem gjorde hvad i STM32-IDE

Oversigt over hvem har lavet hvilke filer under Src mappen i vores StarFighter projekt i STM32-IDE.

	August	Carl	Jacob	Simon
boss_key.c		×	×	
bullets.c				×
collision_detection.c				×
gravity.c				×
led.c		×		×
movement.c				×
spawn_powerups.c				×
background.c	×		×	
enemy.c				×
lcd.c		×		
powerup.c	×			
spaceship.c	×			
title.c		×	×	
init_GPIO.c				×
joystick_reader.c			×	
timer.c		×		×
game_menu.c			×	×
help_menu.c			×	
30010_io.c				
main.c	×		×	
Samlet tid i alt	45 timer	55 timer	55 timer	65 timer

8.2 Hvem gjorde hvad i rapporten

Oversigt over hvem der har lavet væsentlige bidrag til hvad i rapporten.

	August	Carl	Jacob	Simon
Abstract/Resume		×		×
Introduktion	×		×	
Kravspecifikationer	×			
Analyse		×	×	×
Design	×	×		×
Projektet	×			
Brugermanual			×	×
Diskussion	×	×	×	×
Konklusion		×		
Samlet tid i alt	25 timer	25 timer	19 timer	20 timer

Liste over figurer

1 main.c flowchart III
3 Block diagram IV
2 GANTT-chart IV

Referencer

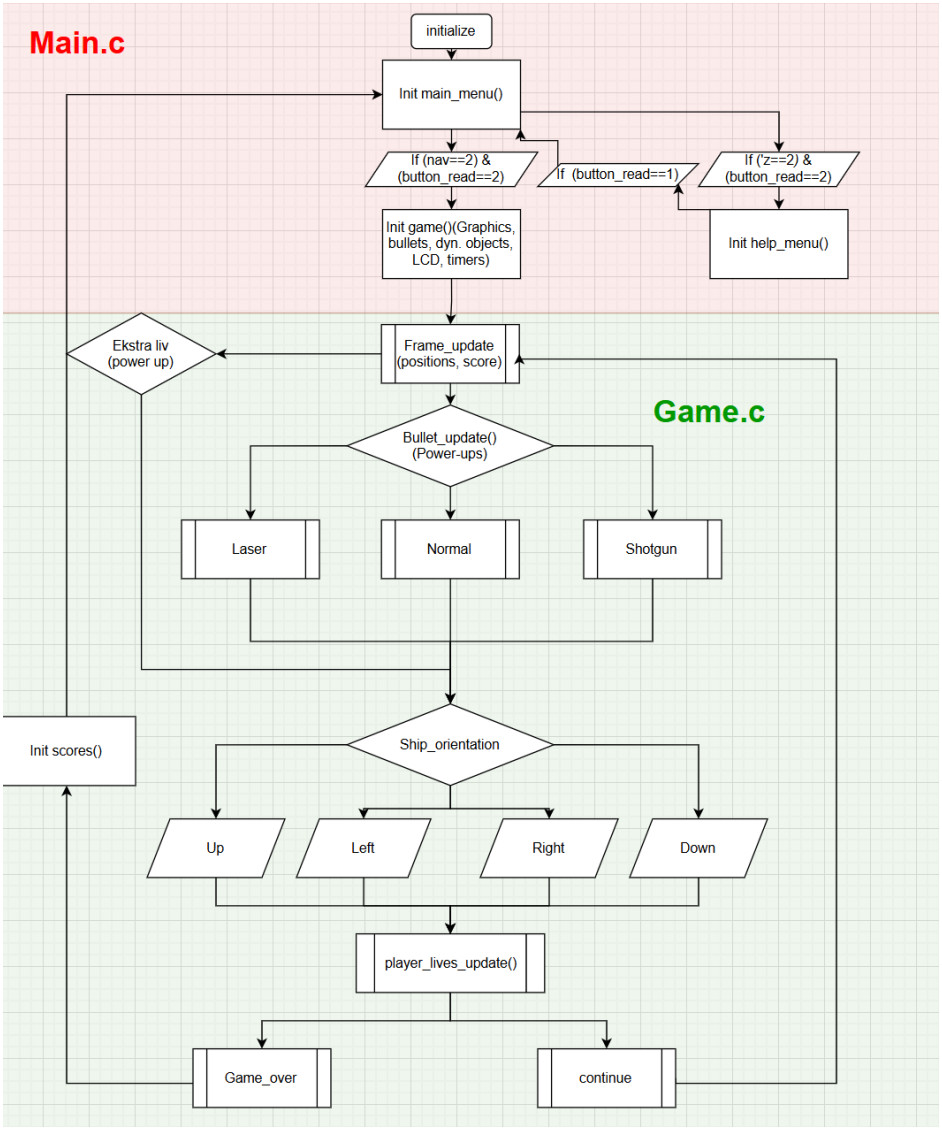
[1] patorjk, "Main controls." Accessed: 21-01-2025.

[2] "Ascii art." Accessed: 20-01-2025.

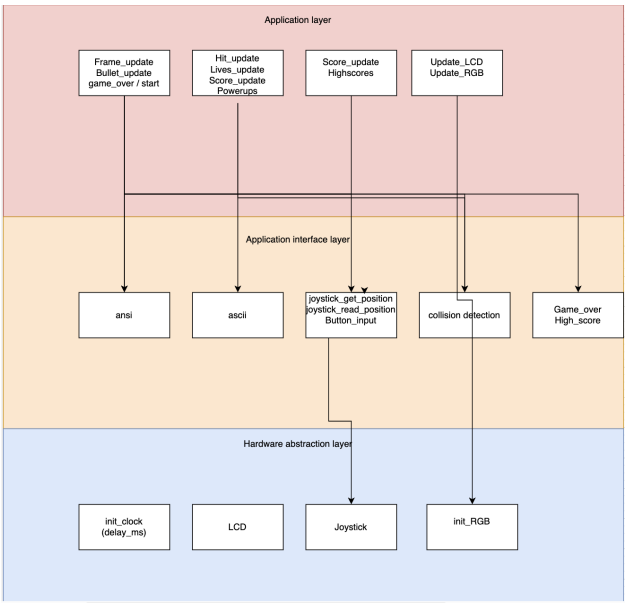
[3] J. M. G. Merayo, "Kursus 30010 uge 2," Januar 2025.

[4] S. life.augmented, "Reference manual." Accessed: 17-01-2025.

9 Appendiks A

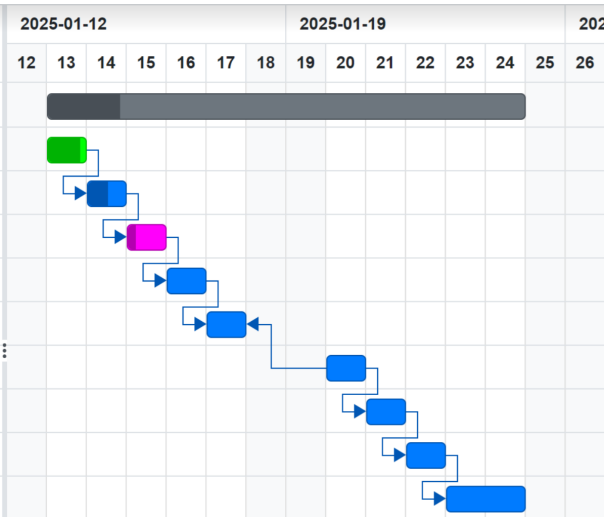


Figur 1: main.c flowchart



Figur 3: Block diagram

ID	Task Name
1	30010 projekt
2	Planlægning af spil
3	GPIO-Opsætning, Grafisk elementer
4	Spillerbevægelse, timer til score
5	Menulogik, bosskey
6	Overfladisk Skud, Asteroidebevægelse, kollisionsdet
7	Enemy spawn, spawn-hastighed, livsystemet
8	Power ups, ekstra liv, udvidet skudmekanik
9	High score implementering og help menu. Tyngdekraft
10	Debugging og finalization



Figur 2: GANTT-chart

```
if ((1900 < x) && (x < 2100) && (1900 < y) && (y < 2100)) {
    new_value = 0;
} else if ((1900 < x) && (x < 2100) && (3700 < y) && (y < 4100)) {
    new_value = 8;
} else if ((3700 < x) && (x < 4100) && (1900 < y) && (y < 2100)) {
    new_value = 2;
} else if ((1900 < x) && (x < 2100) && (0 < y) && (y < 400)) {
    new_value = 4;
} else if ((0 < x) && (x < 400) && (1900 < y) && (y < 2100)) {
    new_value = 6;
} else if ((3000 < x) && (x < 3700) && (3000 < y) && (y < 3700)) {
    new_value = 1;
} else if ((3000 < x) && (x < 3700) && (300 < y) && (y < 1500)) {
    new_value = 3;
} else if ((200 < x) && (x < 1300) && (200 < y) && (y < 1300)) {
    new_value = 5;
} else if ((200 < x) && (x < 1300) && (3000 < y) && (y < 3700)) {
    new_value = 7;
}
```

Liste 11: If-sætninger til normalt fungerende 30010 joystick

10 Appendiks B - Source Code

```
#include "stm32f30x_conf.h" // STM32 config
#include "30010_io.h"           // Input/output library for this course

#include <features/boss_key.h>
#include <features/bullets.h>
#include <features/collision_detection.h>
#include <features/gravity.h>
#include <features/led.h>
#include <features/movement.h>
#include <graphics/ansi.h>
#include <graphics/background.h>
#include <graphics/enemy.h>
#include <graphics/gamestart.h>
#include <graphics/lcd.h>
#include <graphics/spaceship.h>
#include <graphics/Title.h>
#include <init/basic_functions.h>
#include <init/init_GPIO.h>
#include <init/joystick_reader.h>
#include <init/timer.h>
#include <Menus/game_menu.h>
#include <Menus/help_menu.h>
#include <Menus/Main_menu.h>

int main(void) {
        uart_init(345600);
        // initialize everything
        printf("\033[8;76;243t"); // Resolution to: 243x76
        clrscr();
        configure_pins();
        configure_joystick();
        configure_adv_joystick();
        configure_led();
        timer15();
        lcd_init();

        // Main-loop

        uint8_t nav = 0;

        while (1) {
                if (nav == 0) {
                        main_menu(&nav);
```

```
44                  }
45                  if (nav == 1) {
46                          help_menu(&nav);
47                  }
48                  if (nav == 2) {
49                          game_menu();
50                          game_over(&nav);
51                  }
52          }
53  }
```

Liste 12: Main.c

```
1  #ifndef GRAPHICS_ANSI_H_
2  #define GRAPHICS_ANSI_H_
3
4  void clrscr();
5
6  #endif /* GRAPHICS_ANSI_H_ */
```

Liste 13: Ansi.h

```
1
2  #define ESC 0x1B
3  #include <graphics/ansi.h>
4  #include <stdint.h>
5  #include <stdio.h>
6  //void gotoxy(int x, int y) {
7  //    printf("\033[%d;%dH", y, x); // ANSI escape sequence to move
       the cursor
8  //}
9  void fgcolor(uint8_t foreground) {
10 /*  Value      foreground     Value     foreground
11     -------------------------------------------------
12       0        Black            8       Dark Gray
13       1        Red              9       Light Red
14       2        Green           10       Light Green
15       3        Brown           11       Yellow
16       4        Blue            12       Light Blue
17       5        Purple          13       Light Purple
18       6        Cyan            14       Light Cyan
19       7        Light Gray      15       White
20 */
21   uint8_t type = 22;             // normal text
22         if (foreground > 7) {
23           type = 1;                // bold text
24                 foreground -= 8;
```

```
25          }
26    printf("%c[%d;%dm", ESC, type, foreground+30);
27  }
28
29  void bgcolor(uint8_t background) {
30  /* IMPORTANT:   When you first use this function you cannot get back
        to true white background in HyperTerminal.
31     Why is that? Because ANSI does not support true white background (
          ANSI white is gray to most human eyes).
32                  The designers of HyperTerminal, however, preferred
                       black text on white background, which is why
33                  the colors are initially like that, but when the
                       background color is first changed there is no
34                          way comming back.
35     Hint:        Use resetbgcolor(); clrscr(); to force HyperTerminal
          into gray text on black background.
36
37      Value      Color
38      ------------------
39        0        Black
40        1        Red
41        2        Green
42        3        Brown
43        4        Blue
44        5        Purple
45        6        Cyan
46        7        Gray
47  */
48    printf("%c[%dm", ESC, background+40);
49  }
50  //void clrscr() {
51  //  printf("\033[2J\033[H"); // ANSI escape sequence to clear the
        screen and move cursor to top-left
52  //}
53  void color(uint8_t foreground, uint8_t background) {
54  // combination of fgcolor() and bgcolor() - uses less bandwidth
55    uint8_t type = 22;             // normal text
56          if (foreground > 7) {
57            type = 1;                 // bold text
58                  foreground -= 8;
59          }
60    printf("%c[%d;%d;%dm", ESC, type, foreground+30, background+40);
61  }
62
63  void resetbgcolor() {
64  // gray on black text, no underline, no blink, no reverse
```

```
65    printf("%c[m", ESC);
66 }
```

Liste 14: Ansi.c

```

#ifndef GRAPHICS_BACKGROUND_H_
#define GRAPHICS_BACKGROUND_H_

#include <stdint.h>
#include <stdio.h>

#include <parameters.h>

// Function declarations
void draw_background();
void big_planet();
void set_terminal_background();
void stars();
void init_background();
void init_background_section();
void init_baggrund_menu();

#endif /* BACKGROUND_H_ */
```

Liste 15: Background.h

```

#include <graphics/background.h>

void init_background_section(int x1, int y1, int x2, int y2) {
        stars(x1, y1, x2, y2);

        /*
         if (x1 <= 56 && x2 >= 48 && y1 <= 28 && y2 >= 20) {
         //big_planet(48, 20, 4, 2);
         }

         if (x1 <= 112 && x2 >= 104 && y1 <= 44 && y2 >= 36) {
         //big_planet(104, 36, 3, 1);
         }

         if (x1 <= 168 && x2 >= 160 && y1 <= 60 && y2 >= 52) {
         //big_planet(160, 52, 5, 3);
         }
         */
}
```

```
21
22 void init_background() {
23         stars(1, 1, 243, 76);
24 }
25
26 void init_baggrund_menu() {
27         stars(1, 1, 243, 76, 6);
28         big_planet(200, 3, 4, 2);
29         big_planet(59, 3, 1, 3);
30         big_planet(30, 53, 3, 4);
31         big_planet(100, 45, 2, 7);
32         big_planet(210, 45, 5, 6);
33 }
34
35 void stars(int x1, int y1, int x2, int y2, int c) {
36         int star_positions[][2] = { { 11, 11 }, { 17, 12 }, { 20, 9
              }, { 27, 11 }, {
37                         39, 17 }, { 50, 25 }, { 49, 31 }, { 56, 36 },
                               { 74, 43 },
38                         { 80, 39 }, { 76, 53 }, { 86, 14 }, { 93, 19
                              }, { 99, 22 }, { 116,
39                                       30 }, { 123, 38 }, { 126, 19
                                            }, { 140, 20 }, { 138, 33
                                            }, {
40                                       153, 36 }, { 163, 43 }, {
                                            168, 25 }, { 174, 8 },
41                         { 177, 14 }, { 185, 17 }, { 191, 19 }, { 204,
                               32 }, { 213, 27 }, {
42                                       214, 58 }, { 228, 11 }, {
                                            231, 27 }, { 239, 19 },
43                         { 15, 68 }, { 37, 71 }, { 53, 71 }, { 83, 74
                              }, { 99, 71 }, { 124,
44                                       68 }, { 137, 71 }, { 154, 76
                                            }, { 178, 74 }, { 191, 69
                                            }, {
45                                       212, 61 }, { 236, 72 }, { 22,
                                             17 }, { 50, 44 }, { 83,
                                            74 } };
46
47         int num_stars = sizeof(star_positions) / sizeof(
              star_positions[0]);
48
49         fgcolor(COLOR_OF_STARS);
50
51         for (int i = 0; i < num_stars; i++) {
52                 int x = star_positions[i][0];
```

```
53                  int y = star_positions[i][1];
54
55                  if (x >= x1 && x <= x2 && y >= y1 && y <= y2) {
56                          gotoxy(x, y);
57                          printf("*");
58                  }
59          }
60
61          resetbgcolor();
62  }
63
64  void big_planet(int x, int y, int c1, int c2) {
65          //1 line
66          gotoxy(x, y);
67          int i;
68          for (i = 0; i < 5; i++) {
69                  bgcolor(c1);
70                  printf("%c", 32);
71          }
72          gotoxy(x + 5, y);
73          bgcolor(c2);
74          printf("%c", 32);
75
76          //2 line
77          gotoxy(x - 2, y + 1);
78          for (i = 0; i < 6; i++) {
79                  bgcolor(c1);
80                  printf("%c", 32);
81          }
82          gotoxy(x + 4, y + 1);
83          for (i = 0; i < 4; i++) {
84                  bgcolor(c2);
85                  printf("%c", 32);
86          }
87
88          //3 line
89          gotoxy(x - 4, y + 2);
90          for (i = 0; i < 2; i++) {
91                  bgcolor(c2);
92                  printf("%c", 32);
93          }
94          gotoxy(x - 2, y + 2);
95          for (i = 0; i < 7; i++) {
96                  bgcolor(c1);
97                  printf("%c", 32);
98          }
```

```
99          gotoxy(x + 5, y + 2);
100         for (i = 0; i < 5; i++) {
101                 bgcolor(c2);
102                 printf("%c", 32);
103         }
104
105         //4 line
106         gotoxy(x - 4, y + 3);
107         for (i = 0; i < 3; i++) {
108                 bgcolor(c2);
109                 printf("%c", 32);
110         }
111         gotoxy(x - 1, y + 3);
112         for (i = 0; i < 7; i++) {
113                 bgcolor(c1);
114                 printf("%c", 32);
115         }
116         gotoxy(x + 6, y + 3);
117         for (i = 0; i < 4; i++) {
118                 bgcolor(c2);
119                 printf("%c", 32);
120         }
121
122         //5 line
123         gotoxy(x - 4, y + 4);
124         for (i = 0; i < 4; i++) {
125                 bgcolor(c2);
126                 printf("%c", 32);
127         }
128         gotoxy(x, y + 4);
129         for (i = 0; i < 10; i++) {
130                 bgcolor(c1);
131                 printf("%c", 32);
132         }
133
134         //6 line
135         gotoxy(x - 2, y + 5);
136         for (i = 0; i < 3; i++) {
137                 bgcolor(c2);
138                 printf("%c", 32);
139         }
140         gotoxy(x + 1, y + 5);
141         for (i = 0; i < 5; i++) {
142                 bgcolor(c1);
143                 printf("%c", 32);
144         }
```

```
145         gotoxy(x + 6, y + 5);
146         for (i = 0; i < 2; i++) {
147                 bgcolor(c2);
148                 printf("%c", 32);
149         }
150 
151         //7 line
152         gotoxy(x, y + 6);
153         for (i = 0; i < 1; i++) {
154                 bgcolor(c2);
155                 printf("%c", 32);
156         }
157         gotoxy(x + 1, y + 6);
158         for (i = 0; i < 3; i++) {
159                 bgcolor(c1);
160                 printf("%c", 32);
161         }
162         gotoxy(x + 4, y + 6);
163         for (i = 0; i < 2; i++) {
164                 bgcolor(c2);
165                 printf("%c", 32);
166         }
167 
168         resetbgcolor();
169 
170 }
```

Liste 16: Background.c

```
1 
2 #ifndef FEATURES_BOSS_KEY_H_
3 #define FEATURES_BOSS_KEY_H_
4 
5 void boss_key(int cs);
6 
7 #endif /* FEATURES_BOSS_KEY_H_ */
```

Liste 17: Boss$_k$ey.h

```
1 
2 #include <features/boss_key.h>
3 #include <init/joystick_reader.h>
4 
5 void boss_key(int cs) {
6 
7         if (read_button() == 1) {
8                 clrscr();
```

```
9                   uint16_t temp = cs;
10
11                  uint8_t x = 20;
12                  uint8_t y = 10;
13
14                  //inverse(1);
15                          gotoxy(x, y);
16                      printf("
                            ;;;
```

```
35                  gotoxy(x, y + 10);
36                  printf("              ;+
                       XXXXXXXXXXXXXXXXXXXXX$$$$$$$$XXxXxxXXXXXXXXXXXX$$XXXXXXX
                       ;               \n");
37                  gotoxy(x, y + 11);
38                  printf("              ;+
                       XXXXXXXXXXXXXXXXXXXX$$$$$$XXx++++++++
                       xXXXXXXXXXXXX$XXXXXXXXXXXXXXXXX;
                                    \n");
39                  gotoxy(x, y + 12);
40                  printf("              ;+
                       XXXXXXXXXXXXXXXXXXX$$$$$$Xx+++++;;;;;;;;;++
                       xXxxxx++xXXXXXXXXXXXXXXXXX;                \n")
                       ;
41                  gotoxy(x, y + 13);
42                  printf("              ;+
                       XXXXXXXXXXXXXXXXXX$$$$$$Xx
                       +++++++;;;;;;;;;;;;;;;;;;+XXXXXXXXXXXXXXXXX;
                                    \n");
43                  gotoxy(x, y + 14);
44                  printf("              ;+XXXXXXXXXXXXXXXXXX$$$$XXx
                       ++++++++;;;;;;;:::::::;;;;;;;+XXXXXXXXXXXXXXXXX;
                                     \n");
45                  gotoxy(x, y + 15);
46                  printf("              ;+
                       XXXXXXXXXXXXXXXXXX$$$$XXXxxx
                       +++++;;;;;;:::::;;;;;;;++XXXXXXXXXXXXXXXXX;
                                    \n");
47                  gotoxy(x, y + 16);
48                  printf("              ;+
                       XXXXXXXXXXXXXXXXXX$$$$XXXxxx
                       +++++;;;;;;;;;:::;;;;;;;++XXXXXXXXXXXXXXXXX;
                                    \n");
49                  gotoxy(x, y + 17);
50                  printf("              ;+
                       XXXXXXXXXXXXXXXXxX$$$$XXXxxxx
                       ++++;;;;;;;;;;;;;;;;;;+;+XXxXXXXXXXXxXXXX;
                                    \n");
51                  gotoxy(x, y + 18);
52                  printf("              ;+
                       XXXXXXXXXXXxxxX$Xxx$$$$$Xxxxxx
                       ++++;;;;;;;:::;;;;;;;;;++xxxxxxxxxxxxXxxxX;
                                    \n");
53                  gotoxy(x, y + 19);
54                  printf("              ;+
                       xXXxXXxxxxxxXXxxX$X$X$$$Xxxxxx
```

```
    ++++;;;;;;;;::;;;::;;;;+Xxxxxxxxxxxxxxxx;
                \n");
55  gotoxy(x, y + 20);
56  printf("             ;+xxxxxxxxxXxxxxxxXX$XX$$Xx+
    xxXXXXXXXx++;;;;;;;;;;;;;;;XxXxxxxxxxxxxxxxx;
                \n");
57  gotoxy(x, y + 21);
58  printf("             ;+xxxxxxxxxxxxxxxxXx$XX$Xxx+
    xxXXXXXXXXXXx+;;+++x+++++++xxxxxxxxxxxxxxxx;
                \n");
59  gotoxy(x, y + 22);
60  printf("             ;+xxxxxxxxxxxxxxxxxxXXXXXx
    ++++xXXXX$+XXXXX++++XX$$Xx+x+xxx++xxxxxx+xx+
    xxx;              \n");
61  gotoxy(x, y + 23);
62  printf("             ;+xxxxxxxxxxxxxxxxxxXXxXxx
    +++;++x+x++;;+X;:;+XX$$x$X++++++++++++++xx+x+x
    ;               \n");
63  gotoxy(x, y + 24);
64  printf("             ;+xxxxxxx+x+++x+++++xxXXXXXx
    ++++;::::::;;++x;;;++xxxx++;+++++++++++++++x
    ++;              \n");
65  gotoxy(x, y + 25);
66  printf("             ;++++++++++++++++++XXXXXxx
    +++;;;;;;;;;;+++;:;;;;;;++;;;++++++++++++++++++;
                \n");
67  gotoxy(x, y + 26);
68  printf("             ;++++++++++++++++++
    xxXXXXxxx+++;;;;++++x
    +;;;;;;:;;;;;;++++++++++++++++++;
    \n");
69  gotoxy(x, y + 27);
70  printf("             ;++++++++++++++++++
    xXXXxxxXxx++++++Xx
    ;;++;;+;+;;;;;;++++++++++++++++++;
                \n");
71  gotoxy(x, y + 28);
72  printf("             ;;++++++++++++++++++xXXXXx+
    XX+++++++X$$$X+;;++;x
    ++;;++++++++++++++++++;              \n");
73  gotoxy(x, y + 29);
74  printf("             ;;++++++++++++++++++xXXxXx+
    xx+;+xx+x++xXXXX+++;+X
    ++++++++++++++++++++++;              \n");
75  gotoxy(x, y + 30);
76  printf("             ;;+++++++++++++++X$&
```

```
                XxXxxXxxx+;;;+&X++++XX
                +++++;++++++++++++++++++++++;                \
                n");
77              gotoxy(x, y + 31);
78              printf("             ;;+++++++++++++++$&&
                xXXXxXXxx+;+;;+xxx++xxXXXx
                +;;++++++++++++++++++++;                \n");
79              gotoxy(x, y + 32);
80              printf("             ;;+++++++++++++++X&
                $xXXXXXXXx++++++xx+;;;++x
                +;;;+++++;++;;;;++;++;;++;+++;              \n"
                );
81              gotoxy(x, y + 33);
82              printf("             ;;++++++++++;++++X&
                XXXXXXXXXXxx++;;+xxxxx
                +++;+++++;;;;;;;;;;;;;;;;;;;;+;+;;;
                \n");
83              gotoxy(x, y + 34);
84              printf("             ;;++++++;+;+;;;;++
                x$XXXXXXXXXXXXx
                ++;;+;;+++;;;+++;;+;;;;;;;;;;;;;;;;;;;;;;;;;;
                             \n");
85              gotoxy(x, y + 35);
86              printf("             ;;++;++;;:;+;;;;;;+
                XxXXXXXXXXXXXXX
                ++++;;;;;;;;+;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
                             \n");
87              gotoxy(x, y + 36);
88              printf("             ;;;;::::::+;;;;;;;;+
                $XXXXXXXXXXXXXXx
                ++++++++;:::::;;;;;;;;;;;;;;;;;;;;;;;;;;
                             \n");
89              gotoxy(x, y + 37);
90              printf("             ;;::::::::;;;;;;;;;;
                XxxxxXXXXXXXXXXXXxxXxx
                ++;::::::::;;;;;;;;;;;;;;;;;;;;;;              \
                n");
91              gotoxy(x, y + 38);
92              printf("             ;;:::::::;:::;;;:::+
                XxxxxXXXXXXXXXXXXxx
                +++;;;:::::;:::::::::;;;;;;;;;;;;;;;;
                             \n");
93              gotoxy(x, y + 39);
94              printf("             ;;:::::::;:::;:::::::X+x+
                xXXXXXXXXXXXx+++++x
                ;;:::::::::::::::::;;;;;;;;;;              \n"
```

```
                    );
95                  gotoxy(x, y + 40);
96                  printf("              ;::::::::+::::;;:::::::+x++
                    xxxXXXXXXXxxx+++;++$
                    ;;:::::::::::::::::::::::;;;;;;;;               \n"
                    );
97                  gotoxy(x, y + 41);
98                  printf("              ;::::::;:::::;:::::::::x+++++
                    xxxxxxxx++++;+++$
                    +:::::::::::::::::::::::::;;;;;;               \n"
                    );
99                  gotoxy(x, y + 42);
100                 printf("              ;::::::::::;:;:::::::::++x++++
                    x+++++++++;+++++x;;:::::::::::::::::::::::::;;;;
                                \n");
101                 gotoxy(x, y + 43);
102                 printf("
                    ;::::::::::::::::::::::;;++++++++++++;+++;;X
                    :+;:::::::::::::::::::::::::;;;                 \n
                    ");
103                 gotoxy(x, y + 44);
104                 printf("
                    ;::::::::::;:;;;;;:::::;;;;;;++++++;;;;;;+;:::;:::::::.:
                                \n");
105                 gotoxy(x, y + 45);
106                 printf("              ;::::;;:;;+X$$XXX$$
                    :::::;;;;;;;+;;;;;;;;;;++:::::::::.$$X;:+x
                    ;::::::::::::;                \n");
107                 gotoxy(x, y + 46);
108                 printf("              ;:::::::;++;+++xXXXXx
                    ::::::;:;;;;;;;;;;;;;;+:;:::::::::::x
                    +;:::;;;:::::::::::::;                \n");
109                 gotoxy(x, y + 47);
110                 printf("              ;::::::::::;:;;;+XXXXX
                    ;:.:::;;;;;:::;;;:::;;$
                    ;::::::::::::::::::::::::::::::::;
                                \n");
111                 gotoxy(x, y + 48);
112                 printf("              ;:::::::::::::;;+xXXXXXX
                    +.:::;:;:::;;;;;;X
                    +;::::::::::::::::::::::::::::::::;
                                \n");
113                 gotoxy(x, y + 49);
114                 printf("              ;;:::::::::::::;++xxxXXXX
                    ;:::;:::;;;;;;+X
                    ;;::::::::;;::::::::::::::::::::::;
```

```
                    \n");
115             gotoxy(x, y + 50);
116             printf("             ;;:::::::::::::::;;;+++xXXXX
                    :::;;;;;;;;;$
                    ;::::::::;;;:::::::::::::::::::::::::::;
                    \n");
117             gotoxy(x, y + 51);
118             printf("             ;:::::::::::::::::::::;;;;;;XX$
                    +::;;;;;;;xx
                    ;::.:;;:::::::::::::::::::::::::::::;;:;
                    \n");
119             gotoxy(x, y + 52);
120             printf("             ;;:::::::;::::::::::::::;;;;;:;
                    XXX:::;;;;+$
                    ;:::;:::::::::::::::::::::::::::::::;;:;
                    \n");
121             gotoxy(x, y + 53);
122             printf("             ;::::::::;;:::::::::::::::::;;:::
                    xXX+:;;+$
                    ;:::;;:::::::::::::::::::::::::::::::;;;;:;
                    \n");
123             gotoxy(x, y + 54);
124             printf("
                    ;;;;;;:;;;;;;;:;;;;::;;:;;;;;;;;;;;;+++:;;+;;;;;:::;;;;;;:;;;;;
                     \n");
125             //inverse(0);
126
127              x = 130;
128              y = 7;
129
130             gotoxy(x, y);
131             printf("
                    .-----------------------------------------,
                              \n");
132             gotoxy(x, y + 1);
133             printf("                  |
                    .--------------------------------------, |
                              \n");
134             gotoxy(x, y + 2);
135             printf("                  | | _~_
                                                | |
                              \n");
136             gotoxy(x, y + 3);
137             printf("                  | |  \\/  Texas
                    Instruments     T I - 8 6  | |
                              \n");
```

```
138                 gotoxy(x, y + 4);
139                 printf("                    | |
                       .-----------------------------------, | |
                                           \n");
140                 gotoxy(x, y + 5);
141                 printf("                    | | |
                                                      | | |
                                           \n");
142                 gotoxy(x, y + 6);
143                 printf("                    | | | 1 + 1
                                                    | | |
                                           \n");
144                 gotoxy(x, y + 7);
145                 printf("                    | | |
                                                      | | |
                                           \n");
146                 gotoxy(x, y + 8);
147                 printf("                    | | |
                                                 2    | | |
                                           \n");
148                 gotoxy(x, y + 9);
149                 printf("                    | | |
                                                      | | |
                                           \n");
150                 gotoxy(x, y + 10);
151                 printf("                    | | | 6 * 8
                                                    | | |
                                           \n");
152                 gotoxy(x, y + 11);
153                 printf("                    | | |
                                                   14   | | |
                                           \n");
154                 gotoxy(x, y + 12);
155                 printf("                    | | |
                                                      | | |
                                           \n");
156                 gotoxy(x, y + 13);
157                 printf("                    | | | _
                                                       | | |
                                           \n");
158                 gotoxy(x, y + 14);
159                 printf("                    | | |
                                                      | | |
                                           \n");
160                 gotoxy(x, y + 15);
161                 printf("                    | | |
```

```
                                                    | | |
                                \n");
162                     gotoxy(x, y + 16);
163                     printf("                    | | |
                                                    | | |
                                \n");
164                     gotoxy(x, y + 17);
165                     printf("                    | | |
                                                    | | |
                                \n");
166                     gotoxy(x, y + 18);
167                     printf("                    | | |
                                                    | | |
                                \n");
168                     gotoxy(x, y + 19);
169                     printf("                    | | |
                                                    | | |
                                \n");
170                     gotoxy(x, y + 20);
171                     printf("                    | |
                        `-----------------------------------' | |
                                \n");
172                     gotoxy(x, y + 21);
173                     printf("                    | |
                                                        | |
                                \n");
174                     gotoxy(x, y + 22);
175                     printf("                    |
                        `---------------------------------------' |
                                \n");
176                     gotoxy(x, y + 23);
177                     printf("                    |     M1       M2       M3
                                M4       M5     |                    \n");
178                     gotoxy(x, y + 24);
179                     printf("                    |  .-----, .-----,
                        .-----, .-----, .-----,  |                    \
                        n");
180                     gotoxy(x, y + 25);
181                     printf("                    |  |  F1 | |  F2 | |  F3
                         | |  F4 | |  F5 |  |                    \n");
182                     gotoxy(x, y + 26);
183                     printf("                    |  `-----' `-----'
                        `-----' `-----' `-----'  |                    \
                        n");
184                     gotoxy(x, y + 27);
185                     printf("                    |-------._________
```

```
                    --------,--------|
    \n");
186 gotoxy(x, y + 28);
187 printf("                    |
    `--------'                    |
                            \n");
188 gotoxy(x, y + 29);
189 printf("                    |
                                                |
                            \n");
190 gotoxy(x, y + 30);
191 printf("                    |           QUIT    MODE
              _,-._         |                    \n");
192 gotoxy(x, y + 31);
193 printf("                    |  .-----, .-----,
    .-----,  /\\ \\_^_/ /\\      |
                            \n");
194 gotoxy(x, y + 32);
195 printf("                    |  | 2nd | | EXIT| |
    MORE| / /   |   \\ \\     |                    \
    n");
196 gotoxy(x, y + 33);
197 printf("                    |  `-----' `-----'
    `-----' |<|   0   |>|     |                    \
    n");
198 gotoxy(x, y + 34);
199 printf("                    |  alpha   LINK x  INS
         \\ \\  _|_  / /     |                     \n"
    );
200 gotoxy(x, y + 35);
201 printf("                    |  .-----, .-----,
    .-----,  \\/ / v \\ \\/      |
                            \n");
202 gotoxy(x, y + 36);
203 printf("                    |  |ALPHA| |x-VAR| | DEL
     |      -._,-         |                    \n");
204 gotoxy(x, y + 37);
205 printf("                    |  `-----' `-----'
    `-----'                   |                    \
    n");
206 gotoxy(x, y + 38);
207 printf("                    |  SOLVER  SIMULT  POLY
      CATLG-VARS          |                    \n");
208 gotoxy(x, y + 39);
209 printf("                    |  .-----, .-----,
    .-----, .-----, .-----,  |                    \
```

```
                n");
210             gotoxy(x, y + 40);
211             printf("                |  |GRAPH| |TABLE| |
                PRGM| |CUSTM| |CLEAR|  |                  \n"
                );
212             gotoxy(x, y + 41);
213             printf("                |  `-----' `-----'
                `-----' `-----' `-----'  |                  \
                n");
214             gotoxy(x, y + 42);
215             printf("                |  10x  A  SIN-1 B COS-1
                 C TAN-1 D pi   E   |                  \n");
216             gotoxy(x, y + 43);
217             printf("                |  .-----, .-----,
                .-----, .-----, .-----,  |                  \
                n");
218             gotoxy(x, y + 44);
219             printf("                |  | LOG | | SIN | | COS
                 | | TAN | |  ^  |  |                  \n");
220             gotoxy(x, y + 45);
221             printf("                |  `-----' `-----'
                `-----' `-----' `-----'  |                  \
                n");
222             gotoxy(x, y + 46);
223             printf("                |  ex   F  x-1  G  [
                H  ]    I  CALC J   |                  \n");
224             gotoxy(x, y + 47);
225             printf("                |  .-----, .-----,
                .-----, .-----, .-----,  |                  \
                n");
226             gotoxy(x, y + 48);
227             printf("                |  | LN  | | EE  | |  (
                 | |  )  | |     |  |                  \n");
228             gotoxy(x, y + 49);
229             printf("                |  `-----' `-----'
                `-----' `-----' `-----'  |                  \
                n");
230             gotoxy(x, y + 50);
231             printf("                |  ./~  K  MATRX L VECTR
                 M CPLX N  MATH O   |                  \n");
232             gotoxy(x, y + 51);
233             printf("                |  .-----, .-----,
                .-----, .-----, .-----,  |                  \
                n");
234             gotoxy(x, y + 52);
235             printf("                |  | X2  | |  7  | |  8
```

```
                    | |  9  | |  x  |  |                    \n");
236             gotoxy(x, y + 53);
237             printf("                  |  `-----' `-----'
                    `-----' `-----' `-----'  |                    \
                    n");
238             gotoxy(x, y + 54);
239             printf("                  |  /_   P  CONS Q  CONV
                    R  STRNG S LIST T   |                    \n");
240             gotoxy(x, y + 55);
241             printf("                  |  .-----, .-----,
                    .-----, .-----, .-----,  |                    \
                    n");
242             gotoxy(x, y + 56);
243             printf("                  |  |  ,  | |  4  | |  5
                    | |  6  | |  -  |  |                    \n");
244             gotoxy(x, y + 57);
245             printf("                  |  `-----' `-----'
                    `-----' `-----' `-----'  |                    \
                    n");
246             gotoxy(x, y + 58);
247             printf("                  |  RCL  =  BASE U  TEST
                    V  MEM  W  STAT X   |                    \n");
248             gotoxy(x, y + 59);
249             printf("                  |  .-----, .-----,
                    .-----, .-----, .-----,  |                    \
                    n");
250             gotoxy(x, y + 60);
251             printf("                  |  | STO>| |  1  | |  2
                    | |  3  | |  +  |  |                    \n");
252             gotoxy(x, y + 61);
253             printf("                  |  `-----' `-----'
                    `-----' `-----' `-----'  |                    \
                    n");
254             gotoxy(x, y + 62);
255             printf("                  |  OFF     CHAR Y  :
                    z  ANS  _  ENTRY    |                    \n");
256             gotoxy(x, y + 63);
257             printf("                  |  .-----, .-----,
                    .-----, .-----, .-----,  |                    \
                    n");
258             gotoxy(x, y + 64);
259             printf("                  |  | ON  | |  0  | |  .
                    | | (-) | |ENTER|  |                    \n");
260             gotoxy(x, y + 65);
261             printf("                  |  `-----' `-----'
                    `-----' `-----' `-----'  |                    \
```

```
                        n");
262                     gotoxy(x, y + 66);
263                     printf("                    |   -------
                                                          |
                                                \n");
264                     gotoxy(x, y + 67);
265                     printf("                      `\\
                                                          /'
                                                 \n");
266                     gotoxy(x, y + 68);
267                     printf("                        `-._
                                                      _.-'
                                                   \n");
268                     gotoxy(x, y + 69);
269                     printf("                           `--.__
                                                 __.--'
                                                      \n");
270                     gotoxy(x, y + 70);
271                     printf("                                   ``----..._____
                        ,,---                                        \n");
272
273
274                 while (1) {
275                         if (read_button() == 2) {
276                                 cs = temp;
277                                 clrscr();
278                                 init_background();
279                                 break;
280                         }
281
282                 }
283         }
284 }
```

Liste 18: Boss$_k$ey.c

```
1
2  #ifndef FEATURES_BULLETS_H_
3  #define FEATURES_BULLETS_H_
4
5  #include <parameters.h>
6  #include <features/movement.h>
7  #include <init/timer.h>
8  #include <unistd.h>
9  #include <graphics/enemy.h>
10
11 typedef struct {
```

```
12          uint8_t bx;
13          uint8_t by;
14          int8_t vx;
15          int8_t vy;
16          uint8_t active;
17          uint8_t gravity;
18  } bullet;
19
20  bullet bullets_array[MAX_BULLETS];
21
22  void init_bullets(void);
23  void fire_bullet(position *pos, bullet_type typ);
24  void update_bullet_pos(void);
25  void bullet_spawn_correction(bullet bullets_array[MAX_BULLETS],
        position pos,
26                  uint8_t i);
27  void bullet_direction(position *pos, int8_t direction, uint8_t i);
28
29  #endif
```

Liste 19: Bullets.h

```
1
2   #include <features/bullets.h>
3
4   void init_bullets() {
5           for (uint8_t i = 0; i < MAX_BULLETS; i++) {
6                   bullets_array[i].active = 0;
7                   bullets_array[i].gravity = 0;
8           }
9   }
10
11  void fire_bullet(position *pos, bullet_type typ) {
12          static uint32_t last_fire_time = 0;
13          int8_t d;
14          // Fire delay
15          if (cs - last_fire_time < BULLET_FIRE_DELAY) {
16                  return;
17          }
18          last_fire_time = cs; // Update last fire time
19
20          // Create new bullet
21          switch (typ.type) {
22          case 0: // Normal bullets
23                  for (int i = 0; i < MAX_BULLETS; i++) {
24                          if (!bullets_array[i].active) {
25                                  bullets_array[i].bx = pos->x;
```

```
26                                  bullets_array[i].by = pos->y;
27                                  bullets_array[i].gravity = 0;
28
29                                  // Correct bullet spawn position
30                                  bullet_spawn_correction(bullets_array
                                        , *pos, i);
31
32                                  // Bullet direction
33                                  bullet_direction(&*pos, pos->
                                        direction, i);
34
35                                  //Activate bullet
36                                  bullets_array[i].active = 1;
37                                  break;
38                          }
39                  }
40                  break;
41
42          case 1: // Shotgun
43                  d = pos->direction;
44                  d += 7;
45
46                  if (!bullets_array[0].active && !bullets_array[1].
                        active
47                                  && !bullets_array[2].active) {
48                          for (uint8_t i = 0; i < 3; i++) {
49                                  if (!bullets_array[i].active) {
50                                          bullets_array[i].bx = pos->x;
51                                          bullets_array[i].by = pos->y;
52                                          bullets_array[i].gravity = 0;
53
54                                          if (d > 8) {
55                                                  d -= 8;
56                                          }
57
58                                          // Correct bullet spawn
                                                position
59                                          bullet_spawn_correction(
                                                bullets_array, *pos, i);
60
61                                          // Bullet direction
62                                          bullet_direction(&*pos, d, i)
                                                ;
63
64                                          //Activate bullet
65                                          bullets_array[i].active = 1;
```

```
66                                          d += 1;
67                                  }
68                          }
69                  }
70                  break;
71
72          case 2: //Laser
73                  if (!bullets_array[0].active) {
74                          bullets_array[0].bx = pos->x;
75                          bullets_array[0].by = pos->y;
76
77                          // Correct bullet spawn position
78                          bullet_spawn_correction(bullets_array, *pos,
                               0);
79
80                          switch (pos->direction) {
81                          case 2:
82                                  for (uint8_t j = 0; j < MAX_ALIENS; j
                                       ++) {
83                                          if ((bullets_array[0].bx <
                                               aliens_array[j].x)
84                                                  && (
                                                       bullets_array
                                                       [0].by >=
                                                       aliens_array
                                                       [j].y)
85                                                  && (
                                                       bullets_array
                                                       [0].by <
                                                       aliens_array
                                                       [j].y + 5)
                                                       ) {
86                                                  aliens_array[j].
                                                       active = 0;
87                                          }
88                                          gotoxy(bullets_array[0].bx,
                                               bullets_array[0].by);
89                                          for (uint8_t i =
                                               bullets_array[0].bx; i <
                                               243; i++) {
90                                                  printf("%c", 254);
91                                          }
92                                          gotoxy(bullets_array[0].bx,
                                               bullets_array[0].by);
93                                          for (uint8_t i =
                                               bullets_array[0].bx; i <
```

```
                                        243; i++) {
94                                          printf(" ");
95                                      }
96                                  }
97                                  break;
98                          case 4:
99                                  for (uint8_t j = 0; j < MAX_ALIENS; j
                                        ++) {
100                                     if ((bullets_array[0].by <
                                            aliens_array[j].y)
101                                                 && (
                                                        bullets_array
                                                        [0].bx >=
                                                        aliens_array
                                                        [j].x)
102                                                 && (
                                                        bullets_array
                                                        [0].bx <
                                                        aliens_array
                                                        [j].x +
                                                        10)) {
103                                             aliens_array[j].
                                                    active = 0;
104                                     }
105                                     for (uint8_t i =
                                            bullets_array[0].by; i <
                                            76; i++) {
106                                             gotoxy(bullets_array
                                                    [0].bx, i);
107                                             printf("%c", 219);
108                                     }
109                                     for (uint8_t i =
                                            bullets_array[0].by; i <
                                            76; i++) {
110                                             gotoxy(bullets_array
                                                    [0].bx, i);
111                                             printf(" ");
112                                     }
113                                 }
114                                 break;
115                         case 6:
116                                 for (uint8_t j = 0; j < MAX_ALIENS; j
                                        ++) {
117                                     if ((bullets_array[0].bx >
                                            aliens_array[j].x)
118                                                 && (
```

```
                                                    bullets_array
                                                    [0].by >=
                                                    aliens_array
                                                    [j].y)
119                                                 && (
                                                    bullets_array
                                                    [0].by <
                                                    aliens_array
                                                    [j].y + 5)
                                                    ) {
120                                         aliens_array[j].
                                                active = 0;
121                                 }
122                                 gotoxy(0, bullets_array[0].by
                                        );
123                                 for (uint8_t i = 0; i <
                                        bullets_array[0].bx; i++)
                                        {
124                                         printf("%c", 254);
125                                 }
126                                 gotoxy(0, bullets_array[0].by
                                        );
127                                 for (uint8_t i = 0; i <
                                        bullets_array[0].bx; i++)
                                        {
128                                         printf(" ");
129                                 }
130                         }
131                         break;
132                 case 8:
133                         for (uint8_t j = 0; j < MAX_ALIENS; j
                                ++) {
134                                 if ((bullets_array[0].by >
                                        aliens_array[j].y)
135                                                 && (
                                                    bullets_array
                                                    [0].bx >=
                                                    aliens_array
                                                    [j].x)
136                                                 && (
                                                    bullets_array
                                                    [0].bx <
                                                    aliens_array
                                                    [j].x +
                                                    10)) {
137                                         aliens_array[j].
```

```
                                                        active = 0;
138                                                 }
139                                                 for (uint8_t i = 0; i <
                                                        bullets_array[0].by; i++)
                                                        {
140                                                         gotoxy(bullets_array
                                                            [0].bx, i);
141                                                         printf("%c", 219);
142                                                 }
143                                                 for (uint8_t i = 0; i <
                                                        bullets_array[0].by; i++)
                                                        {
144                                                         gotoxy(bullets_array
                                                            [0].bx, i);
145                                                         printf(" ");
146                                                 }
147                                         }
148                                         break;
149                                 }
150                         }
151                         break;
152         }
153 }
154
155 void update_bullet_pos() {
156         for (uint8_t i = 0; i < MAX_BULLETS; i++) {
157                 if (((cs + 11) % MAX_BULLETS) == i) {
158                         if (bullets_array[i].active) {
159                                 // Delete bullet
160                                 gotoxy(bullets_array[i].bx,
                                        bullets_array[i].by);
161                                 printf(" ");
162
163                                 // Calculate new position
164                                 bullets_array[i].bx += bullets_array[
                                        i].vx;
165                                 bullets_array[i].by += bullets_array[
                                        i].vy;
166
167                                 // Check position
168                                 if (bullets_array[i].bx <= 0 ||
                                        bullets_array[i].by <= 0
169                                                 || bullets_array[i].
                                                    bx >= 243
170                                                 || bullets_array[i].
                                                    by >= 76) {
```

```
171                                         bullets_array[i].active = 0;
172                                 } else {
173                                         // Print bullet
174                                         gotoxy(bullets_array[i].bx,
                                               bullets_array[i].by);
175                                         printf("%c", 254);
176                                 }
177                         }
178                 }
179         }
180 }
181
182 void bullet_spawn_correction(bullet bullets_array[MAX_BULLETS],
       position pos,
183                 uint8_t i) {
184         switch (pos.direction) {
185         case 1:
186         case 2:
187         case 3:
188                 bullets_array[i].bx += 6;
189                 bullets_array[i].by += 2;
190                 break;
191         case 4:
192                 bullets_array[i].bx += 3;
193                 bullets_array[i].by += 4;
194                 break;
195         case 5:
196         case 6:
197         case 7:
198                 bullets_array[i].bx -= 1;
199                 bullets_array[i].by += 2;
200                 break;
201         case 8:
202                 bullets_array[i].bx += 3;
203                 bullets_array[i].by -= 1;
204                 break;
205         }
206 }
207
208 void bullet_direction(position *pos, int8_t direction, uint8_t i) {
209         switch (direction) {
210         case 1:
211                 bullets_array[i].vx = 2;
212                 bullets_array[i].vy = -1;
213                 break;
214         case 2: // Right
```

```
215                     bullets_array[i].vx = 2;
216                     bullets_array[i].vy = 0;
217                     break;
218             case 3:
219                     bullets_array[i].vx = 2;
220                     bullets_array[i].vy = 1;
221                     break;
222             case 4: // Down
223                     bullets_array[i].vx = 0;
224                     bullets_array[i].vy = 1;
225                     break;
226             case 5:
227                     bullets_array[i].vx = -2;
228                     bullets_array[i].vy = 1;
229                     break;
230             case 6: // Left
231                     bullets_array[i].vx = -2;
232                     bullets_array[i].vy = 0;
233                     break;
234             case 7:
235                     bullets_array[i].vx = -2;
236                     bullets_array[i].vy = -1;
237                     break;
238             case 8: // Up
239                     bullets_array[i].vx = 0;
240                     bullets_array[i].vy = -1;
241                     break;
242             }
243 }
```

Liste 20: Bullets.c

```
1
2  #ifndef FEATURES_COLLISION_DETECTION_H_
3  #define FEATURES_COLLISION_DETECTION_H_
4
5  #include <parameters.h>
6  #include <features/bullets.h>
7  #include <features/movement.h>
8  #include <graphics/enemy.h>
9  #include <graphics/ansi.h>
10 #include <graphics/background.h>
11 #include <features/spawn_powerups.h>
12 #include <graphics/enemy.h>
13
14 void bullet_collision(void);
15 void spaceship_collision(position pos, uint8_t *health, uint32_t *
```

```
    last_damage_time);
16 void powerup_collision(position pos, uint8_t *health, bullet_type *
    typ,
17                 uint32_t *powerup_time);
18
19 #endif
```

Liste 21: Collision detection.h

```
1
2  #include <features/collision_detection.h>
3
4  void bullet_collision(void) {
5          for (uint8_t i = 0; i < MAX_BULLETS; i++) {
6                  for (uint8_t j = 0; j < MAX_METEORS; j++) {
7                          if (bullets_array[i].active) {
8                                  // Meteor
9                                  if ((bullets_array[i].bx >=
                                       meteors_array[j].x)
10                                                 && (bullets_array[i].
                                                       bx < meteors_array
                                                       [j].x + 20)
11                                                 && (bullets_array[i].
                                                       by >=
                                                       meteors_array[j].y
                                                       )
12                                                 && (bullets_array[i].
                                                       by < meteors_array
                                                       [j].y + 10)) {
13                                         bullets_array[i].active = 0;
14                                         gotoxy(bullets_array[i].bx,
                                                bullets_array[i].by);
15                                         fgcolor(8);
16                                         printf("#");
17                                         resetbgcolor();
18                                 }
19                                 // Aliens
20                                 for (uint8_t j = 0; j < MAX_ALIENS; j
                                       ++) {
21                                         if ((bullets_array[i].bx >=
                                                aliens_array[j].x)
22                                                         && (
                                                            bullets_array
                                                            [i].bx <
                                                            aliens_array
                                                            [j].x + 5)
23                                                         && (
```

```
                                                                bullets_array
                                                                [i].by >=
                                                                aliens_array
                                                                [j].y)
24                                                          && (
                                                                bullets_array
                                                                [i].by <
                                                                aliens_array
                                                                [j].y + 5)
                                                                ) {
25                                                  bullets_array[i].
                                                        active = 0;
26                                                  remove_alien(j);
27
28                                                  gotoxy(bullets_array[
                                                        i].bx,
                                                        bullets_array[i].
                                                        by);
29                                                  printf(" ");
30                                          }
31                                  }
32                          }
33                  }
34          }
35  }
36
37  void spaceship_collision(position pos, uint8_t *health, uint32_t *
        last_damage_time) {
38          // Meteor
39          for (uint8_t i = 0; i < MAX_METEORS; i++) {
40                  if ((pos.x + 5 >= meteors_array[i].x)
41                                  && (pos.x < meteors_array[i].x + 20)
42                                  && (pos.y + 5 >= meteors_array[i].y)
43                                  && (pos.y < meteors_array[i].y + 10))
                                        {
44                          // Damage delay
45                          if (cs - *last_damage_time < DAMAGE_DELAY) {
46                                  return;
47                          }
48                          *last_damage_time = cs; // Update last fire
                                time
49                          *health = 0;
50                  }
51          }
52
53          // Aliens
```

```
54          for (uint8_t i = 0; i < MAX_ALIENS; i++) {
55                  if ((pos.x + 5 >= aliens_array[i].x) && (pos.x <
                        aliens_array[i].x + 5)
56                                  && (pos.y + 5 >= aliens_array[i].y)
57                                  && (pos.y < aliens_array[i].y + 5)) {
58                          remove_alien(i);
59                          // Damage delay
60                          if (cs - *last_damage_time < DAMAGE_DELAY) {
61                                  return;
62                          }
63                          *last_damage_time = cs; // Update last fire
                                time
64                          (*health)--;
65                  }
66          }
67  }
68
69  void powerup_collision(position pos, uint8_t *health, bullet_type *
        typ,
70                  uint32_t *powerup_time) {
71          if (powerups_array[0].active) {
72                  if (cs % 20 == 0) {
73                          // Spaceship
74                          if ((pos.x + 5 >= powerups_array[0].x)
75                                          && (pos.x <= powerups_array
                                                [0].x + 16)
76                                          && (pos.y + 5 >=
                                                powerups_array[0].y)
77                                          && (pos.y <= powerups_array
                                                [0].y + 8)) {
78                                  remove_powerups();
79                                  switch (powerups_array[0].
                                        powerup_effect) {
80                                  case 0:
81                                          typ->type = 1;
82                                          *powerup_time = cs;
83                                          break;
84                                  case 1:
85                                          typ->type = 2;
86                                          *powerup_time = cs;
87                                          break;
88                                  case 2:
89                                          if (*health < 3) {
90                                                  (*health)++;
91                                          }
92                                          break;
```

```
93                                      }
94                              }
95
96                              // Aliens
97                              for (uint8_t i = 0; i < MAX_ALIENS; i++) {
98                                      if ((aliens_array[i].x + 5 >=
                                           powerups_array[0].x)
99                                                      && (aliens_array[i].x
                                                           <= powerups_array
                                                          [0].x + 16)
100                                                     && (aliens_array[i].y
                                                           + 5 >=
                                                          powerups_array[0].
                                                          y)
101                                                     && (aliens_array[i].y
                                                           <= powerups_array
                                                          [0].y + 8)) {
102                                             remove_powerups();
103                                     }
104                             }
105                     }
106             }
107 }
```

Liste 22: Collision detection.c

```
1
2  #ifndef GRAPHICS_ENEMY_H_
3  #define GRAPHICS_ENEMY_H_
4
5  #include <stdlib.h> // For rand()
6  #include <string.h>
7
8  #include <parameters.h>
9  #include <init/basic_functions.h>
10 #include <graphics/background.h>
11 #include <init/timer.h>
12 #include <graphics/ansi.h>
13 #include <features/collision_detection.h>
14
15 typedef struct {
16         int x;
17         int y;
18         uint8_t active;
19         uint8_t powerup_effect;
20 } enemy_state;
21
```

```
22 enemy_state aliens_array[MAX_ALIENS];
23 enemy_state meteors_array[MAX_METEORS];
24 enemy_state powerups_array[MAX_POWERUPS];
25
26 void init_enemy_state(void);
27 void enemy_go(uint8_t *health, uint32_t *last_spawn_time,
28                 uint32_t *last_meteor_time, uint32_t *
                       last_damage_time,
29                 uint8_t alien_speed);
30 void spawn_meteor(uint32_t *last_meteor_time);
31 void spawn_alien(uint32_t *last_spawn_time);
32 void remove_alien(uint8_t i);
33
34 #endif /* GRAPHICS_ENEMY_H_ */
```

Liste 23: Enemy.h

```
1
2 #include <graphics/enemy.h>
3
4 void init_enemy_state(void) {
5         for (int i = 0; i < MAX_ALIENS; i++) {
6                 aliens_array[i].active = 0;
7         }
8         for (int i = 0; i < MAX_METEORS; i++) {
9                 meteors_array[i].active = 0;
10        }
11 }
12
13 void enemy_go(uint8_t *health, uint32_t *last_spawn_time, uint32_t *
       last_meteor_time,
14                 uint32_t *last_damage_time, uint8_t alien_speed) {
15        // METEORS
16        for (int i = 0; i < MAX_METEORS; i++) {
17                if ((cs + 1) % 100 == 0) {
18                        if (!meteors_array[i].active) {
19                                spawn_meteor(last_meteor_time);
20                        }
21                }
22                if ((cs + 3) % METEOR_SPEED == i) {
23                        if (meteors_array[i].active) {
24                                // Delete back of meteor
25                                if (meteors_array[i].x < 243 - 20) {
26                                        for (int j = 0; j < 10; j++)
                                                {
27                                                gotoxy(meteors_array[
                                                        i].x + 20,
```

```
                                                meteors_array[i].y
                                                 + j);
28                                          printf(" ");
29                                      }
30                                      init_background_section(
                                           meteors_array[i].x + 20,
31                                                 meteors_array
                                                    [i].y,
                                                    meteors_array
                                                    [i].x +
                                                    20,
32                                                 meteors_array
                                                    [i].y +
                                                    10,
                                                    COLOR_OF_STARS
                                                    );
33                                  }
34
35                                  // Calculate new position
36                                  meteors_array[i].x--;
37
38                                  // Check position
39                                  if (meteors_array[i].x <= -20) {
40                                          meteors_array[i].active = 0;
41                                  } else {
42                                          // Print front meteor
43                                          if (meteors_array[i].x > 0) {
44                                                  fgcolor(
                                                     COLOR_OF_METEORS);
45                                                  for (uint8_t j = 0; j
                                                      < 10; j++) {
46                                                      gotoxy(
                                                         meteors_array
                                                         [i].x,
                                                         meteors_array
                                                         [i].y + j)
                                                         ;
47                                                      printf("#");
48                                                  }
49                                                  resetbgcolor();
50                                          }
51                                  }
52                          }
53                  }
54          }
55
```

```
56 // ALIENS
57         for (int i = 0; i < MAX_ALIENS; i++) {
58                 if ((cs + 5) % alien_speed == i) {
59                         if (!aliens_array[i].active) {
60                                 spawn_alien(last_spawn_time);
61                         }
62                         if (aliens_array[i].active) {
63                                 // Delete back of alien
64                                 if (aliens_array[i].x < 243 - 4) {
65                                         for (int j = 0; j < 4; j++) {
66                                                 gotoxy(aliens_array[i
                                                    ].x + 4,
                                                    aliens_array[i].y
                                                    + j);
67                                                 printf(" ");
68                                         }
69                                         init_background_section(
                                                    aliens_array[i].x + 4,
70                                                             aliens_array[
                                                                i].y,
                                                                aliens_array
                                                                [i].x + 4,
71                                                             aliens_array[
                                                                i].y + 4,
                                                                COLOR_OF_STARS
                                                                );
72                                 }
73
74                                 // Calculate new position
75                                 aliens_array[i].x--;
76
77                                 // Check position
78                                 if (aliens_array[i].x <= -4) {
79                                         aliens_array[i].active = 0;
80                                         last_damage_time = cs; //
                                                    Update last fire time
81                                         (*health)--;
82                                 } else {
83                                         // Print alien
84                                         fgcolor(COLOR_OF_ALIENS);
85                                         gotoxy(aliens_array[i].x + 1,
                                                     aliens_array[i].y);
86                                         printf("O");
87                                         for (uint8_t j = 1; j < 4; j
                                                    ++) {
88                                                 gotoxy(aliens_array[i
```

```
                                                        ].x, aliens_array[
                                                        i].y + j);
89                                                  printf("O");
90                                          }
91                                          gotoxy(aliens_array[i].x + 4,
                                                aliens_array[i].y + 3);
92                                          printf("O");
93                                          gotoxy(aliens_array[i].x + 1,
                                                aliens_array[i].y + 3);
94                                          printf("   ");
95                                          gotoxy(aliens_array[i].x + 4,
                                                aliens_array[i].y);
96                                          printf(" ");
97
98                                          resetbgcolor();
99                                  }
100                         }
101                 }
102         }
103 }
104
105 void spawn_meteor(uint32_t *last_meteor_time) {
106         // Spawn delay
107         if (cs - *last_meteor_time < METEOR_SPAWN_DELAY) {
108                 return;
109         }
110         *last_meteor_time = cs; // Update last spawn time
111
112         // Create new meteor
113         for (int i = 0; i < MAX_METEORS; i++) {
114                 if (!meteors_array[i].active) {
115                         meteors_array[i].x = 243;
116                         meteors_array[i].y = (cs % 64) + 1;
117
118                         meteors_array[i].active = 1; // Activate
                                meteor
119                         break;
120                 }
121         }
122 }
123
124 void spawn_alien(uint32_t *last_spawn_time) {
125         // Spawn delay
126         if (cs - *last_spawn_time < ALIEN_SPAWN_DELAY) {
127                 return;
128         }
```

```
129         *last_spawn_time = cs; // Update last spawn time
130 
131         // Create new alien
132         for (int i = 0; i < MAX_ALIENS; i++) {
133                 if (!aliens_array[i].active) {
134                         // S t startposition
135                         aliens_array[i].x = 243;
136                         aliens_array[i].y = (cs % 70) + 1;
137 
138                         // Marker alien som aktiv
139                         aliens_array[i].active = 1;
140                         break;
141                 }
142         }
143 }
144 
145 void remove_alien(uint8_t i) { // Remove dead aliens
146         aliens_array[i].active = 0;
147         for (int j = 0; j < 4; j++) {
148                 gotoxy(aliens_array[i].x, aliens_array[i].y + j);
149                 printf("      "); // Clear width of 5 for safety
150         }
151         init_background_section(aliens_array[i].x, aliens_array[i].y,
152                         aliens_array[i].x + 5, aliens_array[i].y + 5,
153                         COLOR_OF_STARS);
154         aliens_array[i].x = 500;
155         aliens_array[i].y = 500;
156 }
```

Liste 24: Enemy.c

```
1 
2 #ifndef MENUS_GAME_MENU_H_
3 #define MENUS_GAME_MENU_H_
4 
5 #include <stm32f30x_conf.h> // STM32 config
6 #include <30010_io.h>
7 
8 #include <stdint.h>
9 #include <features/boss_key.h>
10 #include <features/bullets.h>
11 #include <features/collision_detection.h>
12 #include <features/gravity.h>
13 #include <features/led.h>
14 #include <features/movement.h>
15 #include <features/spawn_powerups.h>
16 #include <graphics/background.h>
```

```
17 #include <graphics/enemy.h>
18 #include <graphics/lcd.h>
19 #include <graphics/spaceship.h>
20 #include <graphics/title.h>
21 #include <init/joystick_reader.h>
22
23 void game_menu(void);
24 void game_over(uint8_t *nav);
25
26 #endif /* MENUS_GAME_MENU_H_ */
```

Liste 25: Gamemenu.h

```
1
2
3 #include <Menus/game_menu.h>
4
5 void game_menu(void) {
6         // Initialize game variables
7         position pos = { 243 >> 2, 76 >> 1, 3 }; // Spawn to in the
             middle to the left
8         position oldpos = { };
9         cs = 0;
10        bullet_type typ = { 0 };
11        uint32_t powerup_time = 0;
12        uint32_t last_damage_time = 0;
13        uint32_t last_spawn_time = 0;
14        uint32_t last_meteor_time = 0;
15        uint32_t last_powerup_time = 0;
16        uint8_t health = 3;
17        uint8_t alien_speed = 15;
18        lcd_init(); //Initialize the lcd display
19
20        // Initialize game features
21        init_enemy_state();
22        init_bullets();
23        init_background();
24        draw_spaceship(pos, 1);
25        lcd_starter(alien_speed);
26
27        // Game while-loop
28        while (1) {
29                // Increase alien speed
30                if (cs % 2500 == 0) {
31                        alien_speed--;
32                }
33
```

```
34                  // Game updates
35                  enemy_go(&health, &last_spawn_time, &last_meteor_time
                        ,
36                                  &last_damage_time, alien_speed);
37                  movement(&pos, &oldpos, typ);
38                  bullet_collision();
39                  spaceship_collision(pos, &health, &last_damage_time);
40                  update_bullet_pos();
41                  gravity(&pos, bullets_array);
42                  update_led(health);
43                  run_lcd(health, typ, alien_speed);
44                  spawn_powerups(&typ, powerup_time, &last_powerup_time
                        );
45                  powerup_collision(pos, &health, &typ, &powerup_time);
46                  boss_key(cs);
47
48                  // Death
49                  if (health == 0) {
50                          break;
51                  }
52          }
53  }
54
55  void game_over(uint8_t *nav) { // Game over
56          // Initialize game over
57          init_background();
58          gameover_title(50, 28);
59          gameover_text(40, 52);
60
61          // Game over while-loop
62          while (1) {
63                  if ((read_button() == 2) | (read_button() == 1)) {
64                          *nav = 0;
65                          clrscr();
66                          break;
67                  }
68          }
69  }
```

Liste 26: Gamemenu.c

```
1
2  #ifndef GRAPHICS_GAMESTART_H_
3  #define GRAPHICS_GAMESTART_H_
4
5  #include <stdlib.h>
6  #include <string.h>
```

```
7
8  #include <graphics/enemy.h>
9  #include <init/basic_functions.h>
10 #include <graphics/background.h>
11
12 gamestart(uint8_t color_of_stars, uint8_t color_of_meteor,
13                 uint8_t color_of_alien, uint8_t meteor_speed, uint8_t
                       alien_speed);
14
15 #endif
```

Liste 27: Gamestart.h

```
1
2  #include <graphics/gamestart.h>
3
4  void game_start() {
5      init_baggrund(); // Initialize the entire background
6      init_enemygo();  // Initialize enemies
7  }
8  gamestart(uint8_t color_of_stars, uint8_t color_of_meteor,
9                 uint8_t color_of_alien, uint8_t meteor_speed, uint8_t
                       alien_speed) {
10         enemygo(color_of_stars, color_of_meteor, color_of_alien,
              meteor_speed,
11                         alien_speed);
12 }
```

Liste 28: Gamestart.c

```
1
2  #ifndef FEATURES_GRAVITY_H_
3  #define FEATURES_GRAVITY_H_
4
5  #include "features/movement.h"
6  #include "features/bullets.h"
7  #include "graphics/enemy.h"
8
9  void gravity(position *pos, bullet bullets_array[MAX_BULLETS]);
10
11 #endif /* FEATURES_GRAVITY_H_ */
```

Liste 29: Gravity.h

```
1
2  #include <graphics/enemy.h>
3
4  void init_enemy_state(void) {
```

```
5           for (int i = 0; i < MAX_ALIENS; i++) {
6                   aliens_array[i].active = 0;
7           }
8           for (int i = 0; i < MAX_METEORS; i++) {
9                   meteors_array[i].active = 0;
10          }
11  }
12
13  void enemy_go(uint8_t *health, uint32_t *last_spawn_time, uint32_t *
        last_meteor_time,
14                  uint32_t *last_damage_time, uint8_t alien_speed) {
15          // METEORS
16          for (int i = 0; i < MAX_METEORS; i++) {
17                  if ((cs + 1) % 100 == 0) {
18                          if (!meteors_array[i].active) {
19                                  spawn_meteor(last_meteor_time);
20                          }
21                  }
22                  if ((cs + 3) % METEOR_SPEED == i) {
23                          if (meteors_array[i].active) {
24                                  // Delete back of meteor
25                                  if (meteors_array[i].x < 243 - 20) {
26                                          for (int j = 0; j < 10; j++)
                                                {
27                                                  gotoxy(meteors_array[
                                                        i].x + 20,
                                                        meteors_array[i].y
                                                         + j);
28                                                  printf(" ");
29                                          }
30                                          init_background_section(
                                                meteors_array[i].x + 20,
31                                                          meteors_array
                                                              [i].y,
                                                              meteors_array
                                                              [i].x +
                                                              20,
32                                                          meteors_array
                                                              [i].y +
                                                              10,
                                                              COLOR_OF_STARS
                                                              );
33                                  }
34
35                                  // Calculate new position
36                                  meteors_array[i].x--;
```

```
37
38                                  // Check position
39                                  if (meteors_array[i].x <= -20) {
40                                          meteors_array[i].active = 0;
41                                  } else {
42                                          // Print front meteor
43                                          if (meteors_array[i].x > 0) {
44                                                  fgcolor(
                                                        COLOR_OF_METEORS);
45                                                  for (uint8_t j = 0; j
                                                       < 10; j++) {
46                                                          gotoxy(
                                                             meteors_array
                                                             [i].x,
                                                             meteors_array
                                                             [i].y + j)
                                                             ;
47                                                          printf("#");
48                                                  }
49                                                  resetbgcolor();
50                                          }
51                                  }
52                          }
53                  }
54          }
55
56  // ALIENS
57          for (int i = 0; i < MAX_ALIENS; i++) {
58                  if ((cs + 5) % alien_speed == i) {
59                          if (!aliens_array[i].active) {
60                                  spawn_alien(last_spawn_time);
61                          }
62                          if (aliens_array[i].active) {
63                                  // Delete back of alien
64                                  if (aliens_array[i].x < 243 - 4) {
65                                          for (int j = 0; j < 4; j++) {
66                                                  gotoxy(aliens_array[i
                                                        ].x + 4,
                                                        aliens_array[i].y
                                                        + j);
67                                                  printf(" ");
68                                          }
69                                          init_background_section(
                                                aliens_array[i].x + 4,
70                                                          aliens_array[
                                                             i].y,
```

```
                                                        aliens_array
                                                        [i].x + 4,
71                                                  aliens_array[
                                                        i].y + 4,
                                                        COLOR_OF_STARS
                                                        );
72                                  }
73
74                                  // Calculate new position
75                                  aliens_array[i].x--;
76
77                                  // Check position
78                                  if (aliens_array[i].x <= -4) {
79                                          aliens_array[i].active = 0;
80                                          last_damage_time = cs; //
                                               Update last fire time
81                                          (*health)--;
82                                  } else {
83                                          // Print alien
84                                          fgcolor(COLOR_OF_ALIENS);
85                                          gotoxy(aliens_array[i].x + 1,
                                                aliens_array[i].y);
86                                          printf("O");
87                                          for (uint8_t j = 1; j < 4; j
                                               ++) {
88                                                  gotoxy(aliens_array[i
                                                       ].x, aliens_array[
                                                       i].y + j);
89                                                  printf("O");
90                                          }
91                                          gotoxy(aliens_array[i].x + 4,
                                                aliens_array[i].y + 3);
92                                          printf("O");
93                                          gotoxy(aliens_array[i].x + 1,
                                                aliens_array[i].y + 3);
94                                          printf("   ");
95                                          gotoxy(aliens_array[i].x + 4,
                                                aliens_array[i].y);
96                                          printf(" ");
97
98                                          resetbgcolor();
99                                  }
100                             }
101                 }
102         }
103 }
```

```
104
105 void spawn_meteor(uint32_t *last_meteor_time) {
106         // Spawn delay
107         if (cs - *last_meteor_time < METEOR_SPAWN_DELAY) {
108                 return;
109         }
110         *last_meteor_time = cs; // Update last spawn time
111
112         // Create new meteor
113         for (int i = 0; i < MAX_METEORS; i++) {
114                 if (!meteors_array[i].active) {
115                         meteors_array[i].x = 243;
116                         meteors_array[i].y = (cs % 64) + 1;
117
118                         meteors_array[i].active = 1; // Activate
                            meteor
119                         break;
120                 }
121         }
122 }
123
124 void spawn_alien(uint32_t *last_spawn_time) {
125         // Spawn delay
126         if (cs - *last_spawn_time < ALIEN_SPAWN_DELAY) {
127                 return;
128         }
129         *last_spawn_time = cs; // Update last spawn time
130
131         // Create new alien
132         for (int i = 0; i < MAX_ALIENS; i++) {
133                 if (!aliens_array[i].active) {
134                         // S t startposition
135                         aliens_array[i].x = 243;
136                         aliens_array[i].y = (cs % 70) + 1;
137
138                         // Marker alien som aktiv
139                         aliens_array[i].active = 1;
140                         break;
141                 }
142         }
143 }
144
145 void remove_alien(uint8_t i) { // Remove dead aliens
146         aliens_array[i].active = 0;
147         for (int j = 0; j < 4; j++) {
148                 gotoxy(aliens_array[i].x, aliens_array[i].y + j);
```

```
149                 printf("      "); // Clear width of 5 for safety
150         }
151         init_background_section(aliens_array[i].x, aliens_array[i].y,
152                         aliens_array[i].x + 5, aliens_array[i].y + 5,
153                         COLOR_OF_STARS);
154         aliens_array[i].x = 500;
155         aliens_array[i].y = 500;
156 }
```

Liste 30: Gravity.c

```
1
2 #ifndef MENUS_HELP_MENU_H_
3 #define MENUS_HELP_MENU_H_
4
5 void help_menu(uint8_t *nav);
6 void help_title_simple(uint8_t x, uint8_t y);
7 void help_text(uint8_t x, uint8_t y);
8
9 #endif /* MENUS_HELP_MENU_H_ */
```

Liste 31: Helpmenu.h

```
1
2
3 #include "stm32f30x_conf.h" // STM32 config
4 #include "30010_io.h"           // Input/output library for this
     course
5 #include <menus/help_menu.h>
6 #include <graphics/background.h>
7
8 void help_menu(uint8_t *nav) {
9         init_background();
10        big_planet(170,57,4,2);
11        big_planet(220,3,4,2);
12        big_planet(30,3,4,2);
13        help_title_simple(100,7);
14        help_text(20,15);
15
16
17        while (1) {
18
19                if (read_button() == 1) {
20                        *nav = 0;
21                        clrscr();
22                        break;
23                }
```

```
24          }
25
26  }
27
28  void help_title_simple(uint8_t x, uint8_t y) {
29      gotoxy(x, y);
30      printf("        _    _    _____    _        ____  \n");
31      gotoxy(x, y + 1);
32      printf("       | |  | |  | ____|  | |      |  _ \\ \n");
33      gotoxy(x, y + 2);
34      printf("       | |_| |  |  _|    | |      | |_) |\n");
35      gotoxy(x, y + 3);
36      printf("       |  _  |  | |___   | |___   |  __/ \n");
37      gotoxy(x, y + 4);
38      printf("       |_| |_|  |_____|  |_____|  |_|    \n");
39  }
40
41  void help_text(uint8_t x, uint8_t y) {
42      gotoxy(x, y);
43      printf(" _____ _              ______ _        _      _
                        _                              _        _
                                _    _
                                                    _
                                            \n");
44      gotoxy(x, y + 1);
45      printf("/  ___| |            |  ___(_)      | |    | |
           (_)                       (_)      | |                      | | (
           _)                                    | |
                                           \n");
46      gotoxy(x, y + 2);
47      printf("\\ `--.| |_ __ _ _ __| |_    _  __ _| |__ | |_ ___ _ __
                 _ ___        __ _ _ __       _ _ __ | |_ ___ _ __ __ _   ___|
           |_ ___    _____       __ _ _ __ ___ __ _   __| | ___
                                      \n");
48      gotoxy(x, y + 3);
49      printf(" `--. \\ __/ _` | '__|  _| | |/ _` | '_ \\| __/ _ \\ '__|
              | / __|     / _` | '_ \\     | | '_ \\| __/ _ \\ '__/ _` |/
           __| __| \\ \\ / / _ \\     / _` | '__/ __/ _` |/ _` |/ _ \\
                                      \n");
50      gotoxy(x, y + 4);
51      printf("/\\__/ / || (_| | |  | |    | | (_| | | | | ||  __/ |
                | \\__ \\    | (_| | | | |    | | | | | ||  __/ | | (_| | (
           __| |_| |\\ V /  __/    | (_| | | | (_| (_| | (_| |  __/
                                      \n");
52      gotoxy(x, y + 5);
53      printf("\\____/ \\__\\\__,_|_|  \\_|    |_|\\__, |_| |_|\\__\\___|_
```

```
      |      |_|___/    \\__,_|_| |_|   |_|_| |_|\\__\\___|_|  \\__,
      _|\\___|\\__|_| \\_/ \\___|    \\__,_|_|  \\___\\__,_|\\__,_
      |\\___|                          \n");
54    gotoxy(x, y + 6);
55    printf("                           __/ |

      \n");
56    gotoxy(x, y + 7);
57    printf("                          |___/

      \n");
58    gotoxy(x, y + 8);
59    printf("                              __   __
                        -
                                             -
      - -       -                      -             -   -
                  \n");
60    gotoxy(x, y + 9);
61    printf("                              \\ \\ / /
                       | |
      | |                 (_) |     | |                       (_)
                | | (_)            \n");
62    gotoxy(x, y + 10);
63    printf("  __ _  __ _ _ __ ___   ___       \\ V /___  _   _     _
      __ | | __ _ _   _      __ _ ___      __ _   ___| |_ __ _ _ __
      _ __  _| | ___ | |_     _ __   __ ___   ___  __ _  __ _| |_ _
      _ __   __ _ \n");
64    gotoxy(x, y + 11);
65    printf(" / _` |/ _` | '_ ` _ \\ / _ \\       \\ // _ \\| | | |
      | '_ \\| |/ _` | | | |    / _` / __|    / _` | / __| __/ _` |
      '__| '_ \\| | |/ _ \\| __|   | '_ \\ / _` \\ \\ / / |/ _` |/ _
      ` | __| | '_ \\ / _` |\n");
66    gotoxy(x, y + 12);
67    printf("| (_| | (_| | | | | | | |  __/_      | | (_) | |_| |   | |_
      ) | | (_| | |_| |   | (_| \\__ \\   | (_| | \\__ \\ || (_| | |
        | |_) | | | (_) | |_    | | | | (_| |\\ V /| | (_| | (_| | |
      _| | | | | (_| |\n");
68    gotoxy(x, y + 13);
69    printf(" \\__, |\\__,_|_| |_| |_|\\___(_)     \\_/\\___/ \\__,_|
        | .__/|_|\\__,_|\\__, |    \\__,_|___/    \\__,_| |___/\\__
      \\__,_|_|  | .__/|_|_|\\___/ \\__|   |_| |_|\\__,_| \\_/ |_|\\__
      , |\\__,_|\\__|_|_| |_|\\__, |\n");
70    gotoxy(x, y + 14);
71    printf("  __/ |                                          | |
                 __/ |                                         |
      |                                       __/ |
```

```
                          __/ |\n");
72      gotoxy(x, y + 15);
73      printf(" |___/                                                    |_|
                  |___/                                                    |_
           |                                          |___/
                          |___/ \n");
74
75      gotoxy(x, y + 16);
76      printf("      _                    _    _       _
                              _           _                    _        _
             _                _ _                  _
                                                \n");
77      gotoxy(x, y + 17);
78      printf("     (_)                | | (_)     | |
                           | |         | |                | |      | | |
            |                   | (_)                    | |
                                                \n");
79      gotoxy(x, y + 18);
80      printf("      _  ___  _   _ ___| |_ _  ___| | __      __ _ _ __
           __| |    ___| |__   ___   ___ | |_    | |_| |__   ___       __
           _| |_  ___ _ __  ___    | |__  _   _
                                    \n");
81      gotoxy(x, y + 19);
82      printf("     | |/ _ \\| | | / __| __| |/ __| |/ /    / _‘ | ’_ \\
           / _‘ |   / __| ’_ \\ / _ \\ / _ \\| __|   | __| ’_ \\ / _ \\
              / _‘ | | |/ _ \\ ’_ \\/ __|   | ’_ \\| | | |
                                    \n");
83      gotoxy(x, y + 20);
84      printf("     | | (_) | |_| \\__ \\ |_| | (__|   <    | (_| | | | |
            (_| |   \\__ \\ | | | (_) | (_) | |_    | |_| | | |  __/   |
           (_| | | |  __/ | | \\__ \\   | |_) | |_| |
                                    \n");
85      gotoxy(x, y + 21);
86      printf("     | |\\___/ \\__, |___/\\__|_|\\___|_|\\_\\    \\__,_|_
           | |_|\\__,_|   |___/_| |_|\\___/ \\___/ \\__|    \\__|_| |_|\\
           ___|    \\__,_|_|_|\\___|_| |_|___/   |_.__/ \\__, |
                                    \n");
87      gotoxy(x, y + 22);
88      printf("    _/ |       __/ |

           __/ |                          \n");
89      gotoxy(x, y + 23);
90      printf(" |__/       |___/

           |___/                           \n");
91      gotoxy(x, y + 24);
```

```
92      printf("                                    _                 _    _
                          _      _ _             _             _    _
                         _____         _ _             _
                                      \n");
93      gotoxy(x, y + 25);
94      printf("                                  (_)               | | | |
                        | |    (_) |           | |           | | | |
                       /  __ \\      | | |           | |
                                     \n");
95      gotoxy(x, y + 26);
96      printf("  _ __   _ __ ___   ___ ___ _ _ __    __ _     | |_| |__
           ___   __        _| |__   _| |_ ___     | |__   _    _| |_| |_ ___   _
           __         | /  \\/ ___ | | | ___   ___| |_
                                    \n");
97      gotoxy(x, y + 27);
98      printf(" | '_ \\| '__/ _ \\/ __/ __| | '_ \\ / _` |    | __| '_ \\
            / _ \\ \\ \\ /\\ / / '_ \\| | __/ _ \\    | '_ \\| | | | __|
           __/ _ \\| '_ \\       | |     / _ \\| | |/ _ \\/ __| __|
                                   \n");
99      gotoxy(x, y + 28);
100     printf(" | |_) | | |  __/\\__ \\__ \\ | | | | (_| |    | |_| | | | |
             __/  \\ V  V /| | | | | ||  __/    | |_) | |_| | |_| || (_) |
            | | |_     | \\__/\\ (_) | | |  __/ (__| |_
                                   \n");
101     gotoxy(x, y + 29);
102     printf(" | .__/|_|  \\___||___/___/_|_| |_|\\__, |     \\__|_| |_
           |\\___|    \\_/\\_/ |_| |_|_|\\__\\___|    |_.__/ \\__,_|\\__|\\
           __\\___/|_| |_(_)     \\____/\\___/|_|_|\\___|\\___|\\__|
                                   \n");
103     gotoxy(x, y + 30);
104     printf(" | |                                  __/ |

           \n");
105     gotoxy(x, y + 31);
106     printf(" |_|                                 |___/

           \n");
107
108     gotoxy(x, y + 32);
109     printf("

           _                                 _    _
                                  _      _             _    _
                           \n");
110     gotoxy(x, y + 33);
111     printf("
```

```
      | |                                  | | | |
                                 | |   | |         | | | |
                        \n");
112     gotoxy(x, y + 34);
113     printf("  _ __   _____       _____ _ __ ______ _   _ _ __  ___
             __ _ _ __   __| |    _ __  _ __ ___  ___ ___    | |_| |__
          ___     _ __ ___  __| |   | |__  _   _| |_| |_ ___  _ __
           \n");
114     gotoxy(x, y + 35);
115     printf(" | '_ \\ / _ \\ \\ /\\ / / _ \\ '__|______| | | | '_ \\/
          __|    / _` | '_ \\ / _` |   | '_ \\| '__/ _ \\/ __/ __|   |
          __| '_ \\ / _ \\   | '__/ _ \\/ _` |   | '_ \\| | | | __| __/
          _ \\| '_ \\    \n");
116     gotoxy(x, y + 36);
117     printf(" | |_) | (_) \\ V  V /  __/ |         | |_| | |_) \\__ \\
             | (_| | | | | (_| |   | |_) | | |  __/\\__ \\__ \\   | |_|
          | | |  __/   | | |  __/ (_| |   | |_) | |_| | |_| || (_) | | |
           |   \n");
118     gotoxy(x, y + 37);
119     printf(" | .__/ \\___/ \\_/\\_/ \\___|_|          \\__,_| .__/|
          ___/    \\__,_|_| |_|\\__,_|   | .__/|_|  \\___||___/___/
          \\__|_| |_|\\___|   |_|  \\___|\\__,_|   |_.__/ \\__,_|\\__|\\
          __\\___/|_| |_|   \n");
120     gotoxy(x, y + 38);
121     printf(" | |                                        | |
                                      | |


          \n");
122     gotoxy(x, y + 39);
123     printf(" |_|                                       |_|
                                      |_|


          \n");
124     gotoxy(x, y + 40);
125     printf("  _                          _   _           _           _
            -               -                       -


          \n");
126     gotoxy(x, y + 41);
127     printf(" | |                       | | (_)         | |         | |
           | |            | |                   | |


          \n");
128     gotoxy(x, y + 42);
129     printf(" | |_ ___       __ _  ___| |_ ___   ____ _| |_ ___    | |
```

```
        _| |__   ___     | |__   ___  ___ ___     | | _____ _   _
                              \n");
130     gotoxy(x, y + 43);
131     printf(" | __/ _ \\     / _` |/ __| __| \\ \\ / / _` | __/ _ \\
          | __| '_ \\ / _ \\   | '_ \\ / _ \\/ __/ __|   | |/ / _ \\ |
         | |             \n");
132     gotoxy(x, y + 44);
133     printf(" | || (_) |   | (_| | (__| |_| |\\ V / (_| | ||  __/   |
           |_| | | |  __/   | |_) | (_) \\__ \\__ \\   |   <  __/ |_| |_
                         \n");
134     gotoxy(x, y + 45);
135     printf("  \\__\\___/     \\__,_|\\___|\\__|_| \\_/ \\__,_|\\__\\
           ___|    \\__|_| |_|\\___|   |_.__/ \\___/|___/___/   |_|\\_\\
           ___|\\__, (_)    \n");
136     gotoxy(x, y + 46);
137     printf("

           __/ |                          \n");
138     gotoxy(x, y + 47);
139     printf("

           |___/                          \n");
140 }
```

Liste 32: Helpmenu.c

```
1
2   #ifndef INIT_INIT_GPIO_H_
3   #define INIT_INIT_GPIO_H_
4
5   #include <stm32f30x_conf.h> // STM32 config
6   #include <30010_io.h> // Input/output library for this course
7   #include <init/joystick_reader.h>
8
9   void configure_pins(void);
10  void configure_joystick(void);
11  void configure_led(void);
12  void set_led(uint8_t R, uint8_t G, uint8_t B);
13
14  #endif
```

Liste 33: init GPIO.h

```
1
2   #include <init/init_GPIO.h>
3
4   void configure_pins(void) {
```

```
5           // Enable clock for GPIO Port A
6           RCC->AHBENR |= RCC_AHBENR_GPIOAEN;
7
8           // Set pin PA0 to input
9           GPIOA->MODER &= ~(0x3 << (0 * 2)); // Clear mode register for
                PA0
10          GPIOA->PUPDR &= ~(0x3 << (0 * 2)); // Clear pull-up/pull-down
                register for PA0
11          GPIOA->PUPDR |= (0x2 << (0 * 2));  // Set pull-down for PA0
12  }
13
14  void configure_joystick(void) {
15          // Enable clocks for GPIO ports
16          RCC->AHBENR |= RCC_AHBENR_GPIOAEN | RCC_AHBENR_GPIOBEN |
                RCC_AHBENR_GPIOCEN;
17
18          // Set joystick pins to input mode
19          GPIOC->MODER &= ~(0x3 << (0 * 2)); // PC0 (Right)
20          GPIOA->MODER &= ~(0x3 << (4 * 2)); // PA4 (Up)
21          GPIOB->MODER &= ~(0x3 << (5 * 2)); // PB5 (Center)
22          GPIOC->MODER &= ~(0x3 << (1 * 2)); // PC1 (Left)
23          GPIOB->MODER &= ~(0x3 << (0 * 2)); // PB0 (Down)
24
25          // Configure pull-down resistors for joystick pins
26          GPIOC->PUPDR |= (0x2 << (0 * 2)); // PC0
27          GPIOA->PUPDR |= (0x2 << (4 * 2)); // PA4
28          GPIOB->PUPDR |= (0x2 << (5 * 2)); // PB5
29          GPIOC->PUPDR |= (0x2 << (1 * 2)); // PC1
30          GPIOB->PUPDR |= (0x2 << (0 * 2)); // PB0
31  }
32
33  uint8_t mcu_read_joystick(void) {
34          uint8_t state = 0;
35
36          // Read joystick inputs and set bits accordingly
37          state |= (GPIOA->IDR & (1 << 4)) ? (1 << 0) : 0; // Up
38          state |= (GPIOB->IDR & (1 << 0)) ? (1 << 1) : 0; // Down
39          state |= (GPIOC->IDR & (1 << 1)) ? (1 << 2) : 0; // Left
40          state |= (GPIOC->IDR & (1 << 0)) ? (1 << 3) : 0; // Right
41          state |= (GPIOB->IDR & (1 << 5)) ? (1 << 4) : 0; // Center
42
43          return state;
44  }
45
46  void configure_led(void) {
47          // Enable clocks for GPIO ports
```

```
48          RCC->AHBENR |= RCC_AHBENR_GPIOAEN | RCC_AHBENR_GPIOBEN |
               RCC_AHBENR_GPIOCEN;
49
50          // Configure PB4 (Red)
51          GPIOB->MODER &= ~(0x3 << (4 * 2)); // Clear mode bits
52          GPIOB->MODER |= (0x1 << (4 * 2)); // Set PB4 as output
53
54          // Configure PC7 (Green)
55          GPIOC->MODER &= ~(0x3 << (7 * 2)); // Clear mode bits
56          GPIOC->MODER |= (0x1 << (7 * 2)); // Set PC7 as output
57
58          // Configure PA9 (Blue)
59          GPIOA->MODER &= ~(0x3 << (9 * 2)); // Clear mode bits
60          GPIOA->MODER |= (0x1 << (9 * 2)); // Set PA9 as output
61  }
62
63  void set_led(uint8_t R, uint8_t G, uint8_t B) {
64          // Turn off all LEDs
65          GPIOB->ODR |= (1 << 4); // Turn off Red
66          GPIOC->ODR |= (1 << 7); // Turn off Green
67          GPIOA->ODR |= (1 << 9); // Turn off Blue
68
69          // Turn on LEDs based on arguments
70          if (R) {
71                  GPIOB->ODR &= ~(1 << 4); // Turn on Red
72          }
73          if (G) {
74                  GPIOC->ODR &= ~(1 << 7); // Turn on Green
75          }
76          if (B) {
77                  GPIOA->ODR &= ~(1 << 9); // Turn on Blue
78          }
79  }
```

Liste 34: init GPIO.c

```
1
2  #ifndef INIT_JOYSTICK_READER_H_
3  #define INIT_JOYSTICK_READER_H_
4
5  #include <stm32f30x_conf.h> // STM32 config
6  #include <30010_io.h> // Input/output library for this course
7  #include <stm32f30x.h> // Replace with your specific device header if
       different
8  #include <stm32f30x_gpio.h>
9  #include <stm32f30x_rcc.h>
10
```

```
11 void configure_adv_joystick(void);
12 int read_adv_joystick_x(void);
13 int read_adv_joystick_y(void);
14 int read_joystick(void);
15
16 #endif
```

Liste 35: Joystick reader.h

```
1
2  #include <init/joystick_reader.h>
3  #include <stdio.h>
4  #include <stm32f30x.h>
5  #include <stm32f30x.h> // Include the STM32 device-specific headers
6
7
8  void configure_adv_joystick(void) {
9
10         // Enable clock for GPIO Port C
11         RCC->AHBENR |= RCC_AHBENR_GPIOCEN;
12
13         RCC->CFGR2  &= ~RCC_CFGR2_ADCPRE12;      // Clear ADC12
               prescaler bits
14         RCC->CFGR2  |=  RCC_CFGR2_ADCPRE12_DIV6; // Set ADC12
               prescaler to 6
15         RCC->AHBENR |=  RCC_AHBPeriph_ADC12;     // Enable clock for
               ADC12
16
17         ADC1->CR    =  0x00000000; // Clear CR register
18         ADC1->CFGR &=  0xFDFFC007; // Clear ADC1 config register
19         ADC1->SQR1 &= ~ADC_SQR1_L; // Clear regular sequence register
               1
20
21         ADC1->CR |= 0x10000000; // Enable internal ADC voltage
               regulator
22         for(int i = 0 ; i < 1000 ; i++) {} // Wait for about 16
               microsecond
23
24         ADC1->CR |= 0x80000000; // Start ADC1 calibration
25         while (!(ADC1->CR & 0x80000000));// Wait for calibration to
               finish
26         for (int i = 0 ; i < 100 ; i++) {} // Wait for a little whil
27
28         ADC1->CR |= 0x00000001; // Enable ADC1 (0x01 -Enable, 0x02 -
               Disable)
29         while (!(ADC1->ISR & 0x00000001));// Wait until ready
30
```

```
31
32          GPIOC->MODER&= ~(0x00000003 << (2 * 2));
33          GPIOC->MODER|=  (0x00000000 << (2 * 2));
34
35          GPIOC->MODER&= ~(0x00000003 << (3 * 2));
36          GPIOC->MODER|=  (0x00000000 << (3 * 2));
37
38          GPIOC->PUPDR &= ~(0x3 << (2 * 2)); // Clear pull-up/pull-down
                register for PC2
39          GPIOC->PUPDR |= (0x2 << (2 * 2));  // Enable pull-down for
               PC2
40
41          GPIOC->PUPDR &= ~(0x3 << (3 * 2)); // Clear pull-up/pull-down
                register for PC3
42          GPIOC->PUPDR |= (0x2 << (3 * 2));  // Enable pull-down for
               PC3
43
44          //buttons setup
45
46          // Set PC10 and PC12 to input mode
47          GPIOC->MODER &= ~(0x3 << (10 * 2)); // Clear mode register
               for PC10
48          GPIOC->MODER &= ~(0x3 << (12 * 2)); // Clear mode register
               for PC12
49
50          // Configure pull-down resistors for PC10 and PC12
51          GPIOC->PUPDR &= ~(0x3 << (10 * 2)); // Clear pull-up/pull-
               down register for PC10
52          GPIOC->PUPDR |= (0x2 << (10 * 2));  // Enable pull-down for
               PC10
53
54          GPIOC->PUPDR &= ~(0x3 << (12 * 2)); // Clear pull-up/pull-
               down register for PC12
55          GPIOC->PUPDR |= (0x2 << (12 * 2));  // Enable pull-down for
               PC12
56
57  }
58
59  int read_adv_joystick_y(void) {  // L ser joystick i y-retningen
60          ADC_RegularChannelConfig(ADC1, ADC_Channel_8, 1,
               ADC_SampleTime_1Cycles5);
61          ADC_StartConversion(ADC1); // Start ADC read
62          while (ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC) == 0); // Wait
               for ADC read
63
64          return ADC_GetConversionValue(ADC1);// Read the ADCvalue
```

```
65 }
66
67 int read_adv_joystick_x(void) {  //L ser joystick i x retningen
68         ADC_RegularChannelConfig(ADC1, ADC_Channel_9, 1,
              ADC_SampleTime_1Cycles5);
69         ADC_StartConversion(ADC1); // Start ADC read
70         while (ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC) == 0); // Wait
              for ADC read
71
72         return ADC_GetConversionValue(ADC1);// Read the ADCvalue
73 }
74
75 int read_joystick(void) {  //Denne funktion outputter joystick
      inputne i 8 forskellige retninger (efter uret)
76         int y = read_adv_joystick_y();
77         int x = read_adv_joystick_x();
78         int old_value = 0;
79         int new_value = 0;
80         if ((1800 < x) & (x < 2000) & (1800 < y) & (y < 2000)) {
81                 new_value = 0;
82         }
83         if ((1700 < x) & (x < 2200) & (2900 < y)) {
84                 new_value = 8;
85         }
86         if ((3000 < x) & (1700 < y) & (y < 2200)) {
87                 new_value = 2;
88         }
89         if ((1700 < x) & (x < 2100) & (y < 1100)) {
90                 new_value = 4;
91         }
92         if ((x < 1100) & (1700 < y) & (y < 2200)) {
93                 new_value = 6;
94         }
95         if ((2100 < x) & (2500 < y)) {
96                 new_value = 1;
97         }
98         if ((2200 < x) & (y < 1600)) {
99                 new_value = 3;
100        }
101        if ((x < 1500) & (y < 1500)) {
102                new_value = 5;
103        }
104        if ((x < 1500) & (2500 < y)) {
105                new_value = 7;
106        }
107
```

```
108         if (new_value != old_value) {
109                 old_value = new_value;
110                 return new_value;
111         }
112 }
113
114 int read_button(void) {         // Returnerer inputten fra de 2
    knapper
115     if (GPIOC->IDR & (1 << 12)) { // Check if PC12 is high
116         return 1; // Button on PC12 pressed
117     }
118     if (GPIOC->IDR & (1 << 10)) { // Check if PC10 is high
119         return 2; // Button on PC10 pressed
120     }
121     else {
122         return 0; // No button pressed
123     }
124 }
```

Liste 36: Joystick reader.c

```
1  #ifndef GRAPHICS_LCD_H_
2  #define GRAPHICS_LCD_H_
3
4  #include <30010_io.h>
5  #include <string.h>
6  #include <charset.h>
7  #include <init/timer.h>
8  #include <stdint.h>
9  #include <features/movement.h>
10 #include <features/bullets.h>
11 #include <Menus/game_menu.h>
12
13 void heart(int start);
14 void remove_heart(int start);
15 void lcd_write_string(const char *str, int start);
16 void points_counter(int start);
17 void shooting_modes_lcd(void);
18 void point_loop(void);
19 void shooting_mode_box(bullet_type typ);
20 void remove_shooting_mode_box(bullet_type typ);
21 void hearts_left(uint8_t health);
22
23 void lcd_starter(void);
24
25 void run_lcd(uint8_t health, bullet_type typ);
26
```

```
27 #endif
```

Liste 37: LCD.h

```
1  #include <graphics/lcd.h>
2
3  uint8_t lcd_buffer[512]; // Declares the buffer
4
5  void heart(int start) {
6          lcd_buffer[start] = 0x00; //Blank strip before printing of
               heart
7          lcd_buffer[start + 1] = 0x06;
8          lcd_buffer[start + 2] = 0x0F;
9          lcd_buffer[start + 3] = 0x1F;
10         lcd_buffer[start + 4] = 0x3F;
11         lcd_buffer[start + 5] = 0x7E;
12         lcd_buffer[start + 6] = 0xFC;
13         lcd_buffer[start + 7] = 0x7E;
14         lcd_buffer[start + 8] = 0x3F;
15         lcd_buffer[start + 9] = 0x1F;
16         lcd_buffer[start + 10] = 0x0F;
17         lcd_buffer[start + 11] = 0x06;
18 }
19
20 void lcd_write_string(const char *str, int start) { // Takes string
       and start position as inputs
21         while (*str) {
22                 uint8_t c = *str++;
23                 c -= 32;
24
25                 for (int i = 0; i < 5; i++) {
26                         lcd_buffer[start + i] = character_data[c][i];
27                 }
28                 start += 6;
29         }
30 }
31
32 void remove_heart(int start) {
33
34         lcd_buffer[start] = 0x00; //Blank strip before printing of
               heart
35         lcd_buffer[start + 1] = 0x06;
36         lcd_buffer[start + 2] = 0x09;
37         lcd_buffer[start + 3] = 0x11;
38         lcd_buffer[start + 4] = 0x21;
39         lcd_buffer[start + 5] = 0x42;
40         lcd_buffer[start + 6] = 0x84;
```

```
41          lcd_buffer[start + 7] = 0x42;
42          lcd_buffer[start + 8] = 0x21;
43          lcd_buffer[start + 9] = 0x11;
44          lcd_buffer[start + 10] = 0x09;
45          lcd_buffer[start + 11] = 0x06;
46  }
47
48  void shooting_modes_lcd(void) {
49
50          lcd_write_string("Shotgun", 384 + 1);
51          lcd_write_string("Normal", 384 + 43 + 4);
52          lcd_write_string("Laser", 511 - 43 + 8);
53
54  }
55
56  void shooting_mode_box(bullet_type typ) {
57
58          uint16_t lines[] = { 256, 384, 256 + 43, 384 + 43, 383 - 43,
               511 - 43, 383,
59                               511 };
60
61          if (typ.type == 1) { // Shotgun
62                  lcd_buffer[lines[0]] = 0xFF; //First line
63                  lcd_buffer[lines[1]] = 0xFF;
64
65                  lcd_buffer[lines[2]] = 0xFF; //Second line
66                  lcd_buffer[lines[3]] = 0xFF;
67
68          }
69          if (typ.type == 0) { //Normal
70                  lcd_buffer[lines[2]] = 0xFF; //Second line
71                  lcd_buffer[lines[3]] = 0xFF;
72
73                  lcd_buffer[lines[4]] = 0xFF; //Third line
74                  lcd_buffer[lines[5]] = 0xFF;
75
76          }
77          if (typ.type == 2) { // Laser
78                  lcd_buffer[lines[4]] = 0xFF; //Third line
79                  lcd_buffer[lines[5]] = 0xFF;
80
81                  lcd_buffer[lines[6]] = 0xFF; //Fourth line
82                  lcd_buffer[lines[7]] = 0xFF;
83          }
84  }
85
```

```
86  void remove_shooting_mode_box(bullet_type typ) {
87          uint16_t lines[] = { 256, 384, 256 + 43, 384 + 43, 383 - 43,
               511 - 43, 383,
88                               511 };
89          if (typ.type != 1) {
90                  lcd_buffer[lines[0]] = 0x00; //First line
91                  lcd_buffer[lines[1]] = 0x00;
92          }
93
94          if ((typ.type != 1) && (typ.type != 0)) {
95                  lcd_buffer[lines[2]] = 0x00; //Second line
96                  lcd_buffer[lines[3]] = 0x00;
97          }
98
99          if ((typ.type != 2) && (typ.type != 0)) {
100                 lcd_buffer[lines[4]] = 0x00; //Third line
101                 lcd_buffer[lines[5]] = 0x00;
102         }
103         if (typ.type != 2) {
104                 lcd_buffer[lines[6]] = 0x00; //Fourth line
105                 lcd_buffer[lines[7]] = 0x00;
106         }
107 }
108
109 void points_counter(int start) {
110         char points_to_string[6]; // Points can print up to 5
               decimals + 1 for /0
111
112         if (cs % (rand() % 200) == 0){//Randomizes the points
113                 snprintf(points_to_string, sizeof(points_to_string),
                       "%d", cs); // Converts points to string
114                 lcd_write_string(points_to_string, 256 - 40);
115         }
116 }
117
118
119
120
121 void hearts_left(int health){
122         if (health == 3) {
123                 heart(128);
124                 heart(128 + 12);
125                 heart(128 + 24);
126         }
127         if (health == 2) {
128                 heart(128);
```

```
129                 heart(128+12);
130                 remove_heart(128 + 24);
131         }
132         if (health == 1) {
133                 heart(128);
134                 remove_heart(128 + 12);
135                 remove_heart(128 + 24);
136         }
137         if (health == 0) {
138                 remove_heart(128);
139                 remove_heart(128 + 12);
140                 remove_heart(128 + 24);
141         }
142 }
143
144 void point_loop(void) { //Loop makes the lcd update, including the
        points
145
146                 points_counter(128 - 40);
147                 lcd_push_buffer(lcd_buffer);
148
149 }
150
151 void lcd_starter(void) { //Initializes the LCD additions
152
153         memset(lcd_buffer, 0x00, 512); //Clears LCD screen
154         heart(128); //Draw Hearts
155         heart(128 + 12);
156         heart(128 + 24);
157
158         lcd_write_string("Lives:", 1);
159         lcd_write_string("Points:", 128 - 40);
160         points_counter(256 - 40); //Remember to write location and
            mode
161         shooting_modes_lcd();
162
163         lcd_push_buffer(lcd_buffer); //Push to lcd
164 }
165
166 void run_lcd(int health, bullet_type typ) {
167         hearts_left(health);
168         point_loop();
169         shooting_mode_box(typ);
170         remove_shooting_mode_box(typ);
171 }
```

Liste 38: LCD.c

```

#ifndef FEATURES_LED_H_
#define FEATURES_LED_H_

#include <init/init_GPIO.h>

void update_led(int health);

#endif
```

Liste 39: Led.h

```

#include <features/led.h>

void update_led(int health) {
        switch (health) {
        case 0:
                set_led(0, 0, 0);
                break;
        case 1:
                set_led(1, 0, 0);
                break;
        case 2:
                set_led(0, 0, 1);
                break;
        case 3:
                set_led(0, 1, 0);
                break;
        }
}
```

Liste 40: Led.c

```

#ifndef MENUS_MAIN_MENU_H_
#define MENUS_MAIN_MENU_H_

#include <stdint.h>

void windows(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2, char
    str[],
                uint8_t z);
void main_menu(uint8_t *nav);

#endif /* MENUS_MAIN_MENU_H_ */
```

Liste 41: Mainmenu.h

```
#define ESC 0x1B
#include <stdint.h>

void main_menu(uint8_t *nav) {

        init_baggrund_menu();
        title_header(5, 10);

        start_title(243 / 3 - 18, 55);
        help_title(243 * 2 / 3 - 17, 55);

        windows(243 / 3 - 20, 55 - 2, 243 / 3 + 20, 60 + 2, 0);
        windows(243 * 2 / 3 - 19, 55 - 2, 243 * 2 / 3 + 19, 60 + 2,
           0);

        int z = 2;
        while (1) {

                if (read_joystick() == 2) {
                        z = z - 1;
                }
                if (read_joystick() == 6) {
                        z = z + 1;
                }
                if (z < 1) {
                        z = 1;
                }
                if (z > 2) {
                        z = 2;
                }

                if (z == 1) {
                        gotoxy(243 / 3 - 20, 63);
                        printf("
                                                                    ")
                           ;
                        gotoxy(243 * 2 / 3 - 19, 63);
                        printf("
                           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^");

                }
                if (z == 2) {
                        gotoxy(243 * 2 / 3 - 19, 63);
```

```
40                          printf("
                                                                    ")
                                ;
41                          gotoxy(243 / 3 - 20, 63);
42                          printf("
                                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^"
                                );
43                  }
44                  if ((z == 2) & (read_button() == 2)) {
45                          //Launch game
46                          *nav = 2;
47                          clrscr(1);
48                          break;
49                  }
50                  if ((z == 1) & (read_button() == 2)) {
51                          //Launch Help
52                          *nav = 1;
53                          clrscr(1);
54                          break;
55                  }
56          }
57
58  }
59
60  void windows(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2, char
        *str,
61                  uint8_t z) {
62          if (z == 1) {
63                  printf("%c[05m", ESC);
64          }
65
66          gotoxy(x1, y1); // Move to the starting position
67          int i;
68          for (i = 1; i < y2 - y1; i++) {
69                  gotoxy(x1, y1 + i);             // Left side
70                  printf("%c", 179);
71
72                  gotoxy(x2, y1 + i);         // Right side
73                  printf("%c", 179);
74          }
75
76          // Draw top and bottom sides
77          for (i = 1; i < x2 - x1; i++) {
78                  gotoxy(x1 + i, y1);             // Top side
79                  printf("%c", 196);
80
```

```
81                  gotoxy(x1 + i, y2);         // Bottom side
82                  printf("%c", 196);
83          }
84          // Draw corners
85          gotoxy(x1, y1);                     // Top-left corner
86          printf("%c", 218);                  // Top-left corner
               character
87
88          gotoxy(x2, y1);                 // Top-right corner
89          printf("%c", 191);                  // Top-right corner
               character
90
91          gotoxy(x1, y2);                  // Bottom-left corner
92          printf("%c", 192);                  // Bottom-left corner
               character
93
94          gotoxy(x2, y2);             // Bottom-right corner
95          printf("%c", 217);                  // Bottom-right corner
               character
96
97          if (z == 1) {
98                  printf("%c[00m", ESC);
99          }
100 }
```

Liste 42: Mainmenu.c

```
1
2   #ifndef FEATURES_MOVEMENT_H_
3   #define FEATURES_MOVEMENT_H_
4
5   #include <stdint.h>
6   #include <stdio.h>
7
8   typedef struct {
9           uint8_t x;
10          uint8_t y;
11          uint8_t direction;
12  } position;
13
14  typedef struct {
15          uint8_t type;
16  } bullet_type;
17
18  #include <init/timer.h>
19  #include <init/init_GPIO.h>
20  #include <init/basic_functions.h>
```

```
21 #include <features/bullets.h>
22 #include <init/joystick_reader.h>
23 #include <graphics/spaceship.h>
24 #include <graphics/background.h>
25
26 void movement(position *pos, position *oldpos, bullet_type typ);
27 void set_pos_and_dir(position *pos, position *oldpos, bullet_type typ
   ,
28                 uint8_t joystick_type, uint8_t buffer);
29
30 #endif
```

Liste 43: Movement.h

```
1
2 #include <features/movement.h>
3
4 void movement(position *pos, position *oldpos, bullet_type typ) {
5         if ((cs + 10) % 7 == 0) {
6                 uint8_t buffer = 0;
7                 // Remove old spaceship
8                 if ((pos->x != oldpos->x) || (pos->y != oldpos->y)) {
9                         switch (oldpos->direction) {
10                        case 1:
11                        case 2:
12                        case 3:
13                        case 5:
14                        case 6:
15                        case 7:
16                                for (int i = 0; i < 4; i++) {
17                                        gotoxy(oldpos->x, oldpos->y +
                                              i);
18                                        printf("      ");
19                                }
20                                init_background_section(oldpos->x,
                                     oldpos->y, oldpos->x + 6,
21                                                oldpos->y + 4, 11);
22                                break;
23                        case 4:
24                        case 8:
25                                for (int i = 0; i < 4; i++) {
26                                        gotoxy(oldpos->x, oldpos->y +
                                              i);
27                                        printf("       ");
28                                }
29                                init_background_section(oldpos->x,
                                     oldpos->y, oldpos->x + 7,
```

```
30                                                      oldpos->y + 4, 11);
31                                      break;
32                              }
33                              oldpos->x = pos->x;
34                              oldpos->y = pos->y;
35                              oldpos->direction = pos->direction;
36                              buffer = 1;
37                      }
38
39                      // Spaceship position and set direction
40                      set_pos_and_dir(pos, oldpos, typ, 1, buffer);
41              }
42      }
43
44      void set_pos_and_dir(position *pos, position *oldpos, bullet_type typ
            ,
45                      uint8_t joystick_type, uint8_t buffer) {
46              if (joystick_type == 0) {
47                      switch (mcu_read_joystick()) {
48                      case 9:
49                              pos->x += 2;
50                              pos->y--;
51                              pos->direction = 1;
52                              break;
53                      case 8: // Right
54                              pos->x += 2;
55                              pos->direction = 2;
56                              break;
57                      case 10:
58                              pos->x += 2;
59                              pos->y++;
60                              pos->direction = 3;
61                              break;
62                      case 2: // Down
63                              pos->y++;
64                              pos->direction = 4;
65                              break;
66                      case 6:
67                              pos->x -= 2;
68                              pos->y++;
69                              pos->direction = 5;
70                              break;
71                      case 4: // Left
72                              pos->x -= 2;
73                              pos->direction = 6;
74                              break;
```

```
75                  case 5:
76                          pos->x -= 2;
77                          pos->y--;
78                          pos->direction = 7;
79                          break;
80                  case 1: // Up
81                          pos->y--;
82                          pos->direction = 8;
83                          break;
84                  }
85
86                  // Print new spaceship
87                          draw_spaceship(*pos, buffer);
88
89                  if (mcu_read_joystick() == 16) { // center
90                          fire_bullet(&*pos, typ);
91                  }
92          }
93
94          if (joystick_type == 1) {
95                  switch (read_joystick()) {
96                  case 1:
97                          pos->x += 2;
98                          pos->y--;
99                          pos->direction = 1;
100                         break;
101                 case 2: // Right
102                         pos->x += 2;
103                         pos->direction = 2;
104                         break;
105                 case 3:
106                         pos->x += 2;
107                         pos->y++;
108                         pos->direction = 3;
109                         break;
110                 case 4: // Down
111                         pos->y++;
112                         pos->direction = 4;
113                         break;
114                 case 5:
115                         pos->x -= 2;
116                         pos->y++;
117                         pos->direction = 5;
118                         break;
119                 case 6: // Left
120                         pos->x -= 2;
```

```
121                         pos->direction = 6;
122                         break;
123                 case 7:
124                         pos->x -= 2;
125                         pos->y--;
126                         pos->direction = 7;
127                         break;
128                 case 8: // Up
129                         pos->y--;
130                         pos->direction = 8;
131                         break;
132                 }
133
134                 // Print new spaceship
135                 draw_spaceship(*pos, buffer);
136
137                 if (read_button() == 2) {
138                         fire_bullet(&*pos, typ);
139                 }
140         }
141 }
```

Liste 44: Movement.c

```
1
2  #ifndef GRAPHICS_POWERUP_H_
3  #define GRAPHICS_POWERUP_H_
4
5  #include <stdint.h>
6  #include <stdio.h>
7  #include <stdlib.h>
8  #include <string.h>
9
10 #include <init/basic_functions.h>
11 #include <graphics/background.h>
12 #include <init/timer.h>
13 #include <graphics/enemy.h>
14 #include <graphics/ansi.h>
15 #include <features/movement.h>
16
17 void drawwindow(uint8_t c);
18 void powerup(uint8_t color);
19
20 #endif /* GRAPHICS_POWERUP_H_ */
```

Liste 45: Powerups.h

```
1
2  #include <graphics/powerup.h>  // Inkluder headeren for grafikdrivere
3  #include <init/basic_functions.h>
4
5  void powerup(uint8_t color) {
6          fgcolor(color);
7
8          // Beregn midten af vinduet (fast st rrelse 16x8)
9          int center_x = powerups_array[0].x + 8;  // Halvdelen af
               bredden
10         int center_y = powerups_array[0].y + 4;  // Halvdelen af
               h jden
11         drawwindow(color);
12         switch (powerups_array[0].powerup_effect) {
13         case 0: //Shotgun
14                 gotoxy(center_x, center_y + 2);
15                 printf("%c", 92);
16                 gotoxy(center_x, center_y);
17                 printf("%c", 45);
18                 gotoxy(center_x + 1, center_y);
19                 printf("%c", 62);
20                 gotoxy(center_x, center_y - 2);
21                 printf("%c", 47);
22
23                 break;
24
25         case 1: //Laser
26                 gotoxy(center_x - 3, center_y);
27                 printf("%c", 45);
28                 gotoxy(center_x - 2, center_y);
29                 printf("%c", 45);
30                 gotoxy(center_x - 1, center_y);
31                 printf("%c", 45);
32                 gotoxy(center_x, center_y);
33                 printf("%c", 45);
34                 gotoxy(center_x + 1, center_y);
35                 printf("%c", 45);
36                 gotoxy(center_x + 2, center_y);
37                 printf("%c", 45);
38                 gotoxy(center_x + 3, center_y);
39                 printf("%c", 45);
40
41                 break;
42
43         case 2: //hjerte
44                 gotoxy(center_x, center_y - 2);
```

```
45                  printf("%c", 47);
46                  gotoxy(center_x - 1, center_y - 2);
47                  printf("%c", 92);
48                  gotoxy(center_x - 2, center_y - 3);
49                  printf("%c", 95);
50                  gotoxy(center_x + 1, center_y - 3);
51                  printf("%c", 95);
52                  gotoxy(center_x + 2, center_y - 2);
53                  printf("%c", 92);
54                  gotoxy(center_x - 3, center_y - 2);
55                  printf("%c", 47);
56                  gotoxy(center_x - 3, center_y - 1);
57                  printf("%c", 92);
58                  gotoxy(center_x - 2, center_y);
59                  printf("%c", 92);
60                  gotoxy(center_x - 1, center_y + 1);
61                  printf("%c", 92);
62                  gotoxy(center_x, center_y + 1);
63                  printf("%c", 47);
64                  gotoxy(center_x + 1, center_y);
65                  printf("%c", 47);
66                  gotoxy(center_x + 2, center_y - 1);
67                  printf("%c", 47);
68
69                  break;
70          }
71
72          resetbgcolor();
73  }
74
75
76
77
78
79
80
81
82
83  // Funktion til at tegne en firkant
84  void drawwindow(uint8_t c) {
85          const int width = 16;  // Fast bredde
86          const int height = 8;  // Fast h jde
87
88          fgcolor(c);  // S t farven
89
90          //   verste   kant
```

```
91          gotoxy(powerups_array[0].x, powerups_array[0].y);
92          for (int i = 0; i < width; i++) {
93                  printf("%c", 205);  // Tegn horisontal linje (ASCII:
                       205 =    )
94          }
95
96          // Sider og indhold
97          for (int i = 1; i < height - 1; i++) {
98                  gotoxy(powerups_array[0].x, powerups_array[0].y + i);
                         // Venstre side
99                  printf("%c", 186);         // Vertikal linje (ASCII:
                       186 =    )
100
101                 // Flyt til h jre kant
102                 gotoxy(powerups_array[0].x + width - 1,
                       powerups_array[0].y + i);
103                 printf("%c", 186);         // Vertikal linje (ASCII:
                       186 =    )
104         }
105
106         // Nederste kant
107         gotoxy(powerups_array[0].x, powerups_array[0].y + height - 1)
               ;
108         for (int i = 0; i < width; i++) {
109                 printf("%c", 205);  // Tegn horisontal linje (ASCII:
                       205 =    )
110         }
111
112         // Tegn hj rnerne
113         gotoxy(powerups_array[0].x, powerups_array[0].y);
                                  //  verste  venstre hj rne
114         printf("%c", 201);                          // ASCII: 201 =


115
116         gotoxy(powerups_array[0].x + width - 1, powerups_array[0].y);
                        //  verste  h jre hj rne
117         printf("%c", 187);                          // ASCII: 187 =


118
119         gotoxy(powerups_array[0].x, powerups_array[0].y + height - 1)
               ;          // Nederste venstre hj rne
120         printf("%c", 200);                          // ASCII: 200 =


121
122         gotoxy(powerups_array[0].x + width - 1, powerups_array[0].y +
               height - 1); // Nederste h jre hj rne
```

```
123         printf("%c", 188);                                  // ASCII: 188 =

124
125         resetbgcolor();
126 }
```

Liste 46: Powerups.c

```
1
2 #ifndef GRAPHICS_SPACESHIP_H_
3 #define GRAPHICS_SPACESHIP_H_
4
5 #include <stdint.h>
6 #include <stdio.h>
7
8 #include <graphics/ansi.h>
9 #include <features/movement.h>
10
11 void draw_spaceship(position pos, uint8_t buffer);
12 void draw_spaceship_right(position pos);
13 void draw_spaceship_left(position pos);
14 void draw_spaceship_up(position pos);
15 void draw_spaceship_down(position pos);
16
17 #endif /* SPACESHIP_H_ */
```

Liste 47: Spaceship.h

```
1
2 #include <graphics/spaceship.h> // Include the corresponding header
     for baggrund
3 #include <init/basic_functions.h>
4
5 void draw_spaceship(position pos, uint8_t buffer) {
6         if (buffer) {
7                 switch (pos.direction) {
8                 case 1:
9                 case 2:
10                case 3:
11                        draw_spaceship_right(pos);
12                        break;
13                case 4:
14                        draw_spaceship_down(pos);
15                        break;
16                case 5:
17                case 6:
18                case 7:
```

```
19                          draw_spaceship_left(pos);
20                          break;
21                  case 8:
22                          draw_spaceship_up(pos);
23                          break;
24                  }
25          }
26  }
27
28  void draw_spaceship_left(position pos) {
29          fgcolor(COLOR_OF_SPACESHIP);
30
31          // 1. linje
32          gotoxy(pos.x + 4, pos.y);
33          printf("%c", 95);  //print _
34          printf("%c", 95);  //print _
35
36          // 2. linje
37          gotoxy(pos.x + 1, pos.y + 1);
38          printf("%c", 95);  //print _
39          printf("%c", 95);  //print _
40          printf("%c", 47);  //
41          gotoxy(pos.x + 5, pos.y + 1);
42          printf("%c", 47);  //
43
44          //3. linje
45          gotoxy(pos.x, pos.y + 2);
46          printf("%c", 60);  //print _
47          printf("%c", 95);  //print _
48          printf("%c", 95);  //print _
49          printf("%c", 95);  //print _
50          printf("%c", 61);  //
51          printf("%c", 125);  //
52
53          // 4. linje
54          gotoxy(pos.x + 3, pos.y + 3);
55          printf("%c", 92);
56          printf("%c", 95);
57          printf("%c", 92);
58
59          resetbgcolor();
60  }
61
62  void draw_spaceship_right(position pos) {
63          fgcolor(COLOR_OF_SPACESHIP);
64
```

```
65          // 1. linje
66          gotoxy(pos.x, pos.y);
67          printf("%c", 95);
68          printf("%c", 95);
69
70          // 2. linje
71          gotoxy(pos.x, pos.y + 1);
72          printf("%c", 92);
73          gotoxy(pos.x + 2, pos.y + 1);
74          printf("%c", 92);
75          printf("%c", 95);
76          printf("%c", 95);
77
78          // 3. linje
79          gotoxy(pos.x, pos.y + 2);
80          printf("%c", 123);
81          printf("%c", 61);
82          printf("%c", 95);
83          printf("%c", 95);
84          printf("%c", 95);
85          printf("%c", 62);
86
87          // 4. linje
88          gotoxy(pos.x, pos.y + 3);
89          printf("%c", 47);
90          printf("%c", 95);
91          printf("%c", 47);
92
93          resetbgcolor();
94  }
95
96  void draw_spaceship_up(position pos) {
97          fgcolor(COLOR_OF_SPACESHIP); // Set the foreground color
98
99          gotoxy(pos.x + 2, pos.y);
100         printf("%c", 47); // Print '/' (ASCII 47)
101         printf("%c", 124); // Print '|' (ASCII 124)
102         printf("%c", 92); // Print '\' (ASCII 92)
103
104         gotoxy(pos.x + 1, pos.y + 1);
105         printf("%c", 95); // Print '_' (ASCII 95)
106         printf("%c", 124); // Print '|' (ASCII 124)
107         printf("%c", 124); // Print '|' (ASCII 124)
108         printf("%c", 124); // Print '|' (ASCII 124)
109         printf("%c", 95); // Print '_' (ASCII 95)
110
```

```
111         gotoxy(pos.x, pos.y + 2);
112         printf("%c", 47); // Print '/' (ASCII 47)
113         printf("%c", 95); // Print '_' (ASCII 95)
114         printf("%c", 35); // Print '#' (ASCII 35)
115         printf("%c", 61); // Print '=' (ASCII 61)
116         printf("%c", 35); // Print '#' (ASCII 35)
117         printf("%c", 95); // Print '_' (ASCII 95)
118         printf("%c", 92); // Print '\' (ASCII 92)
119 
120         resetbgcolor(); // Reset colors to default
121 }
122 
123 void draw_spaceship_down(position pos) {
124         fgcolor(COLOR_OF_SPACESHIP); // Set the foreground color
125 
126         gotoxy(pos.x + 1, pos.y);
127         printf("%c", 95); // Print '_' (ASCII 95)
128         gotoxy(pos.x + 5, pos.y);
129         printf("%c", 95); // Print '_' (ASCII 95)
130 
131         gotoxy(pos.x, pos.y + 1);
132         printf("%c", 92); // Print '/' (ASCII 47)
133         printf("%c", 95); // Print '_' (ASCII 95)
134         printf("%c", 35); // Print '#' (ASCII 35)
135         printf("%c", 61); // Print '=' (ASCII 61)
136         printf("%c", 35); // Print '#' (ASCII 35)
137         printf("%c", 95); // Print '_' (ASCII 95)
138         printf("%c", 47); // Print '\' (ASCII 92)
139 
140         gotoxy(pos.x + 2, pos.y + 2);
141         printf("%c", 124); // Print '|' (ASCII 124)
142         printf("%c", 124); // Print '|' (ASCII 124)
143         printf("%c", 124); // Print '|' (ASCII 124)
144 
145         gotoxy(pos.x + 2, pos.y + 3);
146         printf("%c", 92); // Print '/' (ASCII 47)
147         printf("%c", 124); // Print '|' (ASCII 124)
148         printf("%c", 47); // Print '\' (ASCII 92)
149 
150         resetbgcolor(); // Reset colors to default
151 }
```

Liste 48: Spaceship.c

```
1 #ifndef FEATURES_SPAWN_POWERUPS_H_
2 #define FEATURES_SPAWN_POWERUPS_H_
3 
```

```
4  #include <parameters.h>
5  #include <graphics/powerup.h>
6  #include <graphics/enemy.h>
7
8  void power(void);
9  void spawn_powerups(bullet_type *typ, uint32_t powerup_time,
10                 uint32_t *last_powerup_time);
11 void remove_powerups(void);
12
13 #endif /* FEATURES_SPAWN_POWERUPS_H_ */
```

Liste 49: Spawn powerups.h

```
1
2
3  #include <features/spawn_powerups.h>
4
5  void spawn_powerups(bullet_type *typ, uint32_t powerup_time,
6                  uint32_t *last_powerup_time) {
7          if (((cs - powerup_time) > 1000) && (typ->type)) {
8                  typ->type = 0; // Reset bullet type
9          }
10
11         if (!powerups_array[0].active) {
12                 if ((cs + 16) % 2000 == 0) {
13                         powerups_array[0].x = rand() % 243 >> 1;
14                         powerups_array[0].y = rand() % 76 >> 1;
15                         powerups_array[0].active = 1;
16                         powerups_array[0].powerup_effect = rand() %
                               3;
17                         powerups_array[0].x += 243 >> 2;
18                         powerups_array[0].y += 76 >> 2;
19                         powerup(11);
20                 }
21                 *last_powerup_time = cs; // Update last fire time
22         }
23 }
24
25 void remove_powerups(void) {
26         powerups_array[0].active = 0;
27
28         for (int i = 0; i < 8; i++) {
29                 gotoxy(powerups_array[0].x, powerups_array[0].y + i);
30                 printf("                ");
31         }
32
33         powerups_array[0].x = 1000;
```

```
34          powerups_array[0].y = 1000;
35  }
```

Liste 50: Spawn powerups.c

```
1
2   #ifndef INIT_TIMER_H_
3   #define INIT_TIMER_H_
4
5   #include <stm32f30x_conf.h> // STM32 config
6   #include <30010_io.h> // Input/output library for this course
7   #include <init/init_GPIO.h>
8
9   volatile uint32_t cs;
10
11  void timer15(void);
12  void TIM1_BRK_TIM15_IRQHandler(void);
13  void reset_timer(void);
14  void running(uint8_t toggle);
15
16  #endif
```

Liste 51: Timer.h

```
1
2   #include <init/timer.h>
3
4   void timer15(void) {
5           RCC->APB2ENR |= RCC_APB2Periph_TIM15; // Enable clock line to
                timer 15;
6           TIM15->CR1 = 0x0000; // Configure timer 15
7           TIM15->ARR = 0x027F; // Set reload value
8           TIM15->PSC = 0x03E7; // Set prescale value //0x03E7
9           TIM15->CR1 = 0x0001; // Start timer 15
10
11          TIM15->DIER |= 0x0001;
12
13          NVIC_SetPriority(TIM1_BRK_TIM15_IRQn, 0x1); // Set interrupt
               priority
14          NVIC_EnableIRQ(TIM1_BRK_TIM15_IRQn); // Enable interrupt
15  }
16
17  void TIM1_BRK_TIM15_IRQHandler(void) {
18          cs++;
19          TIM15->SR &= ~0x0001; // Clear interrupt bit
20  }
21
```

```
22 void reset_timer(void) { // resets the timer to 0
23         cs = 0;
24         TIM15->CR1 = 0x0000;
25 }
26
27 void running(uint8_t toggle) {
28         if (toggle == 0) {
29                 TIM15->CR1 |= TIM_CR1_CEN; // Start
30         } else {
31                 TIM15->CR1 &= ~TIM_CR1_CEN; // Stop
32         }
33 }
```

Liste 52: Timer.c

```
1
2 #ifndef GRAPHICS_TITLE_H_
3 #define GRAPHICS_TITLE_H_
4
5 #include <graphics/title.h>
6 #include <stm32f30x_conf.h> // STM32 config
7 #include <30010_io.h> // Input/output library for this course
8 #include <init/basic_functions.h>
9
10 void title_header(uint8_t x, uint8_t y);
11 void start_title(uint8_t x, uint8_t y);
12 void help_title(uint8_t x, uint8_t y);
13 void gameover_title(uint8_t x, uint8_t y);
14 void gameover_text(uint8_t x, uint8_t y);
15
16 #endif /* GRAPHICS_TITLE_H_ */
```

Liste 53: Title.h

```
1
2 #include <graphics/title.h>
3 #include <init/basic_functions.h>
4
5
6 void title_header(uint8_t x, uint8_t y) {
7     gotoxy(x, y);
8     printf("       SSSSSSSSSSSSSS TTTTTTTTTTTTTTTTTTTTT
       AAA               RRRRRRRRRRRRRRR
       FFFFFFFFFFFFFFFFFFFFFIIIIIIIIII        GGGGGGGGGGGGHHHHHHHHH

       HHHHHHHHHTTTTTTTTTTTTTTTTTTTTTEEEEEEEEEEEEEEEEEEEERRRRRRRRRRRRRRRR
          \n");
```

```
9     gotoxy(x, y + 1);
10    printf("     SS:::::::::::::::ST::::::::::::::::::::T         A
         :::A              R:::::::::::::::R  F:::::::::::::::::::FI
         ::::::::I     GGG::::::::::::GH:::::::H     H:::::::HT
         ::::::::::::::::::::TE::::::::::::::::::::ER::::::::::::::R
           \n");
11    gotoxy(x, y + 2);
12    printf("    S:::::SSSSSS::::::ST::::::::::::::::::::T        A
         :::::A             R::::::RRRRRR:::::R F:::::::::::::::::::FI
         ::::::::I   GG::::::::::::::GH:::::::H     H:::::::HT
         ::::::::::::::::::::TE::::::::::::::::::::ER::::::RRRRRR:::::
         R \n");
13    gotoxy(x, y + 3);
14    printf("    S:::::S     SSSSSSST:::::TT:::::::TT:::::T      A
         :::::::A            RR:::::R     R:::::RFF::::::FFFFFFFFF::::
         FII::::::II  G:::::GGGGGGGG::::GHH::::::H     H::::::HHT:::::
         TT:::::::TT:::::TEE::::::EEEEEEEEE::::ERR:::::R     R:::::R\n"
         );
15    gotoxy(x, y + 4);
16    printf("    S:::::S            TTTTTT  T:::::T  TTTTTT     A
         ::::::::::A             R::::R     R:::::R  F:::::F
         FFFFFF  I::::I   G:::::G       GGGGGG  H:::::H     H:::::H
         TTTTTT  T:::::T  TTTTTT  E:::::E       EEEEEE  R::::R     R
         :::::R\n");
17    gotoxy(x, y + 5);
18    printf("    S:::::S                      T:::::T            A:::::A
         :::::A            R::::R     R:::::R  F:::::F               I
         ::::I  G:::::G                H:::::H     H:::::H          T
         :::::T          E:::::E               R::::R     R:::::R\n");
19    gotoxy(x, y + 6);
20    printf("     S::::SSSS                   T:::::T           A:::::A
         A:::::A           R::::RRRRRR:::::R   F::::::FFFFFFFFFF      I
         ::::I  G:::::G               H::::::HHHHH::::::H          T
         :::::T          E::::::EEEEEEEEEE     R::::RRRRRR:::::R \n");
21    gotoxy(x, y + 7);
22    printf("      SS::::::SSSSS              T:::::T          A:::::A
          A:::::A          R::::::::::::RR    F::::::::::::::F     I
         ::::I  G:::::G    GGGGGGGGGG  H::::::::::::::::H          T
         :::::T          E::::::::::::::E     R::::::::::::RR  \n");
23    gotoxy(x, y + 8);
24    printf("        SSS::::::::SS           T:::::T         A:::::A
           A:::::A         R::::RRRRRR:::::R   F::::::::::::::F
         I::::I  G:::::G    G::::::::G  H::::::::::::::::H          T
         :::::T          E::::::::::::::E     R::::RRRRRR:::::R \n");
25    gotoxy(x, y + 9);
26    printf("           SSSSSS::::S         T:::::T        A:::::
```

```
        AAAAAAAAA:::::A         R::::R     R:::::R  F::::::FFFFFFFFFF
            I::::I  G:::::G     GGGGG::::G  H::::::HHHHH::::::H
                 T:::::T           E::::::EEEEEEEEEE     R::::R     R
        :::::R\n");
27      gotoxy(x, y + 10);
28      printf("                  S:::::S         T:::::T       A
        ::::::::::::::::::::::A        R::::R     R:::::R  F:::::F
                       I::::I  G:::::G         G::::G  H:::::H     H
        :::::H           T:::::T           E:::::E               R::::R
            R:::::R\n");
29      gotoxy(x, y + 11);
30      printf("                  S:::::S         T:::::T      A:::::
        AAAAAAAAAAAAA:::::A       R::::R     R:::::R  F:::::F
                       I::::I   G:::::G         G::::G  H:::::H     H
        :::::H           T:::::T           E:::::E       EEEEEE  R::::R
            R:::::R\n");
31      gotoxy(x, y + 12);
32      printf("    SSSSSSS     S:::::S       TT:::::::TT    A:::::A
                      A:::::A    RR:::::R     R:::::RFF:::::::FF
                    II::::::II  G:::::GGGGGGGG::::GHH::::::H     H:::::::
        HH       TT:::::::TT       EE::::::EEEEEEEE:::::ERR:::::R     R
        :::::R\n");
33      gotoxy(x, y + 13);
34      printf("    S::::::SSSSSS:::::S       T:::::::::T  A:::::A
                       A:::::A  R::::::R     R:::::RF::::::::FF
                    I::::::::I   GG:::::::::::::::GH:::::::H     H
        :::::::H       T:::::::::T      E::::::::::::::::::::ER::::::R
            R:::::R\n");
35      gotoxy(x, y + 14);
36      printf("    S::::::::::::::SS       T:::::::::T A:::::A
                         A:::::A R::::::R     R:::::RF::::::::FF
                    I::::::::I     GGG::::::GGG:::GH:::::::H     H
        :::::::H       T:::::::::T      E::::::::::::::::::::ER::::::R
            R:::::R\n");
37      gotoxy(x, y + 15);
38      printf("     SSSSSSSSSSSSSS          TTTTTTTTTTTAAAAAAA
                           AAAAAAARRRRRRR      RRRRRRRFFFFFFFFFF
                    IIIIIIIII         GGGGGG    GGGGHHHHHHHHH
        HHHHHHHHH       TTTTTTTTTT       EEEEEEEEEEEEEEEEEEEERRRRRRR
             RRRRRR\n");
39  }
40
41
42  void start_title(uint8_t x, uint8_t y) {
43
44          gotoxy(x,y);
```

```
45      printf("  _________ __                  __    \n");
46          gotoxy(x,y+1);
47      printf(" /   _____//  |______ ________/  |_ \n");
48          gotoxy(x,y+2);
49      printf(" \\_____  \\\\   __\\__  \\\\_  __ \\   __\\\\n");
50          gotoxy(x,y+3);
51      printf(" /        \\|  |  / __ \\|  | \\/|  |  \n");
52          gotoxy(x,y+4);
53      printf("/_______  /|__| (____  /__|   |__|  \n");
54          gotoxy(x,y+5);
55      printf("        \\/           \\/              \n");
56  }
57
58  void help_title(uint8_t x, uint8_t y) {
59      gotoxy(x, y);
60      printf("   ___ ___          .__           \n");
61      gotoxy(x, y + 1);
62      printf("  |   |  |   ____  |  |  ______  \n");
63      gotoxy(x, y + 2);
64      printf("  |    ~ | _/ __ \\ |  |  \\____ \\ \n");
65      gotoxy(x, y + 3);
66      printf("  |    Y | \\  ___/ |  |__|  |_> >\n");
67      gotoxy(x, y + 4);
68      printf("  |___|_ |  \\___  >|____/|   __/ \n");
69      gotoxy(x, y + 5);
70      printf("       \\/       \\/       |__|    \n");
71  }
72
73  void gameover_title(uint8_t x, uint8_t y) {
74  gotoxy(x, y);
75  printf("          _____          _____        ______  _______
       ______                 _____     ____      ____      ______
              _____   \n");
76  gotoxy(x, y + 1);
77  printf("      ___|\\    \\\\     ___|\\    \\\\      |      \\\\/       \\\\
        ___|\\     \\\\           ____|\\    \\\\   |    |    |    | ___|\\
          \\\\   ___|\\    \\\\  \n");
78  gotoxy(x, y + 2);
79  printf("     /    /\\\\    \\\\   /    /\\\\    \\\\    /          /\\\\     \\\\
       |     \\\\     \\\\         /     /\\\\    \\\\  |    |    |    ||     \\\\
           \\\\ |    |\\\\    \\\\ \n");
80  gotoxy(x, y + 3);
81  printf("    |    |  |____| |    |  |    |  /     /\\\\   / /\\\\     ||
          ,_____/|       /     /  \\\\    \\\\ |    |    |    ||     ,_____
      /||    | |    |\n");
82  gotoxy(x, y + 4);
```

```
83  printf("    |    |    ____ |    |__|    | /     /\\ \\_/ / /    /||
        \\--'\\_|/      |     |    |    ||    |    |    ||     \\--'\\
    _|/|    |/____/ \n");
84  gotoxy(x, y + 5);
85  printf("    |    |   |    ||    .--.    ||     |  \\\|_|/ /    / ||
        /___/|        |     |    |    ||    |    |    ||     /___/|  |
        |\\    \\ \n");
86  gotoxy(x, y + 6);
87  printf("    |    |   |_,  ||    |  |    ||     |       |    |  ||
        \\____|\\       |\\     \\  /    /||\\    \\  /    /||     \\
    ____|\\ |    | |    |\n");
88  gotoxy(x, y + 7);
89  printf("    |\\ ___\\___/  /||____|  |____||\\____\\       |____|  /|
    ____ '     /|      | \\_____\\/____/ || \\ ___\\/___ / ||____ '
        /||____| |____|\n");
90  gotoxy(x, y + 8);
91  printf("    | |   /____ / ||    |  |    || |    |      |    | / |
    /_____/ |       \\ |    ||    | / \\ |   ||   | / |    /_____/ ||
        | |    |\n");
92  gotoxy(x, y + 9);
93  printf("     \\|___|    | / |____|  |____| \\|____|      |____|/  |
    ____|     | /        \\|____||____|/   \\|___||___|/  |____|     |
     /|____| |____|\n");
94  gotoxy(x, y + 10);
95  printf("       \\( |____|/    \\(      )/      \\(          )/
    \\( |_____|/          \\(    )/        \\(    )/      \\( |_____
    |/   \\(     )/  \n");
96  gotoxy(x, y + 11);
97  printf("        '   )/       '      '       '          '         '
       )/              '    '          '    '        '    )/       '
         '   \n");
98  gotoxy(x, y + 12);
99  printf("            '
                                                    '
                                                     '
                   \n");
100 }
101
102
103 void gameover_text(uint8_t x, uint8_t y) {
104     gotoxy(x, y);
105     printf("     ______                                  --
                  --    --                      --
                                          --    --
                                  \n");
106     gotoxy(x, y + 1);
```

```
107     printf("     |    __ \\.----..------..------..------.     |
           |--..--.--.|  |_ |  |_ .------..------.     |  |_ .------.
           .----..------..------.|  |_ |__|.------..--.--..------.     \n");
108     gotoxy(x, y + 2);
109     printf("     |     __/|    _||  -__||__ --||__ --|     |  _  ||  |
           ||   _||   _||  _  ||      |     |   _||  _  |     |  __||  _  ||
                ||   _||  ||      ||  |  ||  -__| __ \n");
110     gotoxy(x, y + 3);
111     printf("     |___|    |__|  |_____||_____||_____|     |_____||_____
           ||____||____||_____||__|__|     |____||_____|     |____||_____||
           __|__||____||__||__|__||_____||_____||__|\n");
112 }
```

Liste 54: Title.c

11 Appendix C - Exercise journal

11.1 Kontrol over skærmen (Exercise. 1)

Efter vi havde downloadet STM-32 og PuTTY og satte dem op, lavede vi en `ansi.c` og en `ansi.h` fil, som vi puttede ind under mapperne, henholdsvis `src` og `inc`. Herefter begyndte vi at fikse `GetOutput` og `GetFeedback`-funktionerne til:

Herefter kunne vi printe den rigtige PRN-kode til terminalen.

Vi har lavet funktioner, der kan:

- Rydde hele skærmen (`clrscr`)
- Rydde kun linjen (`clreol`)
- Flytte markøren (`gotoxy`)
- Tilføje underline (`underline`)
- Få teksten til at blinke (`blink`)
- Inverse farverne (`inverse`)

Disse gør det nemmere for os at arrangere tingene på skærmen.

```
1  void clrscr(void){
2          printf("%c[2J",ESC);
3  }
4
5
6  void clreol(void){
7          printf("%c[K",ESC);
8  }
9
10
11 void gotoxy(uint16_t x,uint16_t y){
12         printf("%c[%d;%d%c",ESC,y,x,'H');
13 }
14
15 void underline(uint8_t x){
16
17         if (x==1){
18                 printf("%c[4m",ESC);
19         }
20         else {
21                 printf("%c[24m",ESC);
22         }
23 }
24
```

```
25 void blink(uint8_t x){
26         if (x==1){
27                 printf("%c[5m",ESC);
28         }
29         else {
30                 printf("%c[25m",ESC);
31         }
32 }
33
34 void inverse(uint8_t x){
35         if (x==1){
36                 printf("%c[7m",ESC);
37         }
38         else {
39                 printf("%c[27m",ESC);
40         }
41 }
```

11.2 Vinduer med titler(Exercise.2)

Vi har også lavet en funktion, der kan tegne firkantede vinduer i terminalen. Disse vinduer kan have en titel og forskellige designs. Det betyder, at vi kan lave en slags brugerflade med små bokse, hvor vi kan vise information.

```
1
2 void window(windowsize win, const char *title, int style){
3         if (win.x1 == 0) {
4                 win.x1 = 1;
5         }
6         if (win.y1 == 0) {
7                 win.y1 = 1;
8         }
9         //Top line
10        if (style == 0){
11                gotoxy(win.x1,win.y1);
12                printf("%c",Lefttopcorner1);
13                printf("%c",Lefttitleholder1);
14                inverse(1);
15                for (int i = win.x1; i < win.x2 - 3; i++){
16                        printf(" ");
17                }
18                gotoxy(win.x1 + 3,win.y1);
19                printf(title);
20                inverse(0);
21                gotoxy(win.x2 - 1,win.y1);
22                printf("%c",Righttitleholder1);
```

```
23                  printf("%c",Righttopcorner1);
24
25                  //Sides
26
27                  for (int i = win.y1 + 1; i < win.y2; i++){
28                          //Left side
29                          gotoxy(win.x1,i);
30                          printf("%c",Vertical1);
31                          //Right side
32                          gotoxy(win.x2, i);
33                          printf("%c",Vertical1);
34                  }
35
36
37                  //Bottom line
38                  gotoxy(win.x1,win.y2);
39                  printf("%c",Leftbottomcorner1);
40                  for (int i = win.x1 + 1; i < win.x2; i++){
41                          printf("%c",Horizontal1);
42                  }
43                  printf("%c",Rightbottomcorner1);
44          }
```

11.3 Trigonometri med Look-Up Tables (LUT)(Exercise.3)

Vi har skabt funktioner som `sinus` og `cos`, der anvender beregnede værdier i en Look-Up Table for hurtigt at beregne sinus- og cosinusværdier. Ved at bruge LUT'er har vi elimineret behovet for tunge beregninger og sikret højere ydeevne på systemer med begrænset hardwarekapacitet.

```
1  uint32_t Sinus(int16_t x){
2          return SIN[x];
3  }
4
5  uint32_t Sinfunc(int x){
6          while (x < 0){
7          x +=512; //If angle is negative
8          }
9
10         while (x > 512){
11         x -= 512; // If angle is more than 360 degrees
12         }
13
14         return (Sinus(x));//printFix(expand(x)));
15 }
16
```

```
uint32_t Cosfunc(int16_t x){
        x += 128;
        while (x < 0){
                x +=512; //If angle is negative
        }

        while (x > 512){
                x -= 512; // If angle is more than 360 degrees
        }

        return (Sinus(x));//printFix(expand(x)));
}
```

11.3.1 Vektorfunktioner

Vi har implementeret funktioner som `rotateVector`, der roterer 2D-vektorer ved hjælp af trigonometri fra LUT'erne. Funktionen opdaterer koordinaterne baseret på rotationsvinklen og anvender faste punktberegninger. Dette har givet os mulighed for effektivt at manipulere vektorer og demonstrere brugen af avancerede matematiske operationer.

```
void initVector(vector_t *v) {
        v->x = 1;
        v->y = 1;
}

void rotateVector(vector_t *v, vector_t tempVect, int z) {

        int32_t x = v -> x;
        int32_t y = v -> y;
        v->x = (tempVect.x * Cosfunc(z)) - (tempVect.y * Sinfunc(z));
        v->y = (tempVect.x * Sinfunc(z)) + (tempVect.y * Cosfunc(z));
}
```

11.4 Ball in window (Exercise. 4)

Projektet blev indledt med etableringen af et vindue i terminalen, som blev genbrugt fra en tidligere opgave. Boldens bevægelse blev implementeret ved at opdatere dens vektor. Når boldens x-position ramte en af vinduets sider, blev vektorens x- eller y-komponent inverteret, hvilket simulerede en refleksion og tillod bolden at "bounce"tilbage fra væggen. Derudover blev en mindre, indre kasse initialiseret på lignende vis. For at overvåge interaktioner med denne kasse blev der implementeret en tællefunktion, som registrerede hver gang boldens bevægelsesvektor blev ændret. For at sikre, at bolden ignorerede den indre kasse, blev der indført en betinget if-else-struktur. Denne struktur kontrollerede, om bolden befandt sig inden for den indre kasses område. Hvis dette var tilfældet, blev boldens position ikke printet. Hvis bolden derimod var uden for kassen, blev dens position vist som normalt.

```
1  void ball(windowsize win, int x, int y, int vx, int vy) {
2  // init ball speed
3          vector_t myvector;
4          initVector(&myvector);
5          myvector.x = vx;
6          myvector.y = vy;
7
8          int counter = 0;
9
10         while (counter < 1000) {
11                 if (counter < 10) {
12                         gotoxy(win.middlex + 3, win.middley);
13                         printf("%d", counter);
14                 }else{
15                         gotoxy(win.middlex + 2,win.middley);
16                         printf("%d",counter);
17                         }
18                 if ((x<=win.x2) && (x> win.x1) && (y <= win.y2) && (y
                       > win.y1)); { //Inside the window
19                         if (!((win.middlex - 6 < x) && (x < win.
                              middlex + 6) && (win.middley - 3 < y) && (
                              y < win.middley + 3))) { //Not in the
                              hitbox
20                                 gotoxy(x,y);
21                                 printf(" "); //Removes the ball
22                         }
23                 }
24
25                 //Calculate position
26                 x += myvector.x;
27                 y += myvector.y;
28
29                 if (y >= win.y2 - 1) {
30                         myvector.y = -myvector.y;
31                         counter += 1;
32                 }
33                 if (y <=win.y1 + 1) {
34                         myvector.y = -myvector.y;
35                         counter += 1;
36                 }
37                 if (x >= win.x2 - 1) {
38                         myvector.x = -myvector.x;
39                         counter +=1;
40                 }
41                 if (x <= win.x1 + 1) {
42                         myvector.x = -myvector.x;
```

```
43                          counter +=1;
44                  }
45
46                  //print o
47                  if ((x<=win.x2) && (x> win.x1) && (y <= win.y2) && (y
                        > win.y1)); { //Inside the window
48                          if (!((win.middlex - 6 < x) && (x < win.
                                middlex + 6) && (win.middley - 3 < y) && (
                                y < win.middley + 3))) { //Not in the
                                hitbox
49                                  gotoxy(x,y);
50                                  printf("o"); // prints the ball
51                          }
52                  }
53          }
54  }
```

11.5 GPIO Joystick(Exercise.5)

Vi lavede også Exercise 5, hvor vi fokuserede på at bruge GPIO til at detektere joystick-interaktioner og styre en RGB LED. Vi fik konfigureret pins til at læse joystickets tilstand og implementerede en funktion, der aflæste retningerne. Derefter skrev vi en funktion, der ændrede LED'ens farve afhængigt af joystickets bevægelser. Til sidst testede vi koden og sikrede, at både aflæsning og farveskift fungerede korrekt. Vi nåede ikke videre til Exercise 6, men planen er at starte på den næste gang. Det var en produktiv dag med godt overblik over opgaven.

```
1
2   #include "../Inc/Exercises/Exercise5.h"
3   #include "stm32f30x.h" // Include the STM32 device-specific headers
4
5
6   void configure_pins(void) {
7       // Enable clock for GPIO Port A
8       RCC->AHBENR |= RCC_AHBENR_GPIOAEN;
9
10      // Set pin PA0 to input
11      GPIOA->MODER &= ~(0x3 << (0 * 2)); // Clear mode register for PA0
12      GPIOA->PUPDR &= ~(0x3 << (0 * 2)); // Clear pull-up/pull-down
            register for PA0
13      GPIOA->PUPDR |= (0x2 << (0 * 2));  // Set pull-down for PA0
14  }
15
16  void configure_joystick(void) {
17      // Enable clocks for GPIO ports
```

```
18      RCC->AHBENR |= RCC_AHBENR_GPIOAEN | RCC_AHBENR_GPIOBEN |
            RCC_AHBENR_GPIOCEN;
19
20      // Set joystick pins to input mode
21      GPIOC->MODER &= ~(0x3 << (0 * 2)); // PC0 (Right)
22      GPIOA->MODER &= ~(0x3 << (4 * 2)); // PA4 (Up)
23      GPIOB->MODER &= ~(0x3 << (5 * 2)); // PB5 (Center)
24      GPIOC->MODER &= ~(0x3 << (1 * 2)); // PC1 (Left)
25      GPIOB->MODER &= ~(0x3 << (0 * 2)); // PB0 (Down)
26
27      // Configure pull-down resistors for joystick pins
28      GPIOC->PUPDR |= (0x2 << (0 * 2)); // PC0
29      GPIOA->PUPDR |= (0x2 << (4 * 2)); // PA4
30      GPIOB->PUPDR |= (0x2 << (5 * 2)); // PB5
31      GPIOC->PUPDR |= (0x2 << (1 * 2)); // PC1
32      GPIOB->PUPDR |= (0x2 << (0 * 2)); // PB0
33  }
34
35  uint8_t readJoystick(void) {
36      uint8_t state = 0;
37
38      // Read joystick inputs and set bits accordingly
39      state |= (GPIOA->IDR & (1 << 4)) ? (1 << 0) : 0; // Up
40      state |= (GPIOB->IDR & (1 << 0)) ? (1 << 1) : 0; // Down
41      state |= (GPIOC->IDR & (1 << 1)) ? (1 << 2) : 0; // Left
42      state |= (GPIOC->IDR & (1 << 0)) ? (1 << 3) : 0; // Right
43      state |= (GPIOB->IDR & (1 << 5)) ? (1 << 4) : 0; // Center
44
45      return state;
46  }
47
48  void configure_led(void) {
49      // Enable clocks for GPIO ports
50      RCC->AHBENR |= RCC_AHBENR_GPIOAEN | RCC_AHBENR_GPIOBEN |
            RCC_AHBENR_GPIOCEN;
51
52      // Configure PB4 (Red)
53      GPIOB->MODER &= ~(0x3 << (4 * 2)); // Clear mode bits
54      GPIOB->MODER |= (0x1 << (4 * 2)); // Set PB4 as output
55
56      // Configure PC7 (Green)
57      GPIOC->MODER &= ~(0x3 << (7 * 2)); // Clear mode bits
58      GPIOC->MODER |= (0x1 << (7 * 2)); // Set PC7 as output
59
60      // Configure PA9 (Blue)
61      GPIOA->MODER &= ~(0x3 << (9 * 2)); // Clear mode bits
```

```
62      GPIOA->MODER |= (0x1 << (9 * 2)); // Set PA9 as output
63  }
64
65  void setLed(uint8_t R, uint8_t G, uint8_t B) {
66      // Turn off all LEDs
67      GPIOB->ODR |= (1 << 4); // Turn off Red
68      GPIOC->ODR |= (1 << 7); // Turn off Green
69      GPIOA->ODR |= (1 << 9); // Turn off Blue
70
71      // Turn on LEDs based on arguments
72      if (R) {
73          GPIOB->ODR &= ~(1 << 4); // Turn on Red
74      }
75      if (G) {
76          GPIOC->ODR &= ~(1 << 7); // Turn on Green
77      }
78      if (B) {
79          GPIOA->ODR &= ~(1 << 9); // Turn on Blue
80      }
81  }
82
83  void writeJoystickToTerminal(uint8_t state) {
84      printf("Joystick State: Up: %d, Down: %d, Left: %d, Right: %d,
           Center: %d\n",
85              (state & (1 << 0)) ? 1 : 0,
86              (state & (1 << 1)) ? 1 : 0,
87              (state & (1 << 2)) ? 1 : 0,
88              (state & (1 << 3)) ? 1 : 0,
89              (state & (1 << 4)) ? 1 : 0);
90  }
91  void JoystickToLED() {
92          uint8_t previousState = 0;
93          int8_t currentState;
94
95          while (1) {
96                  currentState = readJoystick();
97                  // If state has changed, write to terminal and set
                       LED color
98                  if (currentState != previousState) {
99                          writeJoystickToTerminal(currentState);
100             if (currentState == 16){ //Press in (Turn off)
101                 setLed(0,0,0);
102             }
103                         if (currentState == 8) { //right (Red)
104                                 setLed(1,0,0);
105                         }
```

```
106                         if (currentState == 4) { //left (Green)
107                                 setLed(0,1,0);
108                         }
109                         if (currentState == 2) { //down (Blue)
110                                 setLed(0,0,1);
111                         }
112                         if (currentState == 1) { //up - All Colours
                               on(White light)
113                                 setLed(1,1,1);
114                         }
115                         previousState = currentState;
116                 }
117         }
118 }
```